GRAPH PATTERN GUIDE

PAGE 1 — GRAPH FOUNDATIONS

"Think like a FAANG interviewer before writing a single line of code."

1 PLAIN ENGLISH

A Graph is a collection of nodes connected by edges.

Trees are special graphs.

Grids are graphs.

Most interview problems hide graphs.

2 VISUAL EXAMPLES

TREE

⊂ Tree ⊂ Graph

GRAPH

graph TD 1((1)) --- 2((2)) 1((1)) --- 4((4)) 2((2)) --- 5((5)) 4((4)) --- 6((6)) 5((5)) --- 6((6))

May Have Cycles, May Be Disconnected

No Cycles, Connected

GRID (2D)

graph TD 00((0,0)) --- 01((0,1)) 01((0,1)) --- 02((0,2)) 00((0,0)) --- 10((1,0)) 01((0,1)) --- 11((1,1)) 02((0,2)) --- 12((1,2)) 10((1,0)) --- 11((1,1)) 11((1,1)) --- 12((1,2)) 10((1,0)) --- 20((2,0)) 11((1,1)) --- 21((2,1)) 12((1,2)) --- 22((2,2)) 20((2,0)) --- 21((2,1)) 21((2,1)) --- 22((2,2))

Cells as Nodes, Neighbors as Edges

3 BRAIN SWITCH

Interview gives...

- Cities & Roads
- Friends
- Flights
- Courses
- Computer Network
- Maze
- Dependencies
- Social Network

➡️ THINK GRAPH

4 GRAPH TYPES

UNDIRECTED

graph TD 1((1)) --- 2((2)) 1((1)) --- 3((3)) 2((2)) --- 3((3))

Edges have no direction.
Ex: Friends

DIRECTED

graph TD 1((1)) --> 2((2)) 2((2)) --> 3((3)) 1((1)) --> 3((3))

Edges have direction.
Ex: Flights

WEIGHTED

graph TD 1((1)) -- 5 --> 2((2)) 1((1)) -- 2 --> 3((3)) 2((2)) -- 5 --> 3((3))

Edges have weights/costs.
Ex: Roads

UNWEIGHTED

graph TD 1((1)) --- 2((2)) 2((2)) --- 3((3))

All edges have equal weight.
Ex: Social Network

CONNECTED

graph TD 1((1)) --- 2((2)) 1((1)) --- 4((4)) 2((2)) --- 3((3)) 4((4)) --- 3((3))

There is a path between every pair of nodes.
Ex: Network

DISCONNECTED

graph TD 1((1)) --- 2((2)) 3((3)) --- 4((4))

Not all nodes are reachable.
Ex: Components

CYCLIC

graph TD 1((1)) --> 2((2)) 2((2)) --> 3((3)) 3((3)) --> 1((1))

Contains at least one cycle.
Ex: Dependencies

ACYCLIC

graph TD 1((1)) --> 2((2)) 2((2)) --> 3((3))

No cycles.
Ex: Tree

DAG (DIRECTED ACYCLIC GRAPH)

graph TD 1((1)) --> 2((2)) 1((1)) --> 4((4)) 2((2)) --> 3((3)) 4((4)) --> 3((3))

Directed and no cycles.
Ex: Course Schedule

5 GRAPH TERMINOLOGY

graph TD A((A)) -- 1 --> B((B)) A((A)) -- 1 --> C((C)) B((B)) -- 3 --> D((D)) C((C)) -- 2 --> D((D)) C((C)) -- 4 --> E((E)) D((D)) -- 5 --> E((E)) E((E)) --> B((B))

Vertex (Node): The entities (e.g. A, B, C)

Edge: The connections between nodes.

Neighbor: Nodes connected directly by an edge (e.g. A's neighbors are B, C)

Weight: Cost/distance of an edge (e.g. 5)

Degree (A) = 2: Edges connected to A

Path: A → C → E

Cycle: B → C → D → B

Connected Component: A subgraph where any two vertices are connected.

6 TREE vs GRAPH

Aspect	TREE	GENERAL GRAPH
Root	Has a root	No root (usually)
Parent	Each node (except root) has exactly one parent	A node can have multiple parents
Neighbor	Limited by parent-child relationship	Any connected node is a neighbor
Cycles	No cycles	May have cycles
Connected	Always connected	May be connected or disconnected
Edges	For n nodes, edges = n - 1	Edges can be any number
Traversal	DFS / BFS works	DFS / BFS works
Need visited[]	Usually Not Needed	Usually Needed
Interview Examples	Binary Tree, N-ary Tree	Social Network, Roads, Flights, Dependencies

A Tree is always a Graph.

A Graph is NOT always a Tree.

8 AHA MOMENTS

- A Tree is a Graph.
- A Grid is a Graph.
- Representation changes. Algorithm usually doesn't.
- Interview problems rarely say "Graph".
- Always identify the hidden graph first.

7 GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Typical Input	Interview Usage	Memory	Highlight
Edge List	List of edges [u, v, w]	Kruskal, Some Graph Problems	O(E)	Simple to read
Adjacency List ★	[[1,2], [2,3], ...]	Most Common in Interviews	O(V + E)	⭐⭐⭐⭐⭐ **Most commonly used**
Adjacency Matrix	2D Matrix [n x n]	Dense Graphs, Quick Edge Check	O(V^2)	Good for dense graphs
Grid	2D Characters / Integers	Islands, Paths, Flood Fill	O(m x n)	Grid treated as Graph
Node Graph	Nodes with Neighbors	Clone Graph, Complex Structures	O(V + E)	Used in advanced problems

★ **Adjacency List is the most commonly used representation in interviews.**

11 INTERVIEW THINKING

Before writing code ask:

1. **What is the input?**
2. **Is it Directed?**
3. **Is it Weighted?**
4. **Is it Connected?**
5. **Which Representation should I build?**
6. **Which Algorithm family will probably solve it?**

9 PATTERN RECOGNITION

Interview Says	Think	Expected Graph Type
🏙️ Cities / Roads	Graph	Undirected, Often Weighted
✈️ Flights	Graph	Directed, Often Weighted
📚 Courses / Prerequisites	Graph	Directed, DAG
🧱 Grid / Matrix	Graph	Grid Graph (4 / 8 direction)
🖥️ Network (Servers / Computers)	Graph	Undirected or Directed
👥 Friends / Social Network	Graph	Undirected, Unweighted
🌀 Maze / Labyrinth	Graph	Grid Graph
🔗 Dependencies / Build Order	Graph	Directed, DAG

10 COMMON INTERVIEW MISTAKES

- ❌ Forgetting graphs may be disconnected.
- ❌ Confusing Tree with Graph.
- ❌ Ignoring direction (Directed vs Undirected).
- ❌ Ignoring weights.
- ❌ Choosing wrong representation.
- ❌ Thinking Grid is different from Graph.
- ❌ Not defining neighbors clearly.
- ❌ Forgetting to validate with small examples.

12 SUMMARY

REMEMBER

Graph problems are solved in this order.

❓ **Problem →** **Input →** **Representation →** **Helper Structure →** ⚙️ **Algorithm →** **Tiny Logic Change →** ▶️ **Dry Run →** 🚀 **Optimize**

GRAPH PATTERN GUIDE

PAGE 3 — GRAPH HELPER STRUCTURES & INTERVIEW TOOLKIT

"Before choosing an algorithm, choose the helper structures that power it."

PAGE 3

1 PLAIN ENGLISH

Almost every graph problem uses helper structures.

Choosing the right helper structure is often more important than choosing the algorithm.

Build first. Choose helper structures. Then solve.

2 THE GRAPH TOOLBOX (Helper Structures You'll Use in Almost Every Problem)

Structure	Purpose (One-line)	Used In (Typical Problems)	Java Type	Initialization (Example)	Validation / What to Check
★ visited[]	Track visited nodes	DFS, BFS, Cycle Detect, Components	`boolean[]`	`boolean[] visited = new boolean[n];`	Print visited nodes (true indices)
★ dist[]	Store shortest distance from source	Shortest Path, Dijkstra, Multi-Source BFS	`int[]`	`int[] dist = new int[n]; Arrays.fill(dist, INF);`	Print distances (Arrays.toString)
parent[]	Store parent / predecessor of each node	Path Reconstruction, Union Find, Trees	`int[]`	`int[] parent = new int[n]; for (int i=0;i<n;i++) parent[i] = i;`	Print parent array
rank[]	Rank / size for Union Find (Optimized Merge)	Union Find (Disjoint Set)	`int[]`	`int[] rank = new int[n]; (default 0)`	Print rank array
indegree[]	Count incoming edges	Topological Sort (Kahn's Algo)	`int[]`	`int[] indegree = new int[n];`	Print indegree array
dirs[][]	Directions for grid movement	Grid DFS/BFS, Islands, Flood Fill	`int[][]`	`int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};`	Print dirs to verify movement
★ Queue	Process nodes in FIFO order	BFS, Level Order, Kahn's Algo	`Queue<Integer>`	`Queue<Integer> q = new LinkedList<>();`	Print queue content
★ PriorityQueue	Process best candidate first	Dijkstra, Prim's, Minimum Effort	`PriorityQueue<int[]>`	`PriorityQueue<int[]> pq = new PriorityQueue<>();`	Print pq content
HashMap	Mapping / Lookup (Visited, Clone, etc.)	Clone Graph, Store Mappings, Count	`HashMap<K,V>`	`HashMap<K,V> map = new HashMap<>();`	Print map entries
★ List<List<Integer>>	Adjacency List for Unweighted Graph	Most Graph Problems (95%)	`List<List<Integer>>`	`List<List<Integer>> g = new ArrayList<>();`	Print neighbors for each node
List<List<int[]>>	Adjacency List for Weighted Graph	Weighted Graphs, Dijkstra, MST	`List<List<int[]>>`	`List<List<int[]>> g = new ArrayList<>();`	Print (neighbor,weight) for each node

★ Most Frequently Used in Interviews | Adjacency List + visited[] + Queue + PriorityQueue = Your Core Toolkit

3 WHEN DO I NEED WHAT?

Need to avoid revisiting nodes?	→ `visited[]`
Need shortest distance?	→ `dist[]`
Need smallest distance first?	→ `PriorityQueue`
Need BFS (level by level)?	→ `Queue`
Need to merge / union groups?	→ `parent[], rank[]`
Need dependency count?	→ `indegree[]`
Need grid movement?	→ `dirs[][]`
Need clone / mapping / lookup?	→ `HashMap`

4 INITIALIZATION CHEAT SHEET

Structure	Java Initialization
visited[]	`boolean[] visited = new boolean[n];`
dist[]	`int[] dist = new int[n]; Arrays.fill(dist, INF);`
parent[]	`int[] parent = new int[n]; for(int i=0;i<n;i++) parent[i]=i;`
rank[]	`int[] rank = new int[n]; // default 0`
indegree[]	`int[] indegree = new int[n];`
Queue	`Queue<Integer> q = new LinkedList<>();`
PriorityQueue	`PriorityQueue<int[]> pq = new PriorityQueue<>();`
HashMap	`HashMap<K, V> map = new HashMap<>();`
dirs[][]	`int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};`

5 PRINT & VALIDATE (Always Do This!)

Structure	How to Print	Expected Output
visited[]	`Arrays.toString(visited)`	[true, false, true, ...]
dist[]	`Arrays.toString(dist)`	[0, 2, 5, INF, ...]
parent[]	`Arrays.toString(parent)`	[0, 0, 1, 2, ...]
rank[]	`Arrays.toString(rank)`	[0, 1, 0, 1, ...]
indegree[]	`Arrays.toString(indegree)`	[0, 1, 2, 0, ...]
Adjacency List	`for(i) print i → neighbors`	0 → [1,2] 1 → [0,2,3]
Weighted Graph	`for(i) print i → (nbr,w)`	0 → (1,4) (2,7) 1 → (0,4) ...
PriorityQueue	`System.out.println(pq)`	[(2,5), (1,8), ...]
Queue	`System.out.println(q)`	[1, 2, 3, ...]
HashMap	`System.out.println(map)`	{key1=val1, ...}

⚠ Never debug DFS/BFS until helper structures look correct.

6 VISUAL MEMORY MAP (How Everything Connects)

graph LR A(Problem) --> B[Input] B --> C{Representation} C --> H[Helper Structure] H -.-> V[visited] H -.-> D[dist] H -.-> P[parent] H -.-> R[rank] H -.-> I[indegree] H -.-> Q[Queue] H -.-> PQ[PriorityQueue] H -.-> DIRS[dirs] H -.-> M[HashMap] H --> PAT[Pattern DFS/BFS/etc.] PAT --> TLC[Tiny Logic Change]

7 AHA MOMENTS

- Algorithms usually share helper structures.
- Most DFS problems differ only by return type.
- Most BFS problems differ only by queue logic.
- Grid problems always reuse dirs[].
- Weighted graphs almost always need dist[].
- PriorityQueue usually means "best candidate first".
- Union Find is impossible without parent[].
- Topological Sort begins with indegree[].

8 COMMON INTERVIEW MISTAKES

✗ Forgot visited[].

✗ Forgot Arrays.fill(dist, INF).

✗ Mixed node and distance in structures.

✗ Forgot parent[i] = i.

✗ Forgot reverse edge in undirected graph.

✗ Used Queue instead of PriorityQueue.

✗ Forgot dirs[].

✗ Didn't print structures while debugging.

9 INTERVIEW THINKING FLOW (Before Coding)

graph TD P(Problem) ⟶ I[Input] I ⟶ R{Representation} R ⟶ H[Which helper structures do I need?] H ⟶ PAT(Pattern DFS/BFS/Topo/Dijkstra/UF/etc.) PAT ⟶ SOL[Tiny Logic Change] SOL ⟶ DR[Dry Run] DR ⟶ OPT[Optimize]

10 SUMMARY

REMEMBER

✓ Graph interviews are not won by memorizing algorithms.

✓ They are won by choosing the correct helper structures before coding.

✓ Build → Helper Structures → Pattern → Tiny Change → Solve.

✓ Helper structures are your superpower.

Your Toolkit.
Use it Every Time.

★★★★★

GRAPH PATTERN GUIDE

PAGE 2 - GRAPH INPUT TYPES & CONSTRUCTION GUIDE

"Every graph interview begins by converting the input into the right representation."

PAGE 2

1 PLAIN ENGLISH

- Interviewers rarely give you a graph directly.
- Instead they give edges, grids, matrices or node objects.
- Your first job is to identify the input and convert it into the correct graph representation.

2 INPUT TYPES (What Interviewers Give You)

1 EDGE LIST

Example Input

```
[[0,1],[1,2],
[2,3]]
```

graph LR 0((0)) --- 1((1)) 1 --- 2((2)) 2 --- 3((3))

2 WEIGHTED EDGE LIST

Example Input

```
[[2,1,1],
[2,3,1],
[3,4,1]]
```

graph TD 2((2)) -->|1| 1((1)) 2 -->|1| 3((3)) 3 -->|1| 4((4))

3 GRID

Example Input

```
1 1 0 1 0

1 1 1 0 0
```

4 ADJACENCY MATRIX

Example Input

```
0 1 0

1 0 1

0 1 0
```

graph TD 1((1)) --- 0((0)) 1 --- 2((2))

5 NODE GRAPH (OBJECTS)

Example Input

graph TD 1((1)) --- 2((2)) 2 --- 3((3)) 2 --- 4((4))

3 BRAIN SWITCH

Edge List	→ Build AdjList
Weighted Edge List	→ Build Weighted AdjList
Grid	→ Treat Cell as Node
Adjacency Matrix	→ Convert or Traverse Directly
Node Graph	→ Use Existing Neighbors

4 GRAPH CONSTRUCTION GUIDE

Input (Interview Gives)	How to Build	Output (Your Representation)
Edge List `[[u,v], ...]`	Build Adjacency List (Undirected / Directed)	`List<List<Integer>>`
Weighted Edge List `[[u,v,w], ...]`	Build Weighted Adjacency List	`List<List<int[]>>`
Grid `matrix[m][n]`	Use directions (dirs[]) Treat valid cell as node	Implicit Graph
Adjacency Matrix `matrix[n][n]`	Use matrix directly or convert to AdjList	`int[][]` matrix
Node Graph `Node object`	Use existing neighbors directly	Node (given)

5 VISUAL CONVERSION EXAMPLES

1. Edge List → Adjacency List

graph LR 0((0)) --- 1((1)) 1 --- 2((2)) 2 --- 3((3))

2. Weighted Edge → Weighted AdjList

graph TD 0((0)) -->|4| 1((1)) 0 -->|2| 2((2)) 2 -->|5| 1

3. Grid → Implicit Graph

Implicit Graph (4-Directions)

4. Matrix → Graph

graph TD 1((1)) --- 0((0)) 1 --- 2((2))

5. Node Objects → Graph

graph TD 1((1)) --- 2((2)) 2 --- 3((3)) 2 --- 4((4))

GRAPH PATTERN GUIDE

PAGE 1 — GRAPH FOUNDATIONS

"Think like a FAANG interviewer before writing a single line of code."

PAGE 1

1 PLAIN ENGLISH

Graphs model relationships.

Most interview problems hide graphs. Before coding, recognize the hidden graph, understand its type, choose the right representation and the right pattern.

2 VISUAL EXAMPLES

TREE (Special Case of Graph)

graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))

No cycles, Connected, Has a root.

GENERAL GRAPH

graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 3 --> 2 3 --> 5((5))

May have cycles, May be disconnected, No root

GRID (2D)

graph TD 00((0,0)) --- 01((0,1)) 01 --- 02((0,2)) 00 --- 10((1,0)) 01 --- 11((1,1)) 02 --- 12((1,2)) 10 --- 11 11 --- 12 10 --- 20((2,0)) 11 --- 21((2,1)) 12 --- 22((2,2)) 20 --- 21 21 --- 22

Cells are nodes, Neighbors are edges.

3 BRAIN SWITCH

Interview gives...

- Cities & Roads
- Friends / Social Network
- Flights / Airlines
- Courses / Prerequisites
- Computer Network
- Maze / Game Map
- Dependencies / Tasks
- Files / Folders Structure
- Feeds / Connections

→ THINK GRAPH

Whenever you see relationships, think GRAPH.

4 GRAPH TYPES

UNDIRECTED

graph TD 1((1)) --- 2((2)) 1 --- 3((3)) 2 --- 3

Edges have no direction.

DIRECTED

graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 3

Edges have direction.

WEIGHTED

graph TD 1((1)) -- 5 --> 2((2)) 1 -- 7 --> 3((3)) 2 -- 3 --> 3

Edges have weights/costs.

UNWEIGHTED

graph TD 1((1)) --- 2((2)) 1 --- 3((3)) 2 --- 3

All edges have equal weight.

5 GRAPH TERMINOLOGY

graph TD A((A)) -- 1 --> B((B)) A -- 1 --> C((C)) B -- 3 --> D((D)) C -- 2 --> D C -- 5 --> E((E)) D --> E

- **Vertex (Node)**: A, B, C, D, E
- **Edge**: Connection between nodes
- **Degree (of A) = 2**: Edges connected to A
- **Weight**: Cost of an edge
- **Path**: Sequence of vertices (A → C → D → E)
- **Cycle**: Start and end at same node (B → C → D → B)
- **Connected Component**: Set of nodes that are reachable.

6 TREE vs GRAPH

Aspect	TREE	GENERAL GRAPH
Root	Has a root	No root (usually)
Parent	Each node (except root) has exactly one parent	A node can have multiple parents
Neighbor	Limited by parent-child relationship	Any connected node is a neighbor
Cycles	No cycles	May have cycles
Connected	Always connected	May be connected or disconnected
Edges	For n nodes, edges = n - 1	Edges can be any number
Traversal	DFS / BFS works	DFS / BFS works
Need visited[]	Usually Not Needed	**Usually Needed**

A Tree is always a Graph. **A Graph is NOT always a Tree.**

8 AHA MOMENTS

- 💡 **A Tree is a Graph.**
- 💡 **A Grid is a Graph.**
- 💡 **Representation changes. Algorithm usually doesn't.**
- 💡 **Most interview problems hide the graph.**
- 💡 **Outer loop = Multiple Components.**

7 GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Visual	Interview Usage	Memory	Notes
Edge List	List of edges (u, v, w?)	Kruskal, Some Problems	O(E)	Simple to read
Adjacency List	{1: [2, 3], 2: [1, 3], ...}	**Most Common in Interviews**	O(V + E)	**★★★★★ 95% Interview Problems**
Adjacency Matrix	2D Matrix n x n	Dense Graphs, Quick Edge Check	O(V^2)	Good for dense graphs
Grid	2D Characters / Integers	Islands, Paths, Flood Fill	O(m x n)	Grid treated as Graph
Node Graph	Nodes with Neighbors	Clone Graph, Complex Structures	O(V + E)	Used in advanced problems

Adjacency List is the most used representation in interviews.

GRAPH PATTERN GUIDE

MASTER DFS TEMPLATE GUIDE

"One DFS algorithm. Five reusable templates."

1 MASTER IDEA ★★★★★

💡 Almost every Graph DFS interview problem uses one of five reusable templates.

The DFS traversal almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH DFS TEMPLATE ★★★★★

Input	Adjacency List
Caller	dfs(start)
Visited	boolean[]
Return	void / boolean / int
Used In	• Valid Path • Connected Components • Graph Valid Tree

Java Template

```
private void dfs(int node,
List<List<Integer>> graph,
boolean[] visited) {
    visited[node] = true;
// mark visited
    for (int neighbor :
graph.get(node)) { // explore
neighbors
        if (!visited[neighbor])
{
            dfs(neighbor,
graph, visited);
        }
    }
}
```

AHA Graph traversal only changes neighbor source: **graph.get(node)**

3 GRID DFS TEMPLATE ★★★★★

Input	Grid
Caller	dfs(r, c)
Visited	boolean[][]
Movement	dirs[]
Used In	• Number of Islands • Flood Fill • Max Area of Island • Surrounded Regions

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) {
    int n = grid.length;
    int m = grid[0].length;

    // 1. Boundary check
    if (r < 0 || r >= n || c < 0 || c >= m) return;

    // 2. Visited or blocked check (assume '0' or '#' is blocked)
    if (visited[r][c] || grid[r][c] == '0' || grid[r][c] == '#') return;

    // 3. Mark visited
    visited[r][c] = true;

    // 4. Explore 4 directions
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0];
        int nc = c + d[1];
        dfs(nr, nc, grid, visited);
    }
}
```

AHA Grid is an implicit graph.

4 NODE GRAPH TEMPLATE ★★★★★

Input	Node (with neighbors)
Caller	dfs(node)
Visited	HashMap<Node, Node>
Return	Node (cloned node)
Used In	• Clone Graph

Java Template

```
private Node dfs(Node node, Map<Node, Node> visited) {
    if (node == null) return null;

    if (visited.containsKey(node)) {
        return visited.get(node); // already cloned
    }

    // 1. Create clone for current node
    Node clone = new Node(node.val);
    visited.put(node, clone);

    // 2. Clone all neighbors
    for (Node nei : node.neighbors) {

clone.neighbors.add(dfs(nei, visited));
    }

    return clone;
}
```

AHA HashMap replaces visited[].

5 MULTI-START DFS ★★★★

A OUTER LOOP DFS

Pattern:

```
for (all nodes)
    if (!visited[i])
        dfs(i)
```

Used In

- Connected Components
- Number of Provinces
- Number of Islands

B BORDER DFS

Pattern:

```
for (each border cell / node)
    if (!visited[b])
        dfs(b)
```

Used In

- Pacific Atlantic
- Surrounded

Java Snippet

```
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited);
    }
}
```

Java Snippet

```
// Example for grid
for (int c = 0; c < m; c++) {
    dfs(0, c, grid, visited);      // top row
    dfs(n - 1, c, grid, visited);  // bottom row
}
for (int r = 0; r < n; r++) {
    dfs(r, 0, grid, visited);      // left col
    dfs(r, m - 1, grid, visited);  // right col
}
```

💡 **The DFS never changes. Only WHERE it starts changes.**

6 WHAT ACTUALLY CHANGES? ★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES?
Graph DFS (Adjacency List)	Adjacency List	`dfs(start)`	`boolean[]`	void / boolean / int	Neighbors come from `graph.get(node)`
Grid DFS	Grid	`dfs(r, c)`	`boolean[][]`	void / int	Movement using `dirs[]` + boundary checks
Node DFS	Node Graph	`dfs(node)`	`HashMap<Node, Node>`	Node	Visited is HashMap instead of `boolean[]`
Outer Loop DFS	Adj List / Grid	`for all nodes if !visited dfs(i)`	`boolean[]` / `boolean[][]`	int / List / void	Starts DFS from every unvisited node
Border DFS (Multi-Start)	Grid / Graph	`for each border run dfs()`	`boolean[]` / `boolean[][]`	void	Starts DFS from borders / multiple sources

💡 **This table is your 30-second interview revision sheet.**

7 RETURN TYPE GUIDE ★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Marking / Traversal (No direct output)	Flood Fill, Surrounded Regions, Mark Components	We only visit nodes / cells. No value returned.
boolean	Searching / Decision	Valid Path, Graph Valid Tree, Cycle Detection	Returns true as soon as condition is met.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Count Components	Accumulate and return numeric result.
Node	Graph Construction	Clone Graph	Returns a new structure (node).
List	Collecting Results	All Paths, Topological Sort, Articulation Points	Collect multiple answers and return.

⭐ **Changing the return type usually changes the solution.**

8 AHA MOMENTS

🔗 **Adjacency List** changes only neighbor traversal.

▦ **Grid DFS** changes only movement using dirs[].

🟣 **Node DFS** changes visited[] into HashMap.

🔁 **Outer Loop** handles disconnected graphs.

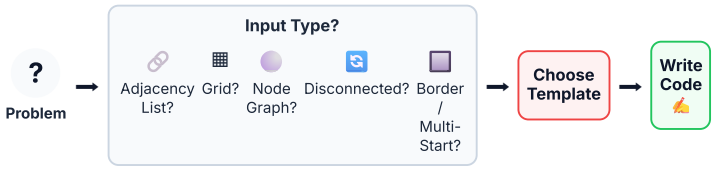
🔲 **Border DFS** changes only starting nodes.

🧠 **95% of interview problems modify only one or two lines from these templates.**

9 COMMON INTERVIEW MISTAKES

- ⓧ **Forgot visited[].**
- ⓧ **Forgot boundary check (for grid).**
- ⓧ **Forgot dirs[] for grid movement.**
- ⓧ **Forgot outer loop for disconnected graphs.**
- ⓧ **Forgot HashMap in Clone Graph.**
- ⓧ **Infinite recursion (no base case).**
- ⓧ **Marked visited after recursion.**
- ⓧ **Forgot disconnected graph handling.**

10 INTERVIEW THINKING



11 PAGE 5 PREVIEW

Next page contains real interview problems.

For each problem, we show:

Base Template → Changed Lines → AHA Moment → Complexity → Interview Tip

Problem (Understand) Input (What given) Representation (How to model) Helper Structure (Visited, dist, etc.) Pattern (DFS / BFS / Union Find / etc.)

Tiny Logic Change (Per Problem) Dry Run (Example) Optimize (Time/Space)

GRAPH PATTERN GUIDE

PAGE 4 — DFS TEMPLATE GUIDE (MASTER JAVA TEMPLATES)

"One algorithm. Multiple templates. Choose the right one."

1 MASTER IDEA ★★★★★

💡 **Almost every Graph DFS problem uses one of a few reusable templates.**

The DFS logic stays almost identical.

Only the input, caller, return type and tiny logic change.

2 GRAPH DFS (ADJACENCY LIST)

Input	Adjacency List (`List<List<Integer>>`)
Used In	Valid Path, Connected Components, Graph Valid Tree
Caller	`dfs(startNode)`
Visited	`boolean[] visited`
Return	void / boolean / int / List (as needed)

Java Template

```
private void dfs(int node,
List<List<Integer>> graph,
                 boolean[] visited) {
    visited[node] = true; // mark
visited
    for (int nei : graph.get(node)) {
// explore neighbors
        if (!visited[nei]) {
            dfs(nei, graph, visited);
        }
    }
}
```

AHA Graph neighbors come from `graph.get(node)` .

3 GRID DFS

Input	Grid (m x n)
Used In	Number of Islands, Flood Fill, Max Area of Island, Rotting Oranges (DFS version)
Caller	`dfs(row, col)`
Visited	`boolean[][] visited`
Helpers	`int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}}`
Return	void / int (as per problem)

Java Template

```
private void dfs(int r, int c, char[][]
grid, boolean[][] visited) {
    int m = grid.length, n =
grid[0].length;
    // boundary check
    if (r < 0 || c < 0 || r ≥ m || c
≥ n) return;
    // visited or blocked check
    if (visited[r][c] || grid[r][c] ==
'0') return;

    visited[r][c] = true; // mark
visited
    // explore 4 directions
    int[][] dirs = {{1,0}, {-1,0},
{0,1}, {0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c +
d[1];
        dfs(nr, nc, grid, visited);
    }
}
```

AHA Grid is just an implicit graph.

4 NODE GRAPH DFS

Input	Node (graph with neighbors)
Used In	Clone Graph
Caller	`dfs(node)`
Visited	`HashMap<Node, Node> map`
Return	Node (cloned node)

Java Template

```java
private Node dfs(Node node,
HashMap<Node, Node> map) {
    if (map.containsKey(node))
return map.get(node);

    Node clone = new
Node(node.val);
    map.put(node, clone); // mark
visited & store clone

    for (Node nei :
node.neighbors) {

clone.neighbors.add(dfs(nei,
map));
    }
    return clone;
}
```

AHA HashMap replaces visited[].

5 DISCONNECTED GRAPH DFS (OUTER LOOP)

Input	Graph (Adj List) / Grid
Used In	Connected Components, Number of Provinces, Number of Islands

Caller	`for (all nodes/cells)`
	`if (!visited)`
	`dfs(i) or dfs(r, c)`

Visited	`boolean[]`
	/
	`boolean[][]`

Return	int / void (depends on problem)

Java Template

```java
// Graph (Adjacency List) example
int count = 0;
boolean[] visited = new
boolean[n];
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited); //
use Graph DFS template
        count++;
    }
}
return count; // number of
components
```

AHA Outer loop discovers disconnected graphs.

6 BORDER DFS (MULTI-START POINTS)

Used In	Pacific Atlantic, Surrounded Regions
Caller	Run DFS from border cells / nodes

Visited	`boolean[][]`
	/
	`boolean[]`

Return	void

Java Template (Grid Example)

```java
public void dfsFromBorders(int m,
int n, char[][] grid,

boolean[][] visited) {
    // Top & Bottom rows
    for (int c = 0; c < n; c++) {
        if (!visited[0][c] &&
grid[0][c] == 'X') dfs(0, c, grid,
visited);
        if (!visited[m-1][c] &&
grid[m-1][c] == 'X') dfs(m-1, c,
grid, visited);
    }
    // Left & Right columns
    for (int r = 0; r < m; r++) {
        if (!visited[r][0] &&
grid[r][0] == 'X') dfs(r, 0, grid,
visited);
        if (!visited[r][n-1] &&
grid[r][n-1] == 'X') dfs(r, n-1,
grid, visited);
    }
}
```

AHA Multiple DFS starting points from borders.

7 RETURN TYPE GUIDE ★★★★★

Return Type	Used For	Example Problems	What It Means / When to Use
void	Marking / Visiting	Flood Fill, Surrounded Regions, Pacific Atlantic	You only need to visit and mark nodes/cells.
boolean	Searching / Decision	Valid Path, Detect Cycle, Bipartite	Need to know if something exists / is reachable.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Good Nodes	Need a number (count, area, size, etc.).
Node	Graph Construction	Clone Graph	Need to build and return nodes / structure.
List / Set	Collecting Answers	All Paths, Components, Topological Sort (DFS order), Eventual Safe States	Need to collect multiple results.

⭐ **Changing the return type usually changes the solution.**

8 TEMPLATE COMPARISON (30-SECOND RECAP) ★★★★★

Template	Input	Caller	Visited	Return	Special Change
🌱 Graph DFS (Adj List)	Adjacency List `List<List<Integer>>`	`dfs(startNode)`	`boolean[]`	void / boolean / int / List	Neighbors from `graph.get(node)`
▦ Grid DFS	Grid (`char[][]` / `int[][]`)	`dfs(r, c)`	`boolean[][]`	void / int / boolean	Boundary check + `dirs[]` movement
🔗 Node DFS	Node (with neighbors)	`dfs(node)`	`HashMap<Node, Node>`	Node	Use HashMap to map original → clone
🔁 Outer Loop DFS	Graph / Grid	`for all nodes/cells if ! visited dfs( ... )`	`boolean[]` / `boolean[][]`	int / List / void	Outer loop to handle disconnected parts
🗺️ Border DFS (Multi-Start)	Grid (heights/char) or Graph	Run DFS from borders	`boolean[][]` / `boolean[]`	void	Multiple starting points

9 AHA MOMENTS ★★★★★

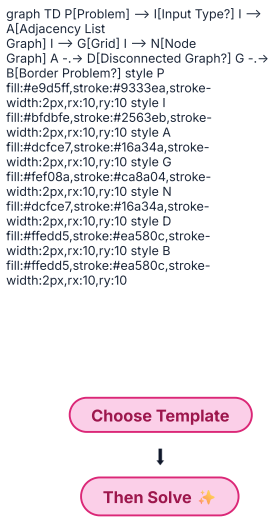
- 🌱 Adjacency List changes only neighbor traversal.
- ▦ Grid DFS changes only movement using `dirs[]` .
- 🔗 Node Graph changes `visited[]` into `HashMap` .
- 🔁 Outer Loop handles disconnected graphs.
- 🗺️ Border DFS changes only where DFS starts.

☑ **Most interview problems modify only one or two lines.**

10 COMMON INTERVIEW MISTAKES ★★★★★

- ❌ Forgot `visited[]` .
- ❌ Marked visited too late (after recursion).
- ❌ Forgot boundary check (for grid).
- ❌ Forgot `dirs[]` for grid movement.
- ❌ Forgot outer loop for disconnected graphs.
- ❌ Forgot multiple starting points (border DFS).
- ❌ Forgot `HashMap` in Clone Graph.
- ❌ Infinite recursion (no base case).

11 INTERVIEW THINKING FLOW ★★★★★



12 PREVIEW OF PAGE 5 ★★★★★

Next page contains DFS problem guide with real interview problems.

- ✓ Valid Path
- ✓ Connected Components
- ✓ Number of Islands
- ✓ Flood Fill
- ✓ Clone Graph
- ✓ Pacific Atlantic
- ✓ Surrounded Regions

Each problem highlights ONLY the changed lines from these templates.

flowchart LR A[" ❓ Problem
(Understand)"] --> B[" 📥 Input
(What given)"] B --> C[" 🔲 Representation
(How to model)"] C --> D[" 🛠️ Helper Structure
(Visited, dist, etc.)"] D --> E[" ⚙️ Pattern
(DFS / BFS /
Union Find / etc.)"] E --> F[" 🔬 Tiny Logic
Change
(Per Problem)"] F --> G[" ⛹️ Dry Run
(Example)"] G --> H[" ⏱️ Optimize
(Time/Space)"] style A fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style B fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style C fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style D fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style E fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style F fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style G fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8 style H fill:#fff,stroke:#cbd5e1,stroke-width:2px,rx:8,ry:8

GRAPH PATTERN GUIDE

PAGE 4 — MASTER DFS TEMPLATE GUIDE

"One DFS algorithm. Five reusable templates."

1 MASTER IDEA

Almost every Graph DFS interview problem uses one of five reusable templates.

The DFS traversal almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH DFS TEMPLATE★★★★★

Input	**Adjacency List**
Caller	**dfs(start)**
Visited	**boolean[]**
Return	**void / boolean / int**
Used In	• **Valid Path** • **Connected Components** • **Graph Valid Tree**

Java Template

```
private void dfs(int node, List<List<Integer>> graph, boolean[] visited) {
    visited[node] = true; // mark visited
    for (int neighbor : graph.get(node)) { // explore neighbors
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}
```

AHA **Graph traversal only changes neighbor source: graph.get(node)**

3 GRID DFS TEMPLATE★★★★★

Input	**Grid**
Caller	**dfs(r, c)**
Visited	**boolean[][]**
Movement	**dirs[]**
Used In	• **Number of Islands** • **Flood Fill** • **Max Area of Island** • **Surrounded Regions**

Java Template

```
private void dfs(int r, int c, char[][] grid, boolean[][] visited) {
    int m = grid.length;
    int n = grid[0].length;

    // 1. Boundary check
    if (r < 0 || r ⩾ m || c < 0 || c ⩾ n) return;

    // 2. Visited or blocked check
    if (visited[r][c] || grid[r][c] == '0' || grid[r][c] == 'X') return;

    // 3. Mark visited
    visited[r][c] = true;

    // 4. Explore 4 directions
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    for (int[] d : dirs) {
        int nr = r + d[0];
        int nc = c + d[1];
        dfs(nr, nc, grid, visited);
    }
}
```

AHA **Grid is an implicit graph.**

4 NODE GRAPH TEMPLATE★★★★

Input	**Node**
Caller	**dfs(node)**
Visited	**HashMap<Node, Node>**
Return	**Node (cloned node)**
Used In	**Clone Graph**

Java Template

```
private Node dfs(Node node, HashMap<Node, Node> map) {
    if (node == null) return null;

    if (map.containsKey(node)) {
        return map.get(node); // already cloned
    }

    // 1. Create clone for current node
    Node clone = new Node(node.val);
    map.put(node, clone);

    // 2. Clone all neighbors
    for (Node nei : node.neighbors) {

clone.neighbors.add(dfs(nei, map));
    }

    return clone;
}
```

AHA **HashMap replaces visited[].**

5 MULTI-START DFS★★★★

A OUTER LOOP DFS

Pattern	Used In
`for (all nodes) if (!visited[i]) dfs(i)`	• Connected Components • Number of Provinces • Number of Islands

Java Snippet

```
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited);
    }
}
```

B BORDER DFS

Pattern	Used In
`for (each border cell / node) if (!visited[b]) dfs(b)`	• Pacific Atlantic • Surrounded Regions

Java Snippet

```
// Example for grid
for (int c = 0; c < n; c++) {
    dfs(0, c, grid, visited);       // top row
    dfs(m - 1, c, grid, visited);   // bottom row
}
for (int r = 0; r < m; r++) {
    dfs(r, 0, grid, visited);       // left col
    dfs(r, n - 1, grid, visited);   // right col
}
```

⭐ The DFS never changes. Only WHERE it starts changes.

6 WHAT ACTUALLY CHANGES?★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES
Graph DFS (Adj List)	Adjacency List	dfs(start)	boolean[]	void / boolean / int	Neighbors come from graph.get(node)
Grid DFS	Grid	dfs(r, c)	boolean[][]	void / int or boolean	Movement using dirs[] + boundary checks
Node DFS	Node Graph	dfs(node)	HashMap<Node, Node>	Node	Visited is HashMap instead of boolean[]
Outer Loop DFS	Adj List / Grid	for all nodes if !visited dfs(i)	boolean[] / boolean[][]	int / List / void	Starts DFS from every unvisited node
Border DFS	Grid / Graph	for each border run dfs()	boolean[] / boolean[][]	void	Starts DFS from borders / multiple sources

⭐ This table is your 30-second interview revision sheet.

7 RETURN TYPE GUIDE★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
void	Marking / Traversal (No direct output)	Flood Fill, Surrounded Regions, Pacific Atlantic	We only visit nodes / cells. No value returned.
boolean	Searching / Decision	Valid Path, Graph Valid Tree, Cycle Detection	Returns true as soon as condition is met.
int	Counting / Aggregation	Number of Islands, Max Area of Island, Count Components	Accumulate and return numeric result.
Node	Graph Construction	Clone Graph	Returns a new structure (node).
List	Collecting Results	All Paths, Topological Sort, Articulation Points	Collect multiple answers and return.

⭐ Changing the return type usually changes the solution.

8 AHA MOMENTS ★★★★★

- Adjacency List changes only neighbor traversal.
- Grid DFS changes only movement using dirs[].
- Node DFS changes visited[] into HashMap.
- Outer Loop handles disconnected graphs.
- Border DFS changes only starting nodes.

💡 95% of interview problems modify only one or two lines from these templates.

9 COMMON INTERVIEW MISTAKES ★★★★★

- ⓧ Forgot visited[].
- ⓧ Forgot boundary check (for grid).
- ⓧ Forgot dirs[] for grid movement.
- ⓧ Forgot outer loop for disconnected graphs.
- ⓧ Forgot HashMap in Clone Graph.
- ⓧ Infinite recursion.
- ⓧ Marked visited after recursion.
- ⓧ Forgot disconnected graph handling.

10 INTERVIEW THINKING★★★★★

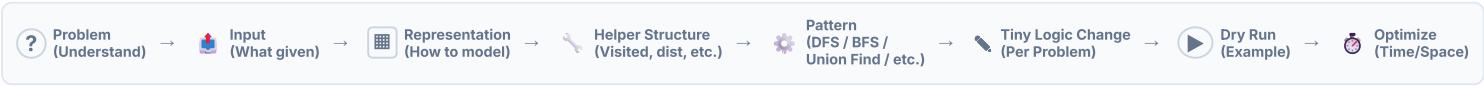
graph LR A((?)) -- Input Type? --> B{Adjacency List?} B --> C{Grid?} C --> D{Node Graph?} D --> E{Disconnected Graph?} E --> F{Border / Multi-Start?} style A fill:#fff,stroke:#64748b style B fill:#f1f5f9,stroke:#94a3b8 style C fill:#f1f5f9,stroke:#94a3b8 style D fill:#f1f5f9,stroke:#94a3b8 style E fill:#f1f5f9,stroke:#94a3b8 style F fill:#f1f5f9,stroke:#94a3b8

Choose Template → Write Code </>

11 PAGE 5 PREVIEW★★★★★

📖 **real interview problems.** For each problem, we show:

graph LR A[Base Template] --> B[Changed Lines] B --> C((AHA Moment)) C --> D[Complexity] D --> E((Interview Tip))


Problem (Understand)
Input (What given)
Representation (How to model)
Helper Structure (Visited, dist, etc.)
Pattern (DFS / BFS / Union Find / etc.)
Tiny Logic Change (Per Problem)
Dry Run (Example)
Optimize (Time/Space)

GRAPH PATTERN GUIDE

PAGE 6 — MASTER BFS TEMPLATE GUIDE

"One BFS algorithm. Five reusable templates."

1 MASTER IDEA★★★★★

💡 **Almost every Graph BFS interview problem uses one of a few reusable templates.**

The BFS traversal pattern almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH BFS TEMPLATE (ADJACENCY LIST)★★★★★

🔗 **Input**	Adjacency List (`List<List<Integer>>`)
📞 **Caller**	`bfs(start)`
✅ **Visited**	`boolean[]`
⬅ BACK **Return**	void / boolean / int
🎯 **Used In**	• Valid Path • Connected Components • Shortest Path (Unweighted) • Bipartite Check

Java Template

```
public void bfs(int start, List<List<Integer>> graph,
boolean[] visited) {
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        // process current node here (modify as per
problem)
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.offer(neighbor);
            }
        }
    }
}
```

AHA Graph neighbors come from `graph.get(node)` . BFS explores level by level.

3 GRID BFS TEMPLATE★★★★★

🗺️ **Input**: Grid

📞 **Caller**:

`bfs(r, c)`

✅ **Visited**:

`boolean[][]`

🏃 **Movement**:

`dirs[]`

🗄️ **Queue**:

`Queue<int[]>`

🎯 **Used In**:

- Rotting Oranges
- Walls and Gates
- Shortest Path in Binary Matrix
- 01 Matrix

Java Template

```
public void bfs(int r, int c, int[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{r, c});
    visited[r][c] = true;

    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int cr = cur[0], cc = cur[1];

        for (int[] d : dirs) {
            int nr = cr + d[0], nc = cc + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA Grid is an implicit graph. We move using `dirs[]`.

4 MULTI-SOURCE BFS TEMPLATE★★★★

💡 **Idea**: Push ALL sources first, then BFS.

🚢 **Queue Init**: Add all sources to queue.

🎯 **Used In**:

- Rotting Oranges
- Walls and Gates
- Distance to Nearest 0
- 01 Matrix
- Pacific Atlantic (variation)
- Multi-Source Shortest Path

Java Template

```
public void multiSourceBfs(List<int[]> sources, int m, int n, boolean[][] visited) {
    int[][] dirs = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    Queue<int[]> q = new LinkedList<>();

    // 1. Add all sources to queue
    for (int[] src : sources) {
        q.offer(src);
        visited[src[0]][src[1]] = true;
    }

    // 2. Process together level by level
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA Multiple starting points become one BFS. Distances spread outwards.

5 LEVEL ORDER BFS TEMPLATE★★★★

💡 **Idea**: Process one level at a time using queue size.

🔑 **Key Line**:

```
int size = q.size();
```

🎯 **Used In**:

- Tree Level Order
- Graph Levels
- Minimum Steps Problems
- Level wise Traversal

Java Template

```
public List<List<Integer>> levelOrder(int start, List<List<Integer>> graph) {
    List<List<Integer>> ans = new ArrayList<>();
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int size = q.size();
// level size
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            int node = q.poll();
            level.add(node);
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        ans.add(level);
    }
    return ans;
}
```

AHA Queue size gives the number of nodes at the current level.

6 DISTANCE BFS TEMPLATE★★★★

💡 **Idea**: Track distance (level) when you reach each node.

📏 **Track**: level variable OR distance[] array

🎯 **Used In**:

- Shortest Path (Unweighted)
- 01 Matrix
- Nearest Exit
- K Steps Problems
- Minimum Steps

Java Template

```
public int bfsShortestPath(int start, int target, List<List<Integer>> graph) {
    int n = graph.size();
    boolean[] visited = new boolean[n];
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;
    int level = 0;

    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            int node = q.poll();
            if (node == target) return level;
            for (int nei : graph.get(node)) {
                if (!visited[nei]) {
                    visited[nei] = true;
                    q.offer(nei);
                }
            }
        }
        level++;    // next distance
    }
    return -1; // not reachable
}
```

AHA First time you reach a node is the shortest distance in unweighted graphs.

7 WHAT ACTUALLY CHANGES?★★★★★

Template	Input	Caller	Visited	Return	ONLY THING THAT CHANGES
🔗 Graph BFS (Adj List)	Adjacency List `List<List<Integer>>`	`bfs(start)`	`boolean[]`	void / boolean / int	Neighbors come from `graph.get(node)`
▦ Grid BFS	Grid `(int[][])`	`bfs(r, c)`	`boolean[][]`	void / int	Movement using `dirs[]` + boundary checks
🚀 Multi-Source BFS	Grid / Graph (Any)	`multiSourceBfs(sources)`	`boolean[][]` or `boolean[]`	void / int	Initial queue contains all sources
📶 Level Order BFS	Graph / Tree (Any)	`bfs(start)`	`boolean[]`	`List<List<Integer>>` / void / int	Use queue size (level) to process by level
📏 Distance BFS	Graph / Grid (Any)	`bfs(start)`	`boolean[]` (or `distance[]`)	int	Track level or distance for each node

💡 **This table is your 30-second interview revision sheet. Memorize it.**

8 RETURN TYPE GUIDE★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
`void`	Traverse / Mark / Modify	Flood Fill, Surrounded Regions, Rotting Oranges	We only visit / update nodes.
`boolean`	Search / Existence Check	Valid Path, Word Ladder, Bipartite Check	Return true when condition is met.
`int`	Distance / Steps / Count	Shortest Path, Nearest Exit, 01 Matrix, Minimum Steps	Count level or distance.
`int[] / int[][]`	Store Distances	Walls and Gates, 01 Matrix, Distance to Nearest	Record best distance per node/cell.
`List / List<List<>>`	Collect Results	Level Order, All Shortest Paths, Top-K Level Problems	Collect multiple answers or levels.

⭐ **Changing the return type usually changes the solution.**

9 AHA MOMENTS ★★★★★

- 💡 BFS uses a Queue → explores level by level.
- ⏱️ First time you visit a node is always the shortest (when all edges have same weight).
- 🚀 Multi-source BFS = start from everywhere at once.
- 🪫 Queue size gives level size (very powerful).
- 🎯 BFS is perfect for shortest path, minimum steps, nearest, distance problems.

10 COMMON INTERVIEW MISTAKES ★★★★★

- ⊗ Forgot to mark visited.
- ⊗ Marked visited too late (after adding to queue).
- ⊗ Forgot queue size when doing level order.
- ⊗ Added the same node multiple times.
- ⊗ Didn't initialize all sources for multi-source BFS.
- ⊗ Used DFS instead of BFS for shortest path.
- ⊗ Infinite loop due to cycle (no visited).
- ⊗ Forgot disconnected graph (use outer loop).

11 INTERVIEW THINKING FLOW ★★★★★

graph TD A[Problem] → B[What do I need?] B → C1[Shortest Path] B → C2[Multiple Sources] B → C3[Traverse by Levels] B → C4[Reachability] B → C5[General Traversal] C1 → D1[Distance BFS] C2 → D2[Multi-Source BFS] C3 → D3[Level Order BFS] C4 → D4[Graph BFS] C5 → D5[Graph / Grid] D1 → E[Choose Template - Write Code] D2 → E D3 → E D4 → E D5 → E

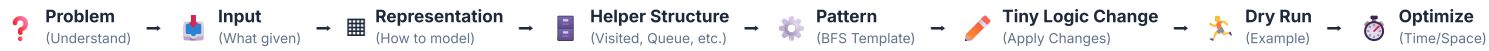
12 PAGE 7 PREVIEW★★★★★

📖 **Next page contains GRAPH BFS PROBLEMS.**

For each problem, you will see:

Base Template → Changed Lines → AHA Moment → Complexity → Interview Tip

Apply these templates with tiny changes and solve any BFS problem with confidence!


Problem
(Understand)
Input
(What given)
Representation
(How to model)
Helper Structure
(Visited, Queue, etc.)
Pattern
(BFS Template)
Tiny Logic Change
(Apply Changes)
Dry Run
(Example)
Optimize
(Time/Space)

GRAPH PATTERN GUIDE

PAGE 6 — MASTER BFS TEMPLATE GUIDE

"One BFS algorithm. Five reusable templates."

1 MASTER IDEA

Almost every Graph BFS interview problem uses one of the five reusable templates.

The BFS traversal pattern almost never changes.

Only the caller, input, return type and tiny logic change.

2 GRAPH BFS TEMPLATE (ADJACENCY LIST) ★★★★★

Input	Adjacency List (`List<List<Integer>>`)
Caller	`bfs(start)`
Visited	`boolean[]`
Return	`void` / `boolean` / `int`
Used In	• Valid Path • Connected Components • Shortest Path (Unweighted) • Bipartite Check

Java Template

```
public void bfs(int start,
List<List<Integer>> graph,
boolean[] visited) {
    Queue<Integer> q = new
LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        // process current
node here (modify as per
problem)
        for (int neighbor :
graph.get(node)) {
            if
(!visited[neighbor]) {

visited[neighbor] = true;

q.offer(neighbor);
            }
        }
    }
}
```

AHA Graph neighbors come from `graph.get(node)`. BFS explores level by level.

3 GRID BFS TEMPLATE★★★★★

Input	Grid
Caller	`bfs(r, c)`
Visited	`boolean[][]`
Movement	`dirs[]`
Queue	`Queue<int[]>`
Used In	• Rotting Oranges • Walls and Gates • Shortest Path in Binary Matrix • 01 Matrix

Java Template

```
public void bfs(int r, int c, int[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{r, c});
    visited[r][c] = true;

    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int cr = cur[0], cc = cur[1];

        for (int[] d : dirs) {
            int nr = cr + d[0], nc = cc + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA Grid is an implicit graph. We move using `dirs[]`.

4 MULTI-SOURCE BFS TEMPLATE★★★★★

Idea	Push ALL sources first, then BFS together.
Queue Init	Add all sources to queue.
Used In	• Rotting Oranges • Walls and Gates • Distance to Nearest 0 • 01 Matrix • Pacific Atlantic (variation) • Multi-Source Shortest Path

Java Template

```
public void bfs(List<int[]> sources, int m, int n,
                int[][] dirs, boolean[][] visited) {
    Queue<int[]> q = new LinkedList<>();

    // 1. Add all sources first
    for (int[] src : sources) {
        q.offer(src);
        visited[src[0]][src[1]] = true;
    }

    // 2. Process together level by level
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n
                && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

AHA Multiple starting points become one BFS. Distances spread outwards.

5 LEVEL ORDER BFS TEMPLATE★★★★★

Idea	Process one level at a time.
Key Line	`int size = q.size();`
Used In	• Tree Level Order • Graph Levels • Minimum Steps Problems • Level wise Traversal

Java Template

```
public List<List<Integer>>
levelOrder(List<List<Integer>>
graph, int start) {
    List<List<Integer>> ans =
new ArrayList<>();
    Queue<Integer> q = new
LinkedList<>();
    boolean[] visited = new
boolean[graph.size()];
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
// nodes in current level
        List<Integer> level =
new ArrayList<>();
        for (int i = 0, size =
q.size(); i < size; i++) {
            int node = q.poll();
            level.add(node);
            for (int nei :
graph.get(node)) {
                if
(!visited[nei]) {
                    visited[nei]
= true;

q.offer(nei);
                }
            }
        }
        ans.add(level);
    }
    return ans;
}
```

AHA Queue size gives the number of nodes at the current level.

6 DISTANCE BFS TEMPLATE★★★★★

Idea	Track distance (level) when you reach each node.
Track	`level` variable OR `distance[ ]` array
Used In	• Shortest Path (Unweighted) • 01 Matrix • Nearest Exit • K Steps Problems • Minimum Steps

Java Template

```
public int bfsShortestPath(int
start, int target,

List<List<Integer>> graph) {
    Queue<Integer> q = new
LinkedList<>();
    boolean[] visited = new
boolean[graph.size()];
    q.offer(start);
    visited[start] = true;
    int level = 0;

    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i <
size; i++) {
            int node = q.poll();
            if (node == target)
return level;
            for (int nei :
graph.get(node)) {
                if
(!visited[nei]) {
                    visited[nei]
= true;

q.offer(nei);
                }
            }
        }
        level++;    // next
distance
    }
    return -1; // not reachable
}
```

AHA First time you reach a node is the shortest distance in unweighted graphs.

7 WHAT ACTUALLY CHANGES?

Template	Input	Caller	Visited	Queue Init	Return	ONLY THING THAT CHANGES
Graph BFS (Adj List)	Adjacency List	`bfs(start)`	`boolean[]`	Add start node	`void` / `boolean` / `int`	Neighbors from `graph.get(node)`
Grid BFS	Grid	`bfs(r, c)`	`boolean[][]`	Add start cell	`void` / `int`	Movement using `dirs[]` + boundary checks
Multi-Source BFS	Grid / Graph (Any)	`bfs(sources)`	`boolean[][]` or `boolean[]`	Add all sources	`void` / `int`	Initial queue contains all sources
Level Order BFS	Graph / Tree (Any)	`bfs(start)`	`boolean[]`	Add start node	`List<List<Integer>>` / `void` / `int`	Use queue size (level) to process level
Distance BFS	Graph / Grid (Any)	`bfs(start)`	`boolean[]` or `distance[]`	Add start node	`int`	Track level or distance for each node

⭐ **This table is your 30-second interview revision sheet. Memorize it.**

8 RETURN TYPE GUIDE★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
`void`	Traverse / Mark / Modify	Flood Fill, Rotting Oranges, Walls and Gates	We only visit / update nodes.
`boolean`	Search / Existence Check	Valid Path, Bipartite Check, Word Ladder	Return true when condition is met.
`int`	Distance / Steps / Count	Shortest Path, Nearest Exit, 01 Matrix, Minimum Steps	Count level or distance.
`int[]` / `int[][]`	Store Distances	Walls and Gates, 01 Matrix, Distance to Nearest	Record best distance per node/cell.
`List` / `List<List<...>>`	Collect Results	Level Order, All Shortest Paths, Top-K Level Problems	Collect multiple answers or levels.

⭐ **Changing the return type usually changes the solution.**

9 AHA MOMENTS★★★★★

- **BFS uses a Queue** → explores level by level.
- **First time you visit a node is always the shortest (in unweighted graphs).**
- **Multi-source BFS** = start from everywhere at once.
- **Queue size** gives level size (very powerful).
- **BFS is perfect for shortest path**, minimum steps, nearest, distance problems.

10 COMMON INTERVIEW MISTAKES★★★★★

- ⊗ Forgot to mark visited.
- ⊗ Marked visited too late (after adding to queue).
- ⊗ Forgot queue size when doing level order.
- ⊗ Added the same node multiple times.
- ⊗ Didn't initialize all sources for multi-source BFS.
- ⊗ Used DFS instead of BFS for shortest path.
- ⊗ Infinite loop due to cycle (no visited).
- ⊗ Forgot disconnected graph (use outer loop).

11 INTERVIEW THINKING FLOW★★★★★

graph TD P[Problem] —> W[What do I need?] W —> S1[Shortest Path (One Source)] W —> S2[Minimum / Number of Steps] W —> S3[Traverse by Level] W —> S4[Reachability / Existence] W —> S5[General Traversal] S1 —> D[Distance BFS] S2 —> M[Multi-Source BFS] S3 —> L[Level Order BFS] S4 —> G1[Graph BFS] S5 —> G2[Graph / Grid BFS] D —> C[Choose Template → Write Code] M —> C L —> C G1 —> C G2 —> C

12 PAGE 7 PREVIEW★★★★★

Next page contains GRAPH BFS PROBLEMS.

For each problem, you will see:

- Base Template
- Changed Lines
- AHA Moment
- Complexity
- Interview Tip

Apply these templates with tiny changes and solve any BFS problem with confidence!

GRAPH PATTERN GUIDE

TOPIC: TOPOLOGICAL SORT PROBLEM GUIDE (APPLY TEMPLATES)

"Same algorithm. Different problems. Only the changing parts."

HOW TO USE THIS PAGE

1. Find the problem.
2. Identify the base template (from Page 8).
3. Read the CHANGED LINES.
4. Understand the AHA.
5. Review complexity & interview tip.
6. Practice writing from memory.

TEMPLATE LEGEND (From Page 8)

- **K** Kahn's (BFS) — Indegree + Queue
- **S** DFS + Stack — Postorder
- **H** Kahn's + Min Heap — Lexicographically Smallest
- **C** Cycle Check — (Any Method)

1 COURSE SCHEDULE K

Base Template: Kahn's Algorithm (BFS)

Problem

Can we finish all courses? (No cycle)

Changed Lines (from Template)

```
// 1. Graph: for each [a, b] → b → a (take b then a)
for (int[] p : prerequisites) {
    int a = p[0], b = p[1];
    graph.get(b).add(a);
    indegree[a]++;
}

// 2. Return true if we process all courses
return count == numCourses;
```

Caller	Return	Key Idea
canFinish(numCourses, prereq)	`boolean`	All courses processed?

AHA: If any course has remaining indegree → cycle.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Think in terms of ordering dependencies.

2 COURSE SCHEDULE II K

Base Template: Kahn's Algorithm (BFS)

Problem

Return any valid order to finish all courses.

Changed Lines (from Template)

```
// 1. Same graph as Course Schedule
for (int[] p : prerequisites) {
    int a = p[0], b = p[1];
    graph.get(b).add(a);
    indegree[a]++;
}
// 2. Store order while processing
List<Integer> order = new ArrayList<>();
 ...
order.add(node);
// 3. Final check
if (order.size() ≠ numCourses)
return new int[0];
// cycle
return order.stream().mapToInt(i -> i).toArray();
```

Caller	Return	Key Idea
findOrder(numCourses, prereq)	`int[]`	Build order while processing

AHA: Kahn's gives order naturally.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
If size < numCourses → cycle, return empty.

3 ALIEN DICTIONARY S

Base Template: DFS + Stack (Postorder)

Problem

Find order of characters from sorted dictionary.

Changed Lines (from Template)

```
// 1. Build graph from first diff char
// Word1 comes before Word2
// Edge: char1 → char2
for (int i = 0; i < words.length - 1; i++) {
    String w1 = words[i], w2 = words[i + 1];
    int len = Math.min(w1.length(), w2.length());
    for (int j = 0; j < len; j++) {
        if (w1.charAt(j) ≠ w2.charAt(j)) {

graph.get(w1.charAt(j)).add(w2.charAt(j));
            break;
        }
    }
}
// 2. DFS postorder gives character order
```

Caller	Return	Key Idea
alienOrder(words)	`String`	Edges from first diff char

AHA: Postorder of DFS = valid character order.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Also handle invalid case: "abc" before "ab".

4 BUILD ORDER K

Base Template: Kahn's Algorithm (BFS)

Problem
Given projects and dependencies, return build order.

Changed Lines (from Template)

```
// 1. Graph: for each [a, b] → b → a
// means build b before a
for (int[] d : dependencies) {
    int a = d[0], b = d[1];
    graph.get(b).add(a);
    indegree[a]++;
}
// 2. Return order list
if (order.size() ≠ numProjects)
return new int[0];
```

Caller	Return	Key Idea
findBuildOrder(numProjects, deps)	`int[]`	Standard topo order

AHA: Same as Course Schedule II.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Exactly same pattern, different story.

5 TASK SCHEDULER (WITH ADVANCED CONSTRAINT) K

Base Template: Kahn's Algorithm (BFS)

Problem
Return order if possible, else empty.

Changed Lines (from Template)

```
// 1. Build graph from conditions
for (int[] c : conditions) {
    int u = c[0], v = c[1]; // u before v
    graph.get(u).add(v);
    indegree[v]++;
}

// 2. Standard Kahn's
// Return order only if all tasks processed
```

Caller	Return	Key Idea
taskOrder(tasks, conditions)	`int[]`	Dependencies between tasks

AHA: If some tasks remain → cycle.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Check indegree[] after processing all nodes.

6 MINIMUM SEMESTERS K

Base Template: Kahn's Algorithm (BFS)

Problem
Return minimum semesters to finish all courses.

Changed Lines (from Template)

```
// 1. Standard graph from prerequisites
// 2. Use level-based processing (BFS levels)
int semesters = 0;
while (!q.isEmpty()) {
    int size = q.size();
    semesters++;
    for (int i = 0; i < size; i++) {
        int node = q.poll();
        for (int nei : graph.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0) q.offer(nei);
        }
    }
}
return (count == numCourses) ? semesters : -1;
```

Caller	Return	Key Idea
minimumSemesters(numCourses, prereq)	`int`	BFS levels = semesters

AHA: Levels in BFS give minimum time.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Count levels, not nodes. That's the answer.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	Main Change
1	Course Schedule	Kahn's (BFS)	Return boolean (processed all?)
2	Course Schedule II	Kahn's (BFS)	Store order (list) & return array
3	Alien Dictionary	DFS + Stack	Build graph from words (First diff)
4	Build Order	Kahn's (BFS)	Same as Course Schedule II
5	Task Scheduler	Kahn's (BFS)	General dependencies
6	Minimum Semesters	Kahn's (BFS)	Level BFS → count semesters

8 TOPO SORT TEMPLATE RECAP

Kahn's (BFS)

- Indegree[]
- Queue of 0 indegree
- Process → reduce indegree
- Detect cycle if not all processed

DFS + Stack

- DFS all nodes
- Push after visiting neighbors
- Reverse stack → order
- Detect cycle with recursion stack

9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
`boolean`	Check if order is possible	Course Schedule
`int[]` / `List<Integer>`	Need the order	Course Schedule II, Build Order
`String`	Order of characters	Alien Dictionary
`int`	Minimum time / levels	Minimum Semesters
`-1` / `Empty`	Cycle exists	All problems when impossible

10 INTERVIEW CHECKLIST

- ☑ Built correct graph direction?
- ☑ Calculated indegree[] correctly?
- ☑ Added all nodes (even with 0 indegree)?
- ☑ Used correct template (Kahn's or DFS)?
- ☑ Handled cycle case?
- ☑ Returned correct type?
- ☑ Time & space complexity?
- ☑ Dry run on small example?
- ☑ Can you explain your approach clearly?

11 INTERVIEW THINKING FLOW

graph LR A((?)) ⟶|Problem
(Understand)| B[Dependencies
Exist?] B ⟶|Yes| C(( )) C ⟶|Directed
Graph
Yes| D{Need Order?} D ⟶|Yes → Topo Sort| E[Kahn's BFS
or DFS + Stack?] D ⟶|No → Other Algo| D E ⟶|BFS if easier| F(Write Code
Template) E ⟶|DFS if recursion
preferred| F F ⟶ G(Test & Verify
Cycle? Output?) G ⟶ H((Win)) classDef default fill:#fff,stroke:#333,stroke-width:2px; classDef decision fill:#fff,stroke:#333,stroke-width:2px,shape:diamond; class D decision;

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)

(?) **Problem** (Understand)→⌨ **Input** (What given)→⌨ **Representation** (Graph + Indegree)→☰ **Helper Structure** (Queue / Stack + Indegree) →⚙ **Pattern** (Topo Sort Template) →✎ **Tiny Logic Change** (Apply Changes) →⟳ **Dry Run** (Example)→⏱ **Optimize** (Time/Space)

GRAPH PATTERN GUIDE

PAGE 8 — MASTER TOPOLOGICAL SORT TEMPLATE GUIDE
(FAANG Cheat Sheet)

"Order matters. Use Topological Sort."

1 MASTER IDEA

Use Topological Sort when dependencies exist.

It gives a valid linear order of tasks.

If impossible → **CYCLE exists.**

TIP: If `order.size() ≠ numCourses`

→ Cycle exists (impossible order).

2 KAHN'S ALGORITHM (BFS TOPOLOGICAL SORT) ★★★★★

Input	Directed Graph (Adj List)
Caller	`topoSort(numCourses, edges)`
Key Idea	Remove 0 indegree nodes repeatedly
Data Structure	`Queue<Integer> + indegree[]`
Return	`List<Integer> order`
Used In	• Course Schedule • Course Schedule II • Alien Dictionary • Build Order

Java Template

```
public List<Integer> topoSort(int numCourses, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) {
        int u = e[1], v = e[0]; // u → v
        graph.get(u).add(v);
        indegree[v]++;
    }

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) q.offer(i); // all starting points
    }

    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int node = q.poll();
        order.add(node);
        for (int nei : graph.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0) q.offer(nei);
        }
    }

    return (order.size() == numCourses) ? order : new ArrayList<>(); // empty → cycle
}
```

AHA MOMENTS

- Indegree 0 nodes are "ready" to take.
- Remove it, reduce indegree of neighbors.
- If all nodes are taken → DAG (no cycle).
- If not all nodes → cycle exists.

COMMON MISTAKES

- Forgot `indegree[]` initialization.
- Didn't add all 0 indegree nodes first.
- Used Stack instead of Queue (wrong order).
- Didn't check cycle (`order.size()` test).

3 TOPO SORT (DFS + STACK METHOD) ★★★★

Input	Directed Graph (Adj List)
Caller	`topoSortDFS(numCourses, edges)`
Key Idea	Finish node → push to stack
Data Structure	`Stack<Integer>` + `visited[]` + `recStack[]`
Return	`List<Integer> order`
Used In	• Course Schedule II (Alt) • Alien Dictionary (Alt) • Build Order (Alt)

Java Template

```
public List<Integer> topoSortDFS(int numCourses, int[]
[] edges) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new
ArrayList<>());
    for (int[] e : edges) graph.get(e[1]).add(e[0]);

    boolean[] visited = new boolean[numCourses];
    boolean[] recStack = new boolean[numCourses];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < numCourses; i++) {
        if (!visited[i]) {
            if (dfs(i, graph, visited, recStack,
stack)) return new ArrayList<>(); // cycle
        }
    }

    List<Integer> order = new ArrayList<>();
    while (!stack.isEmpty()) order.add(stack.pop());
    return order;
}

private boolean dfs(int node, List<List<Integer>>
graph,
                    boolean[] visited, boolean[]
recStack, Stack<Integer> stack) {
    visited[node] = true;
    recStack[node] = true;

    for (int nei : graph.get(node)) {
        if (!visited[nei]) {
            if (dfs(nei, graph, visited, recStack,
stack)) return true;
        } else if (recStack[nei]) {
            return true; // cycle
        }
    }
    recStack[node] = false;
    stack.push(node);
    return false;
}
```

AHA

- Postorder of DFS gives a valid topological order.
- `recStack` detects back-edge → cycle.
- Stack reversed = correct order.

WHEN TO USE WHICH?

Approach	Use When
Kahn's (BFS)	Need lexicographically smallest order
Kahn's (BFS)	Graph is large / iterative preferred
DFS + Stack	Easy cycle detection in recursion
DFS + Stack	Natural for recursive thinking

4 LEXICOGRAPHICALLY SMALLEST ORDER (BFS + MIN HEAP) ★★★★

Change From Kahn's Algorithm	Use PriorityQueue (Min Heap)
Data Structure	`PriorityQueue<Integer> pq`
Use In	• Course Schedule II (Lexico) • Smallest Order Problems

Java Template

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int i = 0; i < numCourses; i++) {
    if (indegree[i] == 0) pq.offer(i);
// min heap instead of queue
}
List<Integer> order = new ArrayList<>();
while (!pq.isEmpty()) {
    int node = pq.poll(); // smallest node first
    order.add(node);
    for (int nei : graph.get(node)) {
        indegree[nei]--;
        if (indegree[nei] == 0) pq.offer(nei);
    }
}
```

AHA Min Heap ensures smallest available node is always picked.

5 CYCLE DETECTION (IN DAG CHECK) ★★★★

Method	Kahn's (BFS)
Check	`order.size() ≠ V`
Result	Cycle Exists
Why?	Some nodes always have `indegree > 0` (cycle)

AHA MOMENT

If we cannot remove all nodes → cycle exists.

COMMON PITFALLS

- Not returning empty list on cycle.
- Confusing 0 indegree concept.
- Not building graph properly.
- Forgetting disconnected components.

TIP Always verify `order.size() == numCourses` (or V).

6 WHAT ACTUALLY CHANGES? ★★★★★

Template	Graph Type	Core Idea	Data Structure	Only Thing That Changes (Queue vs Min Heap)	Typical Problems
Kahn's (BFS)	Directed Acyclic Graph	Remove 0 indegree nodes	Queue + indegree[]	How we pick next node	*Course Schedule, Alien Dictionary, Build Order*
Kahn's (BFS + Min Heap)	DAG	Remove smallest 0 indegree	Min Heap + indegree[]	Use PriorityQueue instead of Queue	*Course Schedule II (Lexico)*
DFS + Stack	DAG	Finish node → push stack	Stack + visited[] + recStack[]	Order from stack (reverse)	*Course Schedule II, Alien Dictionary*
Cycle Detection	Directed Graph	If not all nodes visited → cycle	Same as above	Check `order.size() ≠ V`	*Course Schedule (cycle check)*

7 RETURN TYPE GUIDE ★★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
`List<Integer>`	Valid order	*Course Schedule II, Alien Dictionary*	Provide correct order
`boolean`	Cycle exists?	*Course Schedule*	Feasibility check
`int[]`	Order as array	*Build Order*	Return index-based order
`String`	Order as string	*Alien Dictionary*	Lexico order (characters)

★ **Changing return type usually changes the solution slightly.**

8 INTERVIEW THINKING FLOW ★★★★★

graph TD A[Problem] → B{Dependencies exist?} B -- No → C[Use DFS / BFS] B -- Yes → D[Directed Graph] D → E{Need order?} E -- Yes → F[Topological Sort] E -- No → G[Check cycle?] G -- Yes → H[Cycle Only Check] F → I[Choose Template → Write Code] H → I

9 PAGE 9 PREVIEW ★★★★★

Next page contains **TOPOLOGICAL SORT PROBLEM GUIDE**.

For each problem, you will see:

- Base Template
- Changed Lines
- AHA Moment
- ⏱ Complexity
- Interview Tip

Apply these templates with tiny changes and solve any Topo Sort problem with confidence!

(?) Problem (Understand) → Input (What given) → Representation (Graph + Indegree) → Helper Structure (Queue / Stack / PQ) → Pattern (Topo Sort Template) → Tiny Logic Change (Apply Changes) → Dry Run (Example) → Optimize (Time/Space)

GRAPH PATTERN GUIDE

PAGE 5 — GRAPH DFS PROBLEM GUIDE (APPLY TEMPLATES)

"Same DFS. Different problems. Only the changing parts."

PAGE 5

HOW TO USE THIS PAGE

① Find the problem.
② Identify the Base Template (from Page 4).
③ Read the **CHANGED LINES**.
④ Understand the AHA.
⑤ Write code in the interview.

TEMPLATE LEGEND (From Page 4)

G Graph DFS (Adj List) | Gd Grid DFS | N Node DFS | OL Outer Loop DFS | Bd Border DFS

1 VALID PATH G

Base Template: Graph DFS (Adjacency List)

Problem
Determine if there is a path between source and destination.

Changed Lines (from Template)

```
// 1. Add destination and return
boolean
if (node == destination) return
true;

// 2. Change return type and
propagate
for (int neighbor : graph.get(node))
{
    if (!visited[neighbor]) {
        if (dfs(neighbor,
destination, graph, visited))
            return true;
    }
}
return false;
```

Caller	dfs(source, destination, graph, visited)	Return **boolean**	Visited **boolean[]**

💡 Stop early when destination is found.

Complexity
Time: O(V + E)
Space: O(V)

Interview Tip
Mention early termination optimization.

2 CONNECTED COMPONENTS G

Base Template: Outer Loop DFS (Graph DFS + Outer Loop)

Problem
Count number of connected components in an undirected graph.

Changed Lines (from Template)

```
// 1. Outer loop to cover all nodes
int count = 0;
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        count++;
        dfs(i, graph, visited);
    }
}
// 2. DFS remains same (void)
```

Caller	for all nodes i	Return **int (count)**	Visited **boolean[]**

💡 Each unvisited node starts a new component.

Complexity
Time: O(V + E)
Space: O(V)

Interview Tip
Explain why we need outer loop.

3 NUMBER OF ISLANDS Gd

Base Template: Outer Loop DFS (Grid DFS + Outer Loop)

Problem
Count number of islands (groups of 1s) in a 2D grid.

Changed Lines (from Template)

```
// 1. Outer loop over the entire
grid
int count = 0;
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (grid[r][c] == '1' &&
!visited[r][c]) {
            count++;
            dfs(r, c, grid,
visited);
        }
    }
}
// 2. DFS remains same (mark all
connected land)
```

Caller	for all cells (r, c)	Return **int (count)**	Visited **boolean[][]**

💡 Every new unvisited land cell = new island.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Don't forget to mark visited.

5 PACIFIC ATLANTIC WATER FLOW Bd

Base Template: Border DFS (Multi-Source DFS)

Problem
Find cells that can flow water to both Pacific and Atlantic.

Changed Lines (from Template)

```
// 1. Run DFS from all Pacific borders
for (int r = 0; r < m; r++) dfs(r, 0,
heights, pacific);
for (int c = 0; c < n; c++) dfs(0, c,
heights, pacific);

// 2. Run DFS from all Atlantic borders
for (int r = 0; r < m; r++) dfs(r, n-1,
heights, atlantic);
for (int c = 0; c < n; c++) dfs(m-1, c,
heights, atlantic);

// 3. Collect intersection
if (pacific[r][c] && atlantic[r][c]) add to
answer;
```

Caller	Multiple border starts	Return **List<List<Integer>>**	Visited **boolean[][] (two arrays)**

💡 Flow is reverse. Start DFS from oceans.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Don't do DFS from every cell!

4 FLOOD FILL Gd

Base Template: Grid DFS

Problem
Replace the starting pixel's color with a new color.

Changed Lines (from Template)

```
// 1. Store starting color
char originalColor = image[sr][sc];

// 2. Add color check
if (image[r][c] ≠ originalColor)
return;

// 3. Change the color
image[r][c] = newColor;

// 4. Rest same (visited + 4
directions)
```

Caller	dfs(sr, sc, image, visited, newColor, originalColor)	Return **void**	Visited **boolean[][]**

💡 Only change pixels with the original color.

Complexity
Time: O(m * n)
Space: O(V n)

Interview Tip
Boundary + color check prevent extra work.

QUICK REFERENCE (CHANGES OVERVIEW)

Problem	Base Template	Main Change	Return
1 Valid Path	G	Early stop + boolean	boolean
2 Connected Components	OL	Outer loop + count	int
3 Number of Islands	OL + Gd	Grid outer loop + count	int
4 Flood Fill	Gd	Color check + change	void
5 Pacific Atlantic	Bd	Multi-source + intersect	List<List<Integer>>
6 Surrounded Regions	Bd	Mark safe + flip rest	void
7 Clone Graph	N	HashMap for visited	Node

G Graph DFS · Gd Grid DFS · N Node DFS · OL Outer Loop DFS · Bd Border DFS

6 SURROUNDED REGIONS Bd

Base Template: Border DFS

Problem
Capture all regions not connected to boundary.

Changed Lines (from Template)

```
// 1. Mark boundary connected 'O's
Run DFS from all borders and mark as
'safe'

// 2. Flip remaining 'O's
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (board[r][c] == 'O' &&
!safe[r][c])
            board[r][c] = 'X';
    }
}
```

Caller	All border cells	Return **void**	Visited **boolean[][] (safe)**

💡 Only border-connected regions are safe.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Two step thinking is the key.

7 CLONE GRAPH N

Base Template: Node Graph DFS

Problem
Clone an undirected graph.

Changed Lines (from Template)

```
// Already handled in template
// Key idea:
// If node not in map → create
clone
// For each neighbor → recurse and
add edge
```

Caller	dfs(node, map)	Return **Node (cloned)**	Visited **HashMap<Node, Node>**

💡 HashMap avoids infinite loop in cycles.

Complexity
Time: O(V + E)
Space: O(V)

Interview Tip
Always check map before creating clone.

8 INTERVIEW CHECKLIST

- [x] Identified the correct template?
- [x] Built correct representation?
- [x] Initialized visited structure correctly?
- [x] Handled all starting points?
- [x] Applied boundary / color / condition checks?
- [x] Returned correct type?
- [x] Considered disconnected components?
- [x] Can you explain time/space complexity?
- [x] Dry run on small example?
- [x] Optimized when possible?

Problem (Understand) → **Input** (What given) → **Representation** (How to model) → **Helper Structure** (Visited, dist, etc.) → **Pattern** (Choose DFS Template) → **Tiny Logic Change** (Apply Changes) → **Dry Run** (Example) → **Optimize** (Time/Space)

GRAPH PATTERN GUIDE

PAGE 7 — GRAPH BFS PROBLEM GUIDE (APPLY TEMPLATES)

"Same BFS. Different problems. Only the changing parts."

HOW TO USE THIS PAGE

1. Find the problem.
2. Identify the Base Template (from Page 6).
3. Read the **CHANGED LINES**.
4. Understand the AHA.
5. Review the complexity & interview tip.
6. Practice writing from memory.

TEMPLATE LEGEND (FROM PAGE 6)

Gb Graph BFS (Adj List) Gr Grid BFS Ms Multi-Source BFS Lv Level Order BFS Ds Distance BFS

1 ROTTING ORANGES Ms

Base Template: Multi-Source BFS (Page 6)

Problem
Return minutes until all oranges rot. Return -1 if impossible.

Changed Lines (from Template)

```
// 1. Push all rotten oranges
(sources)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == 2) {
            q.offer(new int[]{i,
j});
        }
    }
}
// 2. Count fresh oranges
fresh = count all 1's in grid;
// 3. While loop level++ gives
time (minutes)
int minutes = 0;
while (!q.isEmpty() && fresh > 0)
{
    int size = q.size();
    minutes++;
    // process level
}
return fresh == 0 ? minutes : -1;
```

Caller	Return	Visited	Key Idea
multiSourceBfs() (all rotten)	int (minutes)	boolean[][]	Spread level by level

💡 **AHA:** Multiple starting rotten oranges spread at the same time.

Complexity
Time: O(n * m)
Space: O(n * m)

Interview Tip
Don't forget to count fresh oranges before BFS.

2 WALLS AND GATES Ms

Base Template: Multi-Source BFS

Problem
Fill each empty room with distance to nearest gate.

Changed Lines (from Template)

```
// 1. Push all gates (0) into
queue
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (rooms[i][j] == 0) {
            q.offer(new int[]{i,
j});
        }
    }
}
// 2. When visiting empty room,
update distance
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) {
        int nr = cur[0] + d[0], nc
= cur[1] + d[1];
        if (inBounds(nr, nc) &&
rooms[nr][nc] == INF) {
            rooms[nr][nc] =
rooms[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr,
nc});
        }
    }
}
```

Caller	Return	Visited	Key Idea
multiSourceBfs() (all gates)	void (in-place)	rooms[][]	Distance from nearest gate

💡 **AHA:** Gates are sources. Distances expand outward.

Complexity
Time: O(n * m)
Space: O(n * m)

Interview Tip
Use original rooms[][] as visited (INF = unvisited).

3 SHORTEST PATH IN BINARY MATRIX Ds

Base Template: Grid BFS (Distance BFS)

Problem
Find shortest path from top-left to bottom-right in binary matrix.

Changed Lines (from Template)

```
// 1. If start or end is blocked
if (grid[0][0] == 1 || grid[n-1]
[n-1] == 1) return -1;
// 2. Start from (0,0)
q.offer(new int[]{0, 0});
visited[0][0] = true;
int steps = 1;
// 3. Level = path length
while (!q.isEmpty()) {
    int size = q.size();
    for (int k = 0; k < size; k++)
{
        int[] cur = q.poll();
        if (cur[0] == n-1 &&
cur[1] == n-1) return steps;
        for (int[] d : dirs8) { //
8 directions
            int nr = cur[0] +
d[0], nc = cur[1] + d[1];
            if (inBounds(nr, nc)
&& grid[nr][nc] == 0 &&
!visited[nr][nc]) {
                visited[nr][nc] =
true;
                q.offer(new int[]
{nr, nc});
            }
        }
    }
    steps++;
}
return -1;
```

Caller	Return	Visited	Key Idea
bfs(0,0)	int (steps)	boolean[][]	Level = path length

💡 **AHA:** First time we reach destination = shortest path.

Complexity
Time: O(n * m)
Space: O(n * m)

Interview Tip
Don't forget 8 directions.

4 WORD LADDER (LENGTH) Gb

Base Template: Graph BFS (Adj List) (Distance BFS)

Problem
Return length of shortest transformation sequence.

Changed Lines (from Template)

```
// 1. Build graph: word → all one-letter transformations
Map<String, List<String>> graph = buildGraph(wordList);
// 2. BFS from beginWord
Queue<String> q = new LinkedList<>();
q.offer(beginWord);
visited.add(beginWord);
int steps = 1;
while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        String cur = q.poll();
        if (cur.equals(endWord)) return steps;
        for (String nei : graph.get(cur)) {
            if (!visited.contains(nei)) {
                visited.add(nei);
                q.offer(nei);
            }
        }
    }
    steps++;
}
return 0;
```

Caller	Return	Visited	Key Idea
bfs(beginWord)	int (length)	HashSet<String>	Level = sequence length

💡 **AHA:** BFS guarantees first time we reach endWord is shortest.

Complexity
Time: O(N * L^2)
Space: O(N * L^2)

Interview Tip
Precompute neighbors to save time.

5 NEAREST EXIT FROM ENTRANCE IN MAZE Gr

Base Template: Grid BFS (Distance BFS)

Problem
Find minimum steps to nearest exit from entrance.

Changed Lines (from Template)

```
// 1. Start from entrance only
q.offer(new int[]{entrance[0], entrance[1]});
visited[entrance[0]][entrance[1]] = true;
int steps = 0;
// 2. Exit = boundary cell (not entrance)
while (!q.isEmpty()) {
    int size = q.size();
    for (int k = 0; k < size; k++) {
        int[] cur = q.poll();
        if (isBoundary(r, c) && (r ≠ entrance[0] || c ≠ entrance[1]))
            return steps;
        for (int[] d : dirs4) {
            int nr = r + d[0], nc = c + d[1];
            if (inBounds(nr, nc) && maze[nr][nc] == '.' && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
    steps++;
}
return -1;
```

Caller	Return	Visited	Key Idea
bfs(entrance)	int (steps)	boolean[][]	First boundary cell found

💡 **AHA:** BFS expands in layers. First exit found is minimum steps.

Complexity
Time: O(n * m)
Space: O(n * m)

Interview Tip
Don't treat entrance itself as exit.

6 01 MATRIX (UPDATE MATRIX) Ms

Base Template: Multi-Source BFS

Problem
For each cell, distance to nearest 0.

Changed Lines (from Template)

```
// 1. Push all cells with 0
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (mat[i][j] == 0) q.offer(new int[]{i, j});
    }
}
// 2. Process and update 1's with distance
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs4) {
        int nr = cur[0] + d[0], nc = cur[1] + d[1];
        if (inBounds(nr, nc) && mat[nr][nc] == 1) {
            mat[nr][nc] = mat[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
}
```

Caller	Return	Visited	Key Idea
multiSourceBfs() (all 0 cells)	int[][] (in-place)	mat[][]	Distance from nearest 0

💡 **AHA:** All zeros start together, distances fill the matrix.

Complexity
Time: O(n * m)
Space: O(n * m)

Interview Tip
Use matrix itself to store distances.

7 CHANGED LINES SUMMARY (QUICK REFERENCE)

#	Problem	Base Template	Only What Changes
1	Rotting Oranges	Multi-Source BFS	Push all rotten, count fresh, level = minutes
2	Walls and Gates	Multi-Source BFS	Push all gates, update room[nr][nc]
3	Shortest Path Binary Matrix	Grid BFS (Distance)	Start (0,0), level = steps, 8 directions
4	Word Ladder (Length)	Graph BFS (Distance)	Build graph, return level when end found
5	Nearest Exit in Maze	Grid BFS (Distance)	Exit = boundary cell, not entrance
6	01 Matrix	Multi-Source BFS	Push all 0's, update 1's with distance

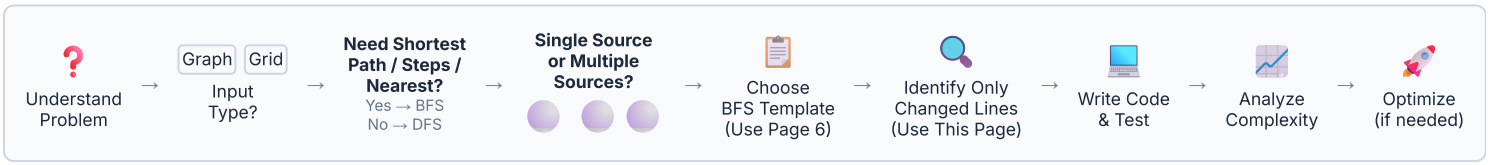
8 BFS TEMPLATES USED (FROM PAGE 6)

- **Gb** **Graph BFS (Adj List)** — Traverse graph level by level using queue.
- **Gr** **Grid BFS** — Traverse grid using 4/8 directions.
- **Ms** **Multi-Source BFS** — Start BFS from multiple sources at once.
- **Lv** **Level Order BFS** — Process nodes level by level using queue size.
- **Ds** **Distance BFS** — Level / steps represent shortest distance.

9 INTERVIEW CHECKLIST

- [x] Did I choose the correct BFS template?
- [x] Is my queue initialization correct?
- [x] Am I marking visited at the right time?
- [x] Is level / steps updated in the right place?
- [x] Did I handle all boundary & edge cases?
- [x] Is my return value correct?
- [x] What is the time & space complexity?
- [x] Can I optimize further?

BFS PROBLEM SOLVING FLOW (INTERVIEW APPROACH)



GRAPH PATTERN GUIDE

MASTER UNION FIND (DISJOINT SET) TEMPLATE GUIDE

"Merge sets. Detect cycles. Count components. Super fast."

1 MASTER IDEA

Union Find helps us manage disjoint sets.

It answers:
Are two nodes connected?

It helps in:
Cycle Detection
Components Count
Dynamic Connectivity

2 CORE OPERATIONS★★★★★

Operation	Purpose
find(x)	Find root/parent of x (With Path Compression)
union(a, b)	Merge two sets (With Union by Rank/Size)
connected(a, b)	Check if a and b belong to same set
components	Number of disjoint sets

TIP: Path Compression + Union by Rank makes operations almost O(1) (α(N)).

4 VARIANTS★★★★

Union by Size (Alternative)
Use size[] instead of rank[]
• Attach smaller tree under bigger tree
• Update size of new root

Path Compression (Essential)
Always compress path in find(x)
for almost O(1) performance.

5 MULTI-SET OPERATIONS★★★★

connected(a, b)	Same root?
union(a, b)	Merge sets
find(x)	Get root of x
components()	Count disjoint sets

Example (N = 6)

graph TD subgraph "Sets: {0,1,2,3}, {4,5}" 1((1)) --> 0((0)) 2((2)) --> 0 3((3)) --> 2 5((5)) --> 4((4)) end

Components = 2

3 UNION FIND TEMPLATE (OPTIMIZED)★★★★★

Java Template

```
class UnionFind {
    private int[] parent;
    private int[] rank;   // or use size[]
    private int count;    // number of components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    // Find with Path Compression
    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    // Union by Rank
    public boolean union(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
        if (rootA == rootB) return false; // already
connected

        if (rank[rootA] < rank[rootB]) parent[rootA] =
rootB;
        else if (rank[rootA] > rank[rootB])
parent[rootB] = rootA;
        else {
            parent[rootB] = rootA;
            rank[rootA]++;
        }
        count--;
        return true;
    }

    public boolean connected(int a, int b) {
        return find(a) == find(b);
    }

    public int components() {
        return count;
    }
}
```

AHA MOMENTS

- find(x) gives ultimate parent (root) of x.
- If two nodes have same root → they are connected.
- Union merges two different roots.
- Path Compression flattens trees.
- Union by Rank keeps trees shallow.
- Amortized time per operation is O(α(N)).

COMMON MISTAKES

- ❌ Forgetting path compression.
- ❌ Not using union by rank/size.
- ❌ Union even when already connected.

7 WHEN TO USE UNION FIND?★★★★★

Use When...

- ✅ You need to check if two nodes are connected.
- ✅ You need to detect a cycle in an undirected graph.
- ✅ You need to count number of components.
- ✅ You need to connect cells dynamically.
- ✅ You have many union/find queries.

Do NOT Use When...

- ❌ Graph is directed and you need cycle detection.
- ❌ You need shortest path.
- ❌ You need ordering of nodes.
- ❌ You need traversal / component details.

graph TD A[Need to manage sets?] —>|Yes| B[Need to check connection?] B —>|Yes| C[Use Union Find] B —>|No| D[Need order / shortest path?] D --
>|Yes| E[Use BFS / Dijkstra / Topo Sort] D —>|No| F[Other problem type]

- ❌ Wrong component count update.
- ❌ Indexing mistake (0 vs 1).

6 WHAT ACTUALLY CHANGES?★★★★

Problem Type	What We Do	How We Use Union Find	Example Problems
Cycle Detection (Undirected)	For each edge (u,v)	If connected(u,v) → cycle Else union(u,v)	Redundant Connection Graph Valid Tree
Cycle Detection (Directed)	Topological sort recommended	Not ideal for directed cycles	Course Schedule (use Topo Sort)
Components Count	For each edge (u,v)	union(u,v) always Return uf.components()	Number of Provinces
Max Area / Island	Union adjacent cells	Map cell → id, union neighbors	Making A Large Island
Dynamic Connectivity	Online union queries	Process union & connected	Friend Circles (online)

TIP: Problem decision depends on whether graph is directed, weighted and what to return.

8 COMPLEXITY★★★★★

find(x)	O(α(N))
union(a, b)	O(α(N))
connected(a, b)	O(α(N))
components()	O(1)

Where α(N) is Inverse Ackermann Function

- α(N) ≤ 4 for all practical N
- Nearly constant time!
- Best data structure for connectivity.

Space Complexity

O(N) for parent[] + rank[]

9 RETURN TYPE GUIDE★★★★

Return Type	Purpose	Typical Problems	Interview Meaning
boolean	Check if connected / cycle exists	Redundant Connection, Graph Valid Tree	Validity / Feasibility check
int	Count components	Number of Provinces	Count groups
void	Just perform unions	Build DSU then answer queries	Preprocessing / Setup
List / Set	List of components	Connected Components (Advanced)	Return actual groups

10 COMMON INTERVIEW MISTAKES★★★★

- ❌ Forgetting path compression.
- ❌ Union without checking roots.
- ❌ Wrong component count.
- ❌ Not handling 0-index / 1-index.
- ❌ Not initializing parent[i] = i.
- ❌ Using DSU for directed cycle.
- ❌ Using DSU for shortest path.
- ❌ Forgetting edge cases.

11 INTERVIEW THINKING FLOW★★★★★

graph LR A((?)) →|Yes| B((C)) B →|Yes| C((88)) C →|Yes| D[Choose Union Find Template] D → E[Write Code] E → F[Dry Run] F → G[Optimize]

12 PAGE 11 PREVIEW★★★★★

Next page contains UNION FIND PROBLEM GUIDE.
For each problem, you will see:

Base Template → Changed Lines → AHA Moment → Complexity → Interview Tip

Apply these templates with tiny changes and solve any Union Find problem with confidence!

PAGE 11

📖 GRAPH PATTERN GUIDE

PAGE 11 — UNION FIND PROBLEM GUIDE (APPLY TEMPLATES)

"Same idea. Different problems. Only the changing parts."

HOW TO USE THIS PAGE

1. Find the problem.
2. Identify the Base Template
3. (from Page 10).
4. **Read the CHANGED LINES.**
5. Understand the AHA.
6. Review complexity & interview tip.
7. Practice writing from memory.

🎯

TEMPLATE LEGEND (From Page 10)

- **B** — **Basic UF** — Find + Union by Rank
- **C** — **Cycle Detection** (Undirected)
- **D** — **Number of** Components
- **I** — **Max Area / Island** (Union Adjacent Cells)
- **P** — **Dynamic** Connectivity (Online Queries)

1 REDUNDANT CONNECTION [C]

Base Template: Cycle Detection (Undirected)

Problem
Find the edge that creates a cycle (first redundant edge).

Changed Lines (from Template)

```
// For each edge
for (int[] e : edges) {
    int u = e[0], v = e[1];
    if (connected(u, v)) return e;  //
cycle found
    union(u, v);
}
return new int[0];
```

Caller	Return	Key Idea
findRedundantConnection(edges)	**int[2] (edge)**	If already connected, this edge is extra.

💡 **AHA:** The first edge connecting two already connected nodes forms a cycle.

Complexity
Time: O(N α(N))
Space: O(N)

Interview Tip
Stop and return immediately when cycle found.

2 GRAPH VALID TREE [C]

Base Template: Cycle Detection + Components Count

Problem
Check if edges form a valid tree.

Changed Lines (from Template)

```
// n = number of nodes
if (edges.length != n - 1) return
false;  // must have n-1 edges

for (int[] e : edges) {
    int u = e[0], v = e[1];
    if (connected(u, v)) return false;
// cycle
    union(u, v);
}
return count == 1;   // single
component
```

Caller	Return	Key Idea
validTree(n, edges)	**boolean**	No cycle + single component

💡 **AHA:** Tree = (n-1 edges) + (No cycle) + (Connected)

Complexity
Time: O(N α(N))
Space: O(N)

Interview Tip
Always check edges.length == n - 1 first.

3 NUMBER OF PROVINCES [D]

Base Template: Components Count

Problem
Count how many connected components (provinces) exist.

Changed Lines (from Template)

```
// isConnected[i][j] = 1 if i and j are
connected
int n = isConnected.length;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (isConnected[i][j] == 1)
union(i, j);
    }
}
return count;   // number of components
```

Caller	Return	Key Idea
findCircleNum(isConnected)	**int**	Union every connected pair, return count

💡 **AHA:** Each component represents one province.

Complexity
Time: O(N^2 α(N))
Space: O(N)

Interview Tip
Matrix input → check only upper triangle.

4 SURROUNDED REGIONS I

Base Template: Max Area / Island (Union Adjacent Cells)

Problem

Flip all 'O' that are surrounded by 'X' to 'X'. 'O' connected to boundary stays 'O'.

Changed Lines (from Template)

```
// 1. Add a dummy node: dummy = m*n
// 2. Union boundary 'O' with dummy
// 3. Union all 'O' with 4 neighbors
// 4. Final: if connected(cell, dummy)
-> keep 'O' else -> 'X'
```

Caller	Return	Key Idea
solve(board)	**void**	Boundary safe 'O' connected to dummy node

💡 **AHA:** Dummy node represents all boundary 'O'.

Complexity
Time: O(M*N α(M*N))
Space: O(M*N)

Interview Tip
Process boundary first, then inner cells.

5 MAKING A LARGE ISLAND I

Base Template: Max Area / Island

Problem

Flip at most one 0 to 1 to get the largest island size.

Changed Lines (from Template)

```
// 1. Union all existing 1's and calc
size of each root
// 2. For each 0 cell, try making it 1
//    Take unique neighbor roots, sum
their sizes + 1
```

Caller	Return	Key Idea
largestIsland(grid)	**int**	Merge unique neighboring islands around 0

💡 **AHA:** Use Set to avoid double counting same island.

Complexity
Time: O(M*N α(M*N))
Space: O(M*N)

Interview Tip
Don't forget +1 for the flipped cell itself.

6 NUMBER OF OPERATIONS TO CONNECT NETWORK C

Base Template: Components Count

Problem

Return minimum operations to connect all computers.

Changed Lines (from Template)

```
// After processing all connections
int components = count; // number of
components
int extra = connections.length - (n -
components);
// extra edges available
return (extra >= components - 1) ?
components - 1 : -1;
```

Caller	Return	Key Idea
makeConnected(n, connections)	**int**	Need (components-1) extra edges

💡 **AHA:** Extra edges = totalEdges - (n - components)

Complexity
Time: O(N α(N))
Space: O(N)

Interview Tip
Check if enough extra edges exist.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	What Changes
1	**Redundant Connection**	Cycle Detection	Return edge when connected(u,v) is true
2	**Graph Valid Tree**	Cycle + Components	Edges must be n-1 & count must be 1
3	**Number of Provinces**	Components Count	Union all connected pairs in matrix
4	**Surrounded Regions**	Max Area / Island	Use dummy node for boundary 'O'
5	**Making a Large Island**	Max Area / Island	Try flipping 0, merge unique neighbor roots
6	**Make Network Connected**	Components Count	Use extra edges to connect components

8 UNION FIND TEMPLATE RECAP (From Page 10)

Find with Path Compression

```
if (parent[x] != x)
    parent[x] = find(parent[x]);
return parent[x];
```

Union by Rank

```
if (rank[a] < rank[b]) parent[a] = b;
else if (rank[a] > rank[b]) parent[b] =
a;
else { parent[b] = a; rank[a]++; }
```

9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
boolean	Check cycle / Validity	Graph Valid Tree
int	Count / Min Ops / Size	Provinces, Operations to Connect, Island Size
int[]	Return edge / Pair	Redundant Connection
void	Modify structure (in-place)	Surrounded Regions

10 INTERVIEW CHECKLIST

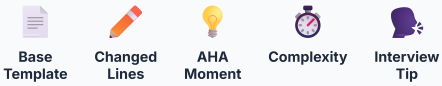
- ☑ Did I initialize `parent[i] = i` and `rank[i] = 1`?
- ☑ Did I use path compression in `find()`?
- ☑ Did I union by rank/size?
- ☑ Am I checking `connected(u, v)` correctly?
- ☑ Am I handling 0-index / 1-index properly?
- ☑ Did I think about components count?
- ☑ Any edge cases (single node, no edges)?
- ☑ Is my time complexity O(α(N)) or O(N α(N))?

11 INTERVIEW THINKING FLOW

12 PAGE 12 PREVIEW

Next page contains DIJKSTRA (SHORTEST PATH) TEMPLATE GUIDE.

For each problem, you will see:



Apply these templates with tiny changes and solve any Dijkstra problem with confidence!

Problem (Understand) → **Input** (What given) → **Representation** (Graph) → **Helper Structure** (Parent[], Rank[]) → **Pattern** (Union Find) → **Tiny Logic Change** (Apply Changes) → **Dry Run** (Example) → **Optimize** (Time/Space)

GRAPH PATTERN GUIDE

MASTER DIJKSTRA (SHORTEST PATH) TEMPLATE GUIDE

"Optimized for non-negative weights."

1 MASTER IDEA

Dijkstra finds the shortest path from a source to all other nodes in a weighted graph with **non-negative weights**.

Always pick the node with the **smallest known distance**.

Once picked, its distance is **final**.

2 CORE COMPONENTS

Component	Purpose
dist[]	Shortest known distance from source to each node
Priority Queue	Pick node with minimum distance (Min Heap)
visited[] / inPQ[]	Avoid processing a node multiple times
Relaxation	If better path found, update distance
Parent[] (opt.)	To reconstruct shortest path

TIP: Dijkstra does NOT work with negative edge weights.

4 DIJKSTRA TEMPLATE (ADJACENCY MATRIX) ★★★★

Java Template

```
public static int[] dijkstraMatrix(int[][] mat,
int n, int src) {
    int[] dist = new int[n];
    boolean[] visited = new boolean[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        // Pick min dist unvisited node
        int u = -1, min = Integer.MAX_VALUE;
        for (int i = 0; i < n; i++)
            if (!visited[i] && dist[i] < min) {
min = dist[i]; u = i; }

        visited[u] = true;

        // Relax all neighbors of u
        for (int v = 0; v < n; v++) {
            if (mat[u][v] > 0 && !visited[v] &&
dist[u] != Integer.MAX_VALUE
                && dist[u] + mat[u][v] < dist[v])
{
                dist[v] = dist[u] + mat[u][v];
            }
        }
    }
    return dist;
}
```

3 DIJKSTRA TEMPLATE (ADJACENCY LIST) ★★★★★

Java Template

```
class Dijkstra {
    static class Pair {
        int node, dist;
        Pair(int n, int d) { node =
n; dist = d; }
    }

    public static int[]
dijkstra(List<List<Pair>> graph, int
n, int src) {
        int[] dist = new int[n];
        Arrays.fill(dist,
Integer.MAX_VALUE);
        dist[src] = 0;

        PriorityQueue<Pair> pq = new
PriorityQueue<>((a, b) -> a.dist -
b.dist);
        pq.offer(new Pair(src, 0));

        boolean[] visited = new
boolean[n];
        while (!pq.isEmpty()) {
            Pair cur = pq.poll();
            int u = cur.node, d =
cur.dist;
            if (visited[u]) continue;
// already finalized
            visited[u] = true;

            for (Pair nei :
graph.get(u)) {
                int v = nei.node, w =
nei.dist;
                if (d + w < dist[v])
{    // relax
                    dist[v] = d + w;
                    pq.offer(new
Pair(v, dist[v]));
                }
            }
        }
        return dist; // dist[i] =
shortest distance from src to i
    }
}
```

AHA MOMENTS

- We always expand the closest unprocessed node.
- First time a node is popped from PQ, its distance is final.
- Relaxation improves neighbors only.
- Work only for non-negative weights.
- Same idea used in GPS navigation!

COMMON MISTAKES

- ❌ Using Dijkstra with negative edges.
- ❌ Forgetting to check if node already visited.
- ❌ Not using long when dist can overflow (use long for large sums).
- ❌ Relaxing from outdated distance.
- ❌ Pushing same node many times without visited check.

7 WHEN TO USE DIJKSTRA?

Use When...

- ✅ All edge weights are non-negative.
- ✅ Need shortest path from one source to all nodes.
- ✅ Graph can be directed or undirected.
- ✅ Need fastest reliable algorithm.

Do NOT Use When...

- ❌ Graph has negative weight edges (Use Bellman-Ford).
- ❌ Need shortest path between all pairs (Use Floyd-Warshall).
- ❌ Graph is unweighted (Use BFS).

Decision Flow

graph TD A[Edge Weights?] A -- All Non-Negative --> B[Single Source Shortest Path] B --> C[Use Dijkstra] A -- Any Negative? --> D[Use Bellman-Ford (or Johnson's)] style C fill:#dcfce7,stroke:#16a34a style D fill:#fee2e2,stroke:#dc2626

10 MINI EXAMPLE (ADJ LIST)

Graph (Undirected)

graph LR 0((0)) -- 4 --- 1((1)) 0 -- 1 --- 2((2)) 1 -- 2 --- 2 1 -- 1 --- 3((3)) 2 -- 5 --- 3 2 -- 8 --- 4((4)) 3 -- 3 --- 4

Source = 0
Dijkstra Steps:

1. dist = [0, ∞, ∞, ∞, ∞]
   PQ = [(0,0)]
2. Pop **0** → relax 1(4), 2(1)
   dist = [0, **4**, 1, ∞, ∞]
3. Pop **2** → relax 1(3), 3(6), 4(9)
   dist = [0, 3, 1, 6, 9]
4. Pop **1** → relax 3(4) (better)
   dist = [0, 3, 1, 4, 9]
5. Pop **3** → relax 4(7) (better)
   dist = [0, 3, 1, 4, 7]
6. Pop **4** → done.

Final dist[] = [0, 3, 1, 4, 7]

5 WHAT CHANGES? ★★★★

Scenario	Change	Only This Part Changes
Different Source	Change src	Initialization dist[src] = 0
Get Path	Add parent[]	When relaxing, set parent[v] = u
Count Paths	Add count[]	If equal dist found, count[v] += count[u]
Min Cost with Stops Limit	Use (node, dist, stops) in PQ	Push with stops+1, check limit
Multiple Queries	Run Dijkstra multiple times	Reuse graph
Undirected Graph	Add both directions in adjacency	Graph build step

6 RETURN TYPE GUIDE ★★★★

Return Type	Use When	Example Problems
int[] dist	Need shortest distance from src to all nodes	Network Delay Time, Dijkstra, Path With Minimum Effort (variant)
long[] dist	Distances can be large (sum > 2^31-1)	Large graphs with big weights
Pair (dist, path)	Need actual path reconstruction	Shortest Path in Weighted Graph
boolean (reachable)	Only check if target is reachable	Is Path Exists with Min Cost ≤ K
Custom Object	Need dist + other info (stops, ways, etc.)	Number of Ways to Arrive at Destination

TIP: For large values use **long**, not **int**.

8 COMPLEXITY ★★★★

Adjacency List + PQ (Best Implementation)

Time: **O((V + E) log V)**
Space: **O(V + E)**

Adjacency Matrix (Simple Implementation)

Time: **O(V^2)**
Space: **O(V^2)**

graph LR 0((0)) -- 4 --- 1((1)) 0 -- 2 --- 2((2)) 1 -- 0 --- 2 1 -- 3 --- 3((3)) 2 -- 3 --- 3

	0	1	2	3
0	0	4	2	0
1	4	0	0	3
2	2	0	0	3
3	0	3	3	0

Where:
V = number of vertices
E = number of edges

9 DIJKSTRA SUMMARY

#	Step	What Happens
1	**Initialize**	dist[src] = 0, others = INF Push (src, 0) in PQ
2	**Extract Min**	Pop node u with smallest dist
3	**Skip if Visited**	If already visited, continue
4	**Relax Neighbors**	For each neighbor v of u, update dist[v] if better path
5	**Repeat**	Continue until PQ is empty
6	**Done**	dist[] has shortest paths

11 INTERVIEW CHECKLIST

- [x] Did I build the graph correctly?
- [x] Did I initialize dist[] correctly?
- [x] Did I use Min Priority Queue?
- [x] Did I check visited[] before relaxing?
- [x] Did I relax all neighbors?
- [x] Did I update and push to PQ only when better?
- [x] Did I handle large values (use long if needed)?
- [x] Did I think about path reconstruction if asked?
- [x] Did I return correct type and values?
- [x] Can I dry run on a small example?

⭐ Interview Tip

Always communicate:
• What the algo does
• Why it works
• Complexity
• How you handle edge cases

12 PAGE 13 PREVIEW

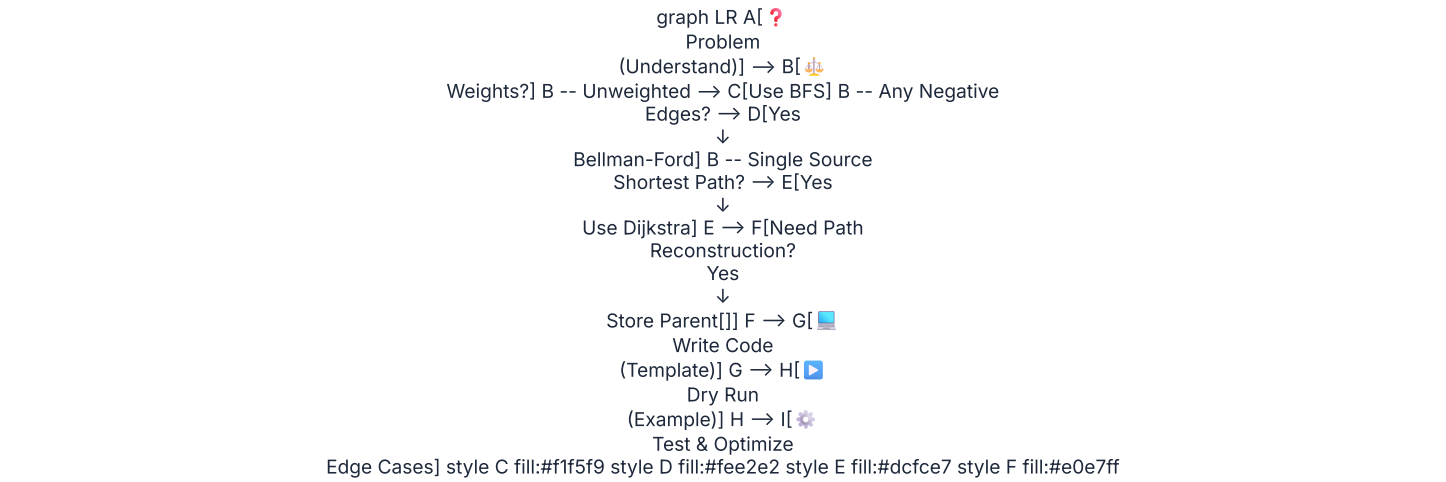
Next page contains DIJKSTRA PROBLEM GUIDE.

For each problem, you will see:

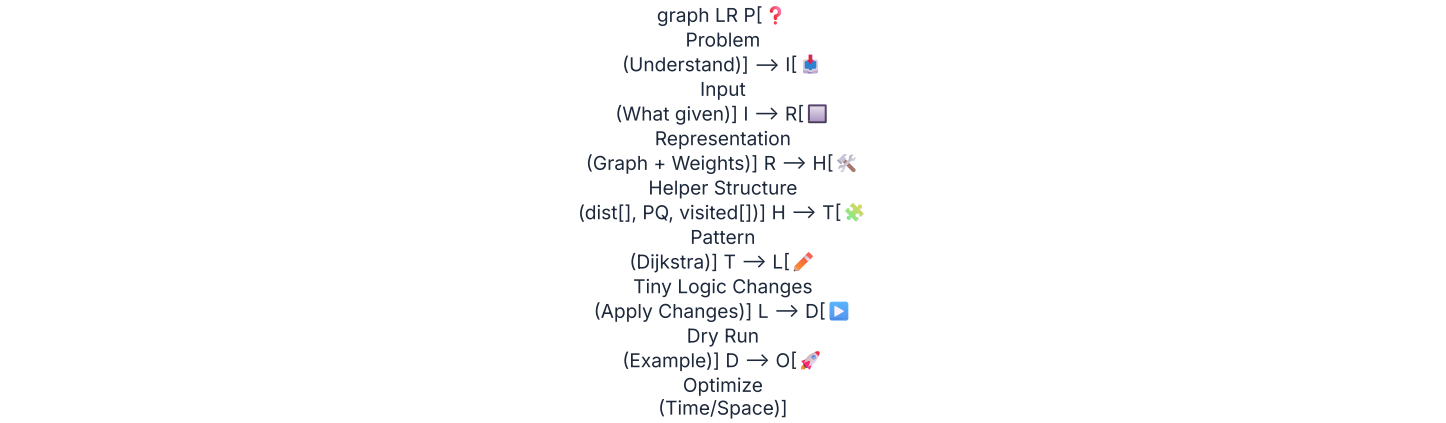
Base Template → Changed Lines → AHA Moment → Complexity → Interview Tip

Apply these templates with tiny changes and solve any Dijkstra problem with confidence!

13 INTERVIEW THINKING FLOW



CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)



PAGE 15

GRAPH PATTERN GUIDE

📖 PAGE 15 — SPECIAL GRAPH PROBLEMS GUIDE (APPLY TEMPLATES)

"These problems test deeper understanding + clean thinking."

HOW TO USE THIS PAGE

① Find the problem.
② Identify the base template.
③ Read the **CHANGED LINES**.
④ Understand the **AHA**.
⑤ Review complexity & interview tip.
⑥ Practice writing from memory.

TEMPLATE LEGEND (From Page 14)

- **A** Network Delay (Dijkstra)
- **B** Dijkstra with Path
- **C** Minimum Effort Path
- **D** Cheapest Flights (K Stops)
- **E** Path with Minimum Effort
- **F** Shortest Path in Undirected Graph

1 CLONE GRAPH — S

Base Template: DFS (Graph Traversal)

Problem
Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph.

Changed Lines (from Template)

```
Map<Node, Node> map = new HashMap<>();
private Node clone(Node node) {
  if (node == null) return null;
  if (map.containsKey(node)) return map.get(node); // already cloned
  Node copy = new Node(node.val);
// clone current
  map.put(node, copy);
  for (Node nei : node.neighbors) {
    copy.neighbors.add(clone(nei));
  }
  return copy;
}
```

Caller	Return	Key Idea
`clone(node)`	`Node`	Use map to avoid infinite loop & duplicates

💡 **AHA:** Store cloned node before traversing neighbors.

Complexity
Time: O(V + E)
Space: O(V)

Interview Tip
Essentially a graph + hash map problem.

2 SURROUNDED REGIONS — I

Base Template: DFS from Boundary

Problem
Capture all regions that are surrounded by 'X'. Regions connected to boundary stay 'O'.

Changed Lines (from Template)

```
// 1. DFS from all boundary 'O'
for (int i = 0; i < m; i++) {
  dfs(i, 0); dfs(i, n-1);
}
for (int j = 0; j < n; j++) {
  dfs(0, j); dfs(m-1, j);
}
// 2. Flip inside 'O' to 'X'
for (int i = 0; i < m; i++) {
  for (int j = 0; j < n; j++) {
    if (board[i][j] == 'O') board[i][j] = 'X';
    if (board[i][j] == 'E') board[i][j] = 'O';
  }
}
// dfs() marks reachable 'O' as 'E'
```

Caller	Return	Key Idea
`solve(board)`	`void`	Mark safe 'O's first, then flip the rest

💡 **AHA:** Only boundary-connected 'O's are safe.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Classic boundary DFS pattern.

3 PACIFIC ATLANTIC WATER FLOW — I

Base Template: Multi-Source BFS / DFS

Problem
Find all cells where water can flow to both Pacific (top/left) and Atlantic (bottom/right) oceans.

Changed Lines (from Template)

```
// 1. BFS/DFS from all Pacific borders (mark visitP)
// 2. BFS/DFS from all Atlantic borders (mark visitA)
// 3. Cells reachable in both → answer
// Condition: move to neighbor with ⩾ height
public List<List<Integer>> pacificAtlantic(int[][] h) {
  int m = h.length, n = h[0].length;
  boolean[][] p = new boolean[m][n];
  boolean[][] a = new boolean[m][n];
  // bfs(pacific) & bfs(atlantic)
  // collect cells where p[i][j] && a[i][j]
}
```

Caller	Return	Key Idea
`pacificAtlantic(h)`	`List<List<Integer>>`	Reverse flow: ocean → cells

💡 **AHA:** Think reverse - water flows from ocean to land.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Two traversals + intersection.

4 ROTTING ORANGES I

Base Template: Multi-Source BFS

Problem
Each minute, a rotten orange makes adjacent fresh oranges rotten. Return time to rot all or -1.

Changed Lines (from Template)

```
Queue<int[]> q = new LinkedList<>
();
int fresh = 0, time = 0;
// 1. Add all rotten oranges to
queue & count fresh
// 2. BFS level by level
while (!q.isEmpty() && fresh > 0) {
  int size = q.size();
  time++;
  while (size-- > 0) {
    int[] cur = q.poll();
    // infect fresh neighbors
  }
}
return fresh == 0 ? time : -1;
```

Caller	Return	Key Idea
`orangesRotting(grid)`	`int`	Level order = minutes

💡 **AHA:** Processing one level = one minute.

Complexity
Time: O(m * n)
Space: O(m * n)

Interview Tip
Classic BFS with multi-source.

5 COURSE SCHEDULE II K

Base Template: Topological Sort (BFS)

Problem
Return any valid order to finish all courses. If impossible, return [].

Changed Lines (from Template)

```
// Instead of just checking cycle,
store order
List<Integer> order = new
ArrayList<>();
Queue<Integer> q = new LinkedList<>
();

// push all indegree 0
while (!q.isEmpty()) {
  int node = q.poll();
  order.add(node);
  for (int nei : graph.get(node)) {
    indegree[nei]--;
    if (indegree[nei] == 0)
q.offer(nei);
  }
}
return order.size() == numCourses ?
order : new ArrayList<>();
```

Caller	Return	Key Idea
`findOrder(n, prerequisites)`	`int[]`	If order size != n, cycle exists

💡 **AHA:** Topo sort can produce order, not just detect cycle.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
BFS topo sort is iterative & safe.

6 GRAPH VALID TREE K

Base Template: Union Find / DFS

Problem
Determine if an undirected graph is a valid tree.

Changed Lines (from Template)

```
// For Union Find
if (edges.length != n - 1) return
false;
for (int[] e : edges) {
  if (union(e[0], e[1])) return
false; // cycle
}
return true;

// OR DFS: Check connected + no
cycle
```

Caller	Return	Key Idea
`validTree(n, edges)`	`boolean`	Tree = (n-1 edges) + connected + no cycle

💡 **AHA:** Quick checks save time: edges == n-1.

Complexity
Time: O(V + E)
Space: O(V)

Interview Tip
Union Find is fastest approach.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	Key Change
1	Clone Graph	DFS	Use HashMap old->new to avoid re-visit
2	Surrounded Regions	DFS from Boundary	Mark boundary 'O's first, then flip
3	Pacific Atlantic	Multi-Source BFS/DFS	Reverse flow from ocean to land
4	Rotting Oranges	Multi-Source BFS	Level order = minutes
5	Course Schedule II	Topo Sort (BFS)	Store order while doing BFS
6	Graph Valid Tree	Union Find / DFS	Edges == n-1 & no cycle & connected

8 COMPLEXITY QUICK GUIDE

Pattern	Time Complexity	Space Complexity
BFS / DFS (Traversal)	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V)
Multi-Source BFS	O(V + E)	O(V)
Topo Sort (BFS)	O(V + E)	O(V)
Union Find	O(E α(V))	O(V)

α(V) = Inverse Ackermann (almost constant)

9 RETURN TYPE GUIDE

Pattern	Return Type	Use When	Example Problems
List<List<Integer>>	List of cells	Collect multiple cells	Pacific Atlantic
int	Value	Time, count, min/max	Rotting Oranges
int[] / List<Integer>	Order / Path	Return order or path	Course Schedule II
boolean	True / False	Validity / existence	Valid Tree
Node	Graph Node	Clone / Copy graph	Clone Graph

10 INTERVIEW CHECKLIST

- ☑ Did I understand the problem clearly?
- ☑ Did I pick the right base template?
- ☑ Did I change only what is needed?
- ☑ Did I handle edge cases?
- ☑ Did I analyze time & space?

☑ Can I explain my approach aloud?
☑ Can I implement without help?

11 INTERVIEW THINKING FLOW

Understand Problem → Identify Graph Type → Pick Base Template → Apply Changes from Template → Test on Examples & Edge Cases → Optimize & Explain

TIP: Before coding - explain your plan. Communication matters!

12 PAGE 16 PREVIEW

Next page contains **GRAPH PATTERN CHEAT SHEETS.**
For each pattern, you will see:

Pattern Summary | When to Use | Key Steps | Template Snapshot | Common Pitfalls | Complexity | Interview Tip

One-page cheatsheets. Quick recall before interviews!

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)

Problem (Understand) → Input (What given) → Representation (Graph + Weights) → Choose Pattern (Best Fit) → Helper Structure (PQ / Visited / DSU) → Logic Change (Apply Minimal) → Dry Run (Example) → Optimize (Time / Space)

GRAPH PATTERN GUIDE

ADVANCED GRAPH PATTERNS GUIDE (APPLY TEMPLATES)
(FAANG Cheat Sheet)

"Advanced patterns = smart modeling + right algorithm + clean implementation."

HOW TO USE THIS PAGE

1. Find the problem.
2. Identify the base template (from Page 13).
3. Read the **CHANGED LINES**.
4. Understand the **AHA**.
5. Review complexity & interview tip.
6. Practice writing from memory.

TEMPLATE LEGEND (From Page 13)

- **A** Network Delay Time
- **B** Dijkstra with Path
- **C** Minimum Effort Path
- **D** Cheapest Flights (K Stops)
- **E** Path with Minimum Effort
- **F** Shortest Path in Undirected Graph

1 SWIM IN RISING WATER — C

Base Template: Dijkstra (Minimize Max Edge)

Problem
Find minimum time to reach bottom-right cell.
Water level must be ≥ max height on the path.

Changed Lines (from Template)

```
int n = heights.length;
int[][] dirs = {{1,0}, {-1,0}, {0,1},
{0,-1}};
PriorityQueue<int[]> pq = new
PriorityQueue<>(
    (a,b) → a[0] - b[0]); // effort,
r, c
pq.offer(new int[]{heights[0][0], 0,
0});
int[][] effort = new int[n][n];
for (int[] row : effort)
Arrays.fill(row, Integer.MAX_VALUE);
effort[0][0] = heights[0][0];

// When relaxing neighbor (nr, nc)
int newEff = Math.max(curEff,
heights[nr][nc]);
if (newEff < effort[nr][nc]) {
    effort[nr][nc] = newEff;
    pq.offer(new int[]{newEff, nr,
nc});
}
return effort[n-1][n-1];
```

Caller	Return	Key Idea
swimInWater(grid)	int	Minimize maximum edge on path

💡 **AHA:** Path cost = max(height) on the path.

Complexity
Time: $O(N^2 \log N)$
Space: $O(N^2)$

Interview Tip
Use Dijkstra with max() relaxation.

2 PATH WITH MAX PROBABILITY — B

Base Template: Dijkstra + Parent[]

Problem
Given success probability on edges, find path from start to end with maximum probability.

Changed Lines (from Template)

```
double[] prob = new double[n];
Arrays.fill(prob, 0.0);
prob[start] = 1.0;
PriorityQueue<double[]> pq = new
PriorityQueue<>(
    (a,b) → Double.compare(b[0],
a[0])); // max-heap
pq.offer(new double[]{1.0, start});

// When relaxing
double newProb = prob[node] *
edgeProb;
if (newProb > prob[next]) {
    prob[next] = newProb;
    parent[next] = node;
    pq.offer(new double[]{newProb,
next});
}
return prob[end];
```

Caller	Return	Key Idea
maxProbability(n, edges, succProb, start, end)	double	Maximize product of probabilities

💡 **AHA:** Use max-heap. Multiply, don't add.

Complexity
Time: $O((V+E) \log V)$
Space: $O(V + E)$

Interview Tip
Initialize prob[start] = 1.0
All others = 0.0

3 MINIMUM MULTIPLICATION TO REACH END — A

Base Template: Dijkstra (Single Source)

Problem
Given multipliers and modulo, find min multiplications to reach end.

Changed Lines (from Template)

```
PriorityQueue<int[]> pq = new
PriorityQueue<>((a,b) → a[0]-b[0]);
int[] dist = new int[100000];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[start] = 0;
pq.offer(new int[]{0, start});
while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int steps = cur[0], node =
cur[1];
    if (node == end) return steps;
    for (int x : arr) {
        int next = (node * x) %
100000;
        if (steps + 1 < dist[next]) {
            dist[next] = steps + 1;
            pq.offer(new int[]
{steps+1, next});
        }
    }
}
return -1;
```

Caller	Return	Key Idea
minimumMultiplications(arr, start, end)	int	Dijkstra on mod space graph

💡 **AHA:** Nodes are numbers [0 ... mod-1].

Complexity
Time: O(mod * K log mod)
Space: O(mod)

Interview Tip
Use mod graph. Avoid revisiting with larger steps.

4 MOST STONES REMOVED WITH SAME ROW/COL F

Base Template: Connected Components (DSU)

Problem
Stones are connected if they share row or column.
Return max stones removable.

Changed Lines (from Template)

```
// Map rows and columns to DSU nodes
int base = 10001; // shift for
columns
DSU dsu = new DSU(base * 2);
for (int[] s : stones) {
    int r = s[0];
    int c = s[1] + base;
    dsu.union(r, c);
    rowSet.add(r);
    colSet.add(c);
}
int components = 0;
for (int r : rowSet) if (dsu.find(r)
== r) components++;
for (int c : colSet) if (dsu.find(c)
== c) components++;
return stones.length - components;
```

Caller	Return	Key Idea
removeStones(stones)	int	Answer = stones - number of components

💡 **AHA:** Make virtual nodes for rows and columns.

Complexity
Time: O(N α(N))
Space: O(N)

Interview Tip
DSU beats DFS/BFS for large N.

5 COURSE SCHEDULE II (FIND ORDER) F

Base Template: Topological Sort (Kahn's BFS)

Problem
Return any valid order to finish all courses. If impossible, return [].

Changed Lines (from Template)

```
Queue<Integer> q = new LinkedList<>
();
for (int i = 0; i < n; i++)
    if (indegree[i] == 0) q.offer(i);

List<Integer> order = new ArrayList<>
();
while (!q.isEmpty()) {
    int node = q.poll();
    order.add(node);
    for (int nei : graph.get(node)) {
        if (--indegree[nei] == 0)
q.offer(nei);
    }
}
return order.size() == n ? order :
new int[0];
```

Caller	Return	Key Idea
findOrder(numCourses, prerequisites)	int[]	Topological order using BFS

💡 **AHA:** If order size < n, cycle exists.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Kahn's BFS is iterative and safe.

6 EVENTUAL SAFE STATES F

Base Template: Topological Sort (Reverse Graph)

Problem
Return all nodes that are NOT part of any cycle.

Changed Lines (from Template)

```
// Build reverse graph
for (int u = 0; u < n; u++)
    for (int v : graph[u]) {
        revGraph.get(v).add(u);
        outdegree[u]++;
    }
// Start BFS with terminal nodes
(outdegree 0)
for (int i = 0; i < n; i++)
    if (outdegree[i] == 0)
q.offer(i);
List<Integer> safe = new ArrayList<>
();
while (!q.isEmpty()) {
    int node = q.poll();
    safe.add(node);
    for (int pre :
revGraph.get(node)) {
        if (--outdegree[pre] == 0)
q.offer(pre);
    }
}
Collections.sort(safe);
return safe;
```

Caller	Return	Key Idea
eventualSafeNodes(graph)	List<Integer>	Reverse BFS from terminal nodes

💡 **AHA:** Nodes not in any cycle become safe.

Complexity
Time: O(V + E)
Space: O(V + E)

Interview Tip
Reverse graph trick is key.

7 CHANGED LINES SUMMARY

#	Problem	Base Template	Main Changes
1	Swim in Rising Water	C	Use max() in relax; grid as graph
2	Path with Max Probability	B	Max-heap; multiply probabilities
3	Min Multiplications	A	Nodes = [0..mod-1]; steps + 1
4	Most Stones Removed	F (DSU)	Map rows/cols; components count
5	Course Schedule II	F (Topo)	Store order while BFS
6	Eventual Safe States	F (Topo Rev)	Reverse graph, start from terminals

8 COMPLEXITY QUICK GUIDE

Pattern	Time Complexity	Space Complexity
Dijkstra (Min/Max)	O((V+E) log V)	O(V + E)
Dijkstra (Adj List)	O((V+E) log V)	O(V + E)
DSU (Union Find)	O(N α(N))	O(N)
Topological Sort (BFS)	O(V + E)	O(V + E)
BFS / DFS	O(V + E)	O(V)

V = vertices, E = edges, α(N) is inverse Ackermann (~constant).

9 RETURN TYPE GUIDE

Pattern	Return Type	Use When
Shortest Distance	int / long	Min cost / steps
Path / Order	int[] / List<Integer>	Need sequence
Probability	double	Max probability path
Exist / Possible	boolean	Cycle / schedule
List of Nodes	List<Integer>	Safe nodes, result set

10 INTERVIEW CHECKLIST

- [x] Did I model the graph correctly?
- [x] Did I pick the right base template?
- [x] Did I initialize data structures correctly?
- [x] Did I handle edge cases (n=1, empty)?
- [x] Did I avoid overflow (long / mod)?
- [x] Is my complexity optimal?
- [x] Can I explain my approach clearly?

11 INTERVIEW THINKING FLOW

graph LR A((?)) --> B[Model Graph] B --> C[Identify Pattern] C --> D[Pick Template] D --> E[Apply & Write Code] E --> F[Test & Verify Edge Cases] F --> G((Optimize & Explain))

TIP: Always start with small example. Dry run on paper. Then code.

12 PAGE 15 PREVIEW

Next page contains SPECIAL GRAPH PROBLEMS GUIDE.

For each problem, you will see:

💡 Problem Idea | 🔍 Key Observations | 📄 Template Used | ✨ Trick / Twist | ⏱ Complexity | ⭐ Interview Tip

Master special problems. Solve any graph question in the interview!

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA) Problem (Understand) → Input (What given) → Representation (Graph + Weights) → Helper Structure (PQ / DSU / Queue) → Pattern (Choose Right One) → Tiny Logic Change (Apply to Problem) → Dry Run (Example) → Optimize (Time/Space)

GRAPH PATTERN GUIDE

DIJKSTRA PROBLEM GUIDE (APPLY TEMPLATES)

Same algorithm. Different problems. Only the changing parts.

HOW TO USE THIS PAGE

1. Find the problem.
2. Identify the base template (from Page 12).
3. **Read the CHANGED LINES.**
4. Understand the AHA.
5. Review complexity & interview tip.
6. Practice writing from memory.

TEMPLATE LEGEND (From Page 12)

A Network Delay Time (Single Source)

B Dijkstra with Path (Parent[])

C Minimum Effort Path (Minimize Max Edge)

D Cheapest Flights (K Stops)

E Path With Minimum Effort

F Find Shortest Path (Undirected Weighted)

1 NETWORK DELAY TIME A

Base Template: Dijkstra (Single Source Shortest Path)

Problem

Given a directed weighted graph, find time for all nodes to receive signal from node k. Return -1 if impossible.

Changed Lines (from Template)

```
// Build graph from times[u, v, w]
for (int[] t : times) {
    graph.get(t[0]).add(new int[]
{t[1], t[2]});
}
// Answer = max distance among all
nodes
int ans = 0;
for (int i = 1; i ⩽ n; i++) {
    if (dist[i] == INF) return -1;
    ans = Math.max(ans, dist[i]);
}
return ans;
```

Caller: networkDelayTime(times, n, k)

Return: int (time or -1)

Key Idea: All nodes receive signal?

💡 **AHA:** The last node to get the signal = max shortest distance.

Complexity
Time: O((V+E) log V)
Space: O(V+E)

Interview Tip
Don't forget to check unreachable nodes.

2 DIJKSTRA WITH PATH B

Base Template: Dijkstra + Parent[]

Problem

Return shortest distance and the actual path from src to dest.

Changed Lines (from Template)

```
int[] parent = new int[n];
Arrays.fill(parent, -1);
// When relaxing
if (dist[v] > dist[u] + w) {
    dist[v] = dist[u] + w;
    parent[v] = u; // track parent
    pq.offer(new int[]{v,
dist[v]});
}
// Reconstruct Path
List<Integer> path = new
ArrayList<>();
for (int at = dest; at ≠ -1; at =
parent[at])
    path.add(at);
Collections.reverse(path);
```

Caller: shortestPath(n, edges, src, dest)

Return: List<Integer> path

Key Idea: Track parent & reconstruct

💡 **AHA:** Parent[] builds the path while Dijkstra builds distances.

Complexity
Time: O((V+E) log V)
Space: O(V+E)

Interview Tip
Check if dest is reachable (dist[dest] == INF).

3 MINIMUM EFFORT PATH C

Base Template: Dijkstra (Minimize Max Edge)

Problem

In a grid, effort of path = max absolute diff between adjacent cells. Find minimum effort from top-left to bottom-right.

Changed Lines (from Template)

```
// weight = max effort along the
path
int w = Math.max(effort[u],
Math.abs(heights[x][y] -
heights[nx][ny]));

if (w < effort[nx][ny]) {
    effort[nx][ny] = w;
    pq.offer(new int[]{nx, ny,
w});
}
```

Caller: minimumEffortPath(heights)

Return: int (min effort)

Key Idea: Minimize max edge on path

💡 **AHA:** Instead of sum, we minimize the maximum edge.

Complexity
Time: O(m*n log(m*n))
Space: O(m*n)

Interview Tip
This is Dijkstra with custom relaxation.

4 CHEAPEST FLIGHTS WITHIN K STOPS D

Base Template: Dijkstra with Stops Limit

Problem

Find cheapest price from src to dst with at most k stops.

Changed Lines (from Template)

```
PriorityQueue<int[]> pq = new
PriorityQueue<>
(Comparator.comparingInt(a →
a[0]));
pq.offer(new int[]{0, src, 0}); //
cost, node, stops
while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int cost = cur[0], u = cur[1],
stops = cur[2];
    if (u == dst) return cost;
    if (stops > k) continue;
    for (int[] nei : graph.get(u))
{
        int v = nei[0], w =
nei[1];
        pq.offer(new int[]{cost +
w, v, stops + 1});
    }
}
return -1;
```

Caller: findCheapestPrice(n, flights, src, dst, k) **Return:** int (min cost or -1) **Key Idea:** State includes stops

💡 **AHA:** We need (cost, node, stops) state.

Complexity
Time: O(K*E log E)
Space: O(K*V)

Interview Tip
Do NOT use plain dist[]; stops make it different.

5 PATH WITH MINIMUM EFFORT E

Base Template: Dijkstra (Minimize Max Edge)

Problem

Same as #3 but emphasize general approach on any grid/graph where path cost = max edge cost.

Changed Lines (from Template)

```
// Generic for any graph
int newEffort =
Math.max(curEffort, edgeWeight);
if (newEffort < effort[v]) {
    effort[v] = newEffort;
    pq.offer(new int[]{v,
newEffort});
}
```

Caller: minEffortPath(graph, src, dst) **Return:** int (min effort) **Key Idea:** Minimize the maximum edge

💡 **AHA:** Works on grids, graphs, elevations, resistances.

Complexity
Time: O((V+E) log V)
Space: O(V+E)

Interview Tip
Use when path cost is NOT sum but maximum.

6 SHORTEST PATH IN UNDIRECTED WEIGHTED GRAPH F

Base Template: Dijkstra (Undirected)

Problem

Given an undirected weighted graph, return shortest distance from src to all nodes.

Changed Lines (from Template)

```
// Build undirected graph
graph.get(u).add(new int[]{v, w});
graph.get(v).add(new int[]{u, w});

// Rest same as base template
```

Caller: shortestPath(n, edges, src) **Return:** int[] dist **Key Idea:** Undirected edges both ways

💡 **AHA:** Only graph construction changes.

Complexity
Time: O((V+E) log V)
Space: O(V+E)

Interview Tip
Don't add duplicate edges.

7 CHANGED LINES SUMMARY

#	Problem	Template	Main Changes
1	Network Delay Time	A	Answer = max(dist[]), check unreachable
2	Dijkstra with Path	B	parent[] track + reconstruct path
3	Minimum Effort Path	C	Relax using max(edge, curEffort)
4	Cheapest Flights (K Stops)	D	State = (cost, node, stops)
5	Path with Minimum Effort	E	Minimize maximum edge on path
6	Undirected Weighted Graph	F	Add edges both directions

8 Dijkstra Template Recap (From Page 12)

Core Code (Adjacency List)

1. Initialize dist[] = INF, src = 0
2. PQ = (dist, node), push (0, src)
3. While pq not empty:
   a. Pop (d, u)
   b. If visited[u] continue
   c. For each neighbor (v, w):
      if dist[v] > d + w → update & push
4. Return dist[]

Works for all above problems with small changes.

9 RETURN TYPE GUIDE

Return Type	Use When	Example Problems
int[] dist	All-pairs from single source	1, 6
int	Need final value (max / min effort)	1, 3, 5
List<Integer> path	Need actual shortest path	2
int (cost)	Cheapest with constraints (K stops)	4
Custom (dist + path)	Need both distance & path	2

10 INTERVIEW CHECKLIST

- ✅ Did I initialize dist[src] = 0 correctly?
- ✅ Did I use PriorityQueue (min-heap) correctly?
- ✅ Did I skip if visited node is already processed?
- ✅ Did I relax correctly (dist[v] > dist[u] + w)?
- ✅ Did I handle unreachable nodes?
- ✅ Did I handle special constraint (stops / max edge)?
- ✅ Is my time complexity O((V+E) log V)?

⭐ Interview Tip

Relaxation is the heart.

PQ always gives the next closest node.

Small changes solve different problems!

11 INTERVIEW THINKING FLOW

graph TD A[Do all edges have non-negative weights?] -- YES --> B[Need exact shortest distance?] B -- NO --> C[Use Dijkstra Template] B -- YES --> D[Any limit on stops/edges?] D -- NO --> C D -- YES --> E[Use Dijkstra with Constraint e.g. K Stops] A -- NO --> F[Use Bellman-Ford Allow negative] C --> G[Re-evaluate Problem] E --> G

12 PAGE 14 PREVIEW

Next page contains ADVANCED GRAPH PATTERNS GUIDE.

For each pattern, you will see:

⚙️ Pattern Idea | 📋 Template Snapshot | 🧩 Common Problems | 🔑 Key Variations | 💡 Interview Tips

Master patterns. Solve any graph problem in the interview!

❓ **Problem** (Understand) → 📥 **Input** (What given) → **Representation** (Graph + Weights) → 🎯 **Choose Pattern** (Dijkstra Variant) → 🛠️ **Helper Structure** (PQ + dist[] + others) → 🖍️ **Apply Changes** (Per Problem) → ▶️ **Dry Run** (Example) → ⏱️ **Optimize** (Time/Space)

PAGE 16

📖 GRAPH PATTERN GUIDE

PAGE 16 — GRAPH CHEAT SHEETS & QUICK REFERENCE

"Fast recall. Confident coding. Crack any graph problem."

HOW TO USE THIS PAGE

1. Use this page for quick recall.
2. Before coding, pick the right template.
3. Check complexity & edge cases.
4. Use interview tips to avoid mistakes.
5. Keep practicing. Patterns become reflex.
6. You've got the roadmap. Keep going!

🎯

TEMPLATE QUICK INDEX

1 ADJACENCY LIST (GRAPH BUILDING)

```
// n = number of nodes (0 to n-1)
List<List<Integer>> buildGraph(int
n, int[][]edges, boolean isDirected)
{
  List<List<Integer>> graph = new
ArrayList<>();
  for (int i = 0; i < n; i++)
graph.add(new ArrayList<>());

  for (int[] e : edges) {
    int u = e[0], v = e[1];
    graph.get(u).add(v);
    if (!isDirected)
graph.get(v).add(u);
  }
  return graph;
}
```

💡 Tip: Use List for neighbors. Avoid duplicates if needed.

2 BREADTH FIRST SEARCH (BFS)

```
List<Integer> bfs(int start,
List<List<Integer>> g) {
  int n = g.size();
  boolean[] vis = new boolean[n];
  Queue<Integer> q = new
LinkedList<>();
  List<Integer> order = new
ArrayList<>();
  q.offer(start); vis[start] = true;
  while (!q.isEmpty()) {
    int node = q.poll();
    order.add(node);
    for (int nei : g.get(node)) {
      if (!vis[nei]) {
        vis[nei] = true;
        q.offer(nei);
      }
    }
  }
  return order;
}
```

💡 Tip: BFS = Level by level. Queue + Visited.

3 DEPTH FIRST SEARCH (DFS)

```
void dfs(int node,
List<List<Integer>> g, boolean[]
vis,
         List<Integer> order) {
  vis[node] = true;
  order.add(node);
  for (int nei : g.get(node)) {
    if (!vis[nei]) dfs(nei, g, vis,
order);
  }
}
// Wrapper
List<Integer> dfsTraversal(int
start, List<List<Integer>> g) {
  int n = g.size();
  boolean[] vis = new boolean[n];
  List<Integer> order = new
ArrayList<>();
  dfs(start, g, vis, order);
  return order;
}
```

💡 Tip: DFS = Go deep, then backtrack. Recursion / Stack.

4 TIME COMPLEXITY GUIDE

Operation / Algorithm	Time Complexity
BFS / DFS (Adjacency List)	O(V + E)
BFS / DFS (Adjacency Matrix)	O(V^2)
Dijkstra (Binary Heap)	O((V + E) log V)
Dijkstra (Min Heap)	O((V + E) log V)
Topological Sort (Kahn's BFS)	O(V + E)
Topological Sort (DFS)	O(V + E)
Union Find (amortized)	O(α(V))
Floyd Warshall (All Pairs)	O(V^3)

V = vertices, E = edges, α = inverse Ackermann (≈ constant)

5 SPACE COMPLEXITY GUIDE

Algorithm / Structure	Space Complexity
Adjacency List	O(V + E)
Adjacency Matrix	O(V^2)
BFS / DFS	O(V) (visited + queue/stack)
Dijkstra	O(V) (dist[]) + O(E) (adj list)
Union Find	O(V)
Floyd Warshall	O(V^2)

Space depends on representation & data structures.

6 GRAPH REPRESENTATION COMPARISON

Type	Space	Add Edge	Check Edge (u,v)	Iterate Neighbors
Adjacency List	O(V + E)	O(1)	O(deg(u))	O(deg(u))
Adjacency Matrix	O(V^2)	O(1)	O(1)	O(V)

💡 Use List for sparse graphs (E << V^2). Use Matrix for dense graphs.

7 WHEN TO USE WHAT?

Problem Type	Use
Shortest Path (Weighted)	Dijkstra (A, B, C, E)
Shortest Path (Unweighted)	BFS (F)
All Pairs Shortest Path	Floyd Warshall (L)
Cycle / Connected / Tree	Union Find / DFS (J, K)
Order / Scheduling	Topological Sort (H)
Grid / Multi-Source	BFS (I)

💡 Understand the goal → pick the right tool.

8 DIJKSTRA TEMPLATE (MIN HEAP)

```
int[] dijkstra(int n, List<List<int[]>> g, int src) {
  int[] dist = new int[n];
  Arrays.fill(dist, Integer.MAX_VALUE);
  dist[src] = 0;
  PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0]-b[0]);
  pq.offer(new int[]{0, src}); // (dist, node)
  while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int d = cur[0], u = cur[1];
    if (d > dist[u]) continue;
    for (int[] e : g.get(u)) {
      int v = e[0], w = e[1];
      if (d + w < dist[v]) {
        dist[v] = d + w;
        pq.offer(new int[]{dist[v], v});
      }
    }
  }
  return dist;
}
```

9 TOPOSORT (KAHN'S BFS)

```
int[] topoSort(int n, List<List<Integer>> g) {
  int[] indeg = new int[n];
  for (int u = 0; u < n; u++)
    for (int v : g.get(u)) indeg[v]++;
  Queue<Integer> q = new LinkedList<>();
  for (int i = 0; i < n; i++) if (indeg[i] == 0) q.offer(i);

  int[] order = new int[n]; int idx = 0;
  while (!q.isEmpty()) {
    int u = q.poll();
    order[idx++] = u;
    for (int v : g.get(u))
      if (--indeg[v] == 0) q.offer(v);
  }

  if (idx != n) return new int[0]; // cycle
  return order;
}
```

10 UNION FIND (DSU)

```
class DSU {
  int[] parent, rank;
  DSU(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
  }
  int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
  }
  boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) parent[px] = py;
    else if (rank[py] < rank[px]) parent[py] = px;
    else { parent[py] = px; rank[px]++; }
    return true;
  }
}
```

11 ALL PAIRS SHORTEST PATH

```
// Floyd Warshall
void floydWarshall(int[][] dist) {
  int n = dist.length;
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (dist[i][k] != INF && dist[k][j] != INF
            && dist[i][k] + dist[k][j] < dist[i][j])
          dist[i][j] = dist[i][k] + dist[k][j];
}
```

💡 INF = a very large value (e.g. 1e9)

12 COMMON PITFALLS

#	Mistake	Why it Fails	Fix
1	Not marking visited	Infinite loop / TLE	**Mark visited when pushing (BFS)**
2	Wrong graph direction	Wrong answer	**Check if graph is directed/undirected**
3	Forget edge weights	Wrong path	**Use weighted template (Dijkstra)**
4	PriorityQueue order wrong	TLE / Wrong path	**Min-heap by distance**
5	Integer overflow	Wrong result	**Use long for distance**
6	Cycle in topo sort	Invalid order	**Check if result size < V**

13 EDGE CASE CHECKLIST

- ☑ Empty graph / single node
- ☑ Disconnected components
- ☑ Self loop
- ☑ Multiple edges between same nodes
- ☑ Large weights / zero weights
- ☑ Graph with no cycles
- ☑ Graph with cycles
- ☑ Maximum constraints (V, E)

14 QUICK RECALL TABLE

Goal	Go-To Template
Shortest time / distance (positive weights)	**Dijkstra (A, B, C, E)**
Fewest steps (unweighted)	**BFS (F)**
Order of tasks / courses	**Topological Sort (H)**
Detect cycle / components	**Union Find / DFS (J, K)**
Grid / multi-source	**BFS (I)**
All pairs shortest path	**Floyd Warshall (L)**

15 GRAPH PATTERN FLOW (RECAP)

Tip: Think → Choose → Code → Test → Optimize → Succeed!

16 INTERVIEW TIP BOX

- **Talk aloud. Explain your approach.**
- **State assumptions & edge cases.**
- **Write clean, modular code.**
- **Start with brute force if unsure.**
- **Optimize step by step.**

Confidence • Clarity • Crack FAANG!

17 NEXT PAGE PREVIEW

Next page (Page 17) contains

GRAPH PATTERN ROADMAP.

Follow the path.

Master the patterns.

Crack the interviews!

CHEAT PIPELINE (INTERVIEW SUCCESS FORMULA)


Problem (Understand)
Input (What given)
Representation (Graph + Weights)
Helper Structure (Queue / PQ / UF / Stack)
Pattern (Pick the Right One)
Apply (Implement)
Dry Run (Example)
Optimize (Time /Space)

GRAPH PATTERN GUIDE

GRAPH PATTERN ROADMAP & SPACED PACTICE PLAN

"Consistency builds pattern recognition. Pattern recognition wins interviews."

HOW TO USE THIS PAGE

1. Follow the roadmap in order.
2. Solve problems by pattern, not randomly.
3. Track your progress and weak areas.
4. Revise regularly using spaced repetition.
5. Revisit cheat sheets before interviews.

You've got this. Keep showing up! 🎯

GRAPH PATTERN ROADMAP (RECOMMENDED FLOW)

1 Foundations	2 Core Traversals	3 Grid / Matrix	4 Topological	5 Advanced	6 Mastery
• What is Graph?	• DFS & BFS	• Islands, Area	• DAG, Order	• Shortest Path	• Hard Problems
• Representations	• Connected	• Multi-Source BFS	• Course Schedule	• Multiple Sources	• Simulation
• Basic Traversals	• Components	• Water Flow	• Cycle Detection	• Optimization	• Edge Cases

***Tip: Master 1 step at a time. Revise previous steps while learning new ones.**

GRAPH PATTERN SUMMARY (At a Glance)

	Pattern	What it Solves	Key Idea	Main Tools	Examples (Problems)	Time Complexity (Typical)
1	**Network Delay Time**	Time from source to all nodes	Shortest path from single source	**Dijkstra (PQ)**	Network Delay Time	O((V + E) log V)
2	**Dijkstra with Path**	Shortest distance + actual path	Track parent while relaxing	**Dijkstra + Parent[]**	Path With Min Cost	O((V + E) log V)
3	**Minimum Effort Path**	Minimize maximum edge	Minimize the max along path	**Dijkstra (custom relax)**	Path With Minimum Effort	O((V + E) log V)
4	**Cheapest Flights (K Stops)**	Shortest path with stop limit	Use stops dimension	**Dijkstra / BFS + stops**	Cheapest Flights Within K Stops	O(K*E log V)
5	**Topological Sort**	Order tasks / DAG processing	Indegree 0 → queue	**BFS (Kahn's)**	Course Schedule II	O(V + E)
6	**Cycle Detection**	Detect cycle in directed graph	Back edge in DFS	**DFS + Rec Stack**	Course Schedule (I)	O(V + E)
7	**Undirected Shortest Path**	Shortest path unweighted	Level by level expansion	**BFS**	Shortest Path in UD Graph	O(V + E)
8	**Multi-Source BFS**	From many sources at once	Put all sources in queue	**BFS (multi-source)**	Rotting Oranges, 01 Matrix	O(V + E)
9	**Disjoint Set (Union Find)**	Check / Merge components	Union by rank + path compression	**DSU**	Valid Tree, Accounts Merge	O(α(V))
10	**Floyd Warshall**	All pairs shortest path	DP on intermediate nodes	**DP (3 loops)**	All Pairs Shortest Path	O(V^3)
11	**Strongly Connected Components**	Group of nodes mutually reachable	Kosaraju / Tarjan	**DFS**	SCC Problems	O(V + E)
12	**Special Patterns**	Cloning, Regions, Water Flow	Use pattern + appropriate traversal	**DFS / BFS**	Clone Graph, Surrounded Regions, Pacific Atlantic	Varies

SPACED PRACTICE PLAN (4 WEEK PLAN)

Week	Focus	New Problems (2-3 / day)	Revision (Spaced)	Goal
Week 1	**Foundations + Core Traversals**	DFS, BFS, Connected Components, Islands, Clone Graph	Day 4: Review Day 1 Day 7: Review Day 1-3	**Strong basics + confidence**
Week 2	**Topological + BFS Applications**	Course Schedule, Topo Sort, Multipliers, Rotting Oranges	Day 11: Review Day 1-7 Day 14: Review Day 8-12	**Solve medium comfortably**
Week 3	**Shortest Paths**	Dijkstra variations, Min Effort, Cheapest Flights K Stops	Day 18: Review Day 1-14 Day 21: Review Day 15-18	**Master shortest path patterns**
Week 4	**Advanced + Mixed**	Floyd Warshall, Union Find, Hard graph problems	Day 25: Review Day 1-21 Day 28: Review Day 22-25	**Tackle hard problems**

💡 **Tip: Active recall + spaced repetition = long term retention.**

REVISION SCHEDULE (SPACED REPETITION)

- **Day 1-3** Learn new concepts & solve problems
- **Day 4** Review Day 1 (without looking)
- **Day 7** Review Day 1-3
- **Day 11** Review Day 1-7
- **Day 14** Review Day 8-12
- **Day 18** Review Day 1-14
- **Day 21** Review Day 15-18
- **Day 25** Review Day 1-21
- **Day 28** Review Day 22-25
- **Day 30** Full mock + weak area revision

ERROR LOG (Track & Improve)

Maintain an error log.
Write down:

- Problem name
- What went wrong?
- Concept gap
- Correct approach
- Date of review

"The goal is not to never make mistakes.
The goal is to never repeat the same mistake."

INTERVIEW STRATEGY ROADMAP

Understand the Problem
- Clarify
- Constraints
- Examples

→

Identify Graph Type
- Directed / Undirected
- Weighted / Unweighted
- Grid / Tree / DAG

→

Pick the Right Pattern
- Match pattern
- Choose algorithm
- Plan steps

→

Implement Cleanly
- Write modular code
- Handle edge cases
- Test small cases

→

Verify & Optimize
- Check results
- Optimize if needed
- Explain clearly

KEY MINDSET REMINDERS

- [x] Always draw the graph.
- [x] Think in patterns, not solutions.
- [x] Start simple (brute force), then optimize.
- [x] Dry run on small examples.
- [x] Handle edge cases & constraints.
- [x] Explain your thought process.
- [x] Code clean, modular, readable.

Think → Plan → Code → Test → Optimize

COMPLEXITY REMINDERS

BFS / DFS:	O(V + E)
Dijkstra (PQ):	O((V + E) log V)
Topo Sort:	O(V + E)
Union Find:	O(α(V))
Floyd Warshall:	O(V^3)
Space (Adj List):	O(V + E)

Tip: Use Adj List for sparse graphs.

COMMON PITFALLS

- ⊗ Not marking visited nodes.
- ⊗ Wrong data structure (Stack vs Queue vs PQ).
- ⊗ Not handling disconnected components.
- ⊗ Integer overflow in distances.
- ⊗ Ignoring constraints (K stops, weights, etc.).
- ⊗ Forgetting to return / edge cases.

Tip: Slow down, think & check!

QUICK CHECK BEFORE SUBMIT

- [x] Handled all edge cases?
- [x] Correct data structure used?
- [x] Visited / parent tracked?
- [x] Return value correct?
- [x] Complexity acceptable?
- [x] Dry run matches output?

Tip: 10 sec checklist saves 10 min regret!

INTERVIEW TIPS

- ⭐ Communicate throughout.
- ⭐ Break problem into small steps.
- ⭐ Use examples to explain.
- ⭐ If stuck, think out loud.
- ⭐ Optimize only after correct solution.
- ⭐ Stay calm, think patterns.

Remember: Interview is a conversation!

NEXT PAGE PREVIEW

Next page (Page 18) contains
GRAPH PATTERN PRACTICE TRACKER & PROBLEM INDEX.
Track, practice, analyze and crack FAANG!

THE SUCCESS FORMULA

Problem (Understand) → **Input** (What given) → **Representation** (Graph Model) → **Pattern** (Identify) → **Algorithm** (Pick Right One) → **Implementation** (Code Clean) → **Test** (Dry Run) → **Analyze** (Complexity) → **Optimize** (If Needed) → **Success** (Interview)

Plan well. Practice smart. Stay consistent. You will get there! 💪

PAGE 17

GRAPH PATTERN GUIDE

PAGE 17 — COMPLETE GRAPH ALGORITHMS CHEAT SHEET (DSA ONLY)

"One page. All core graph algorithms. Fast reference before interviews."

HOW TO USE THIS PAGE

1. Identify the pattern.
2. Pick the right algorithm.
3. Use the template.
4. Handle edge cases.
5. Optimize & code clean.

ALGORITHM INDEX

1 DFS (Traversal)

Recursive Template

```
void dfs(int node,
List<List<Integer>> g) {
    boolean[] vis;
    vis[node] = true;
    // process node
    for(int nei :
g.get(node)) {
        if(!vis[nei])
dfs(nei, g, vis);
    }
}
```

Time: O(V + E) Space: O(V)

Use for: component, path, cycle, topo (DFS)

2 BFS (Traversal)

Iterative Template

```
void bfs(int src,
List<List<Integer>> g) {
    boolean[] vis = new
boolean[g.size()];
    Queue<Integer> q = new
LinkedList<>();
    q.offer(src); vis[src]
= true;
    while(!q.isEmpty()) {
        int node =
q.poll();
        // process node
        for(int nei :
g.get(node)) {
            if(!vis[nei]) {
                vis[nei] =
true;

q.offer(nei);
            }
        }
    }
}
```

Time: O(V + E) Space: O(V)

Use for: shortest path (unweighted), level order

3 Connected Components

DFS Template

```
int count = 0;
void dfs(int node,
List<List<Integer>> g,
         boolean[] vis) {
    vis[node] = true;
    for(int nei :
g.get(node))
        if(!vis[nei])
dfs(nei, g, vis);
}

int countComponents(int n,
int[][] edges) {
    List<List<Integer>> g =
buildGraph(n, edges);
    boolean[] vis = new
boolean[n];
    for(int i=0; i<n; i++)
{
        if(!vis[i]) {
count++;
            dfs(i, g, vis);
        }
    }
    return count;
}
```

Time: O(V + E) Space: O(V)

Use for: number of connected components

4 Number of Islands

BFS/DFS on Grid

```
int[][] dirs = {{1,0},
{-1,0},{0,1},{0,-1}};

int numIslands(char[][]
grid) {
    int m=grid.length,
n=grid[0].length, cnt=0;
    boolean[][] vis = new
boolean[m][n];
    for(int i=0; i<m; i++)
{
        for(int j=0; j<n;
j++) {
            if(grid[i]
[j]=='1' && !vis[i][j]) {
                cnt++;
                bfs(i, j,
grid, vis); // or dfs
            }
        }
    }
    return cnt;
}
```

Time: O(M * N) Space: O(M * N)

Use for: island counting problems

5 Clone Graph
BFS Template

```
Node cloneGraph(Node node)
{
    if(node == null) return
null;
    Map<Node, Node> map =
new HashMap<>();
    Queue<Node> q = new
LinkedList<>();
    q.offer(node);
    map.put(node, new
Node(node.val));
    while(!q.isEmpty()) {
        Node cur =
q.poll();
        for(Node nei :
cur.neighbors) {

if(!map.containsKey(nei)) {

map.put(nei, new
Node(nei.val));

q.offer(nei);
            }

map.get(cur).neighbors.add(
map.get(nei));
        }
    }
    return map.get(node);
}
```

Time: O(V + E) Space: O(V)

💡 Use for: deep copy of connected graph

6 Course Schedule (I)
Cycle Detection (BFS - Kahn's)

```
boolean canFinish(int n,
int[][] pre) {
    List<List<Integer>> g =
buildGraph(n, pre);
    int[] indeg = new
int[n];
    for(int i=0; i<n; i++)
        for(int nei :
g.get(i)) indeg[nei]++;

    Queue<Integer> q = new
LinkedList<>();
    for(int i=0; i<n; i++)
if(indeg[i]==0) q.offer(i);

    int cnt = 0;
    while(!q.isEmpty()) {
        int node =
q.poll(); cnt++;
        for(int nei :
g.get(node)) {
            if(--
indeg[nei]==0)
q.offer(nei);
        }
    }
    return cnt == n;
}
```

Time: O(V + E) Space: O(V)

💡 Use for: detect cycle in DAG (can finish?)

7 Course Schedule (II)
Topological Sort (BFS)

```
int[] findOrder(int n,
int[][] pre) {
    List<List<Integer>> g =
buildGraph(n, pre);
    int[] indeg = new
int[n];
    for(int i=0; i<n; i++)
        for(int nei :
g.get(i)) indeg[nei]++;

    Queue<Integer> q = new
LinkedList<>();
    for(int i=0; i<n; i++)
if(indeg[i]==0) q.offer(i);

    int[] order = new
int[n]; int idx = 0;
    while(!q.isEmpty()) {
        int node =
q.poll(); order[idx++] =
node;
        for(int nei :
g.get(node)) {
            if(--
indeg[nei]==0)
q.offer(nei);
        }
    }
    return idx == n ? order
: new int[0];
}
```

Time: O(V + E) Space: O(V)

💡 Use for: return valid order / topological sort

8 Pacific Atlantic Water Flow
Multi-Source BFS (Reverse)

```
int pacificAtlantic(int[][]
h) {
    int m=h.length,
n=h[0].length;
    boolean[][] pac = new
boolean[m][n];
    boolean[][] atl = new
boolean[m][n];

    Queue<int[]> qp = new
LinkedList<>();
    Queue<int[]> qa = new
LinkedList<>();
    // push borders to
correspond queues
    // bfs to mark
reachable cells
    // intersect both
    List<List<Integer>> res
= new ArrayList<>();
    for(int i=0; i<m; i++)
for(int j=0; j<n; j++)
        if(pac[i][j] &&
atl[i][j])

res.add(Arrays.asList(i,
j));
    return res;
}
```

Time: O(M * N) Space: O(M * N)

💡 Use for: water flow / reachability from borders

9 Rotting Oranges
Multi-Source BFS

```
int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, time = 0;
    // init queue with rotten oranges
    // count fresh
    while(!q.isEmpty() && fresh > 0) {
        int size = q.size(); time++;
        while(size-- > 0) {
            int[] cur = q.poll();
            for(int[] d : dirs) {
                int x = cur[0]+d[0], y = cur[1]+d[1];

if(valid(x,y) && grid[x][y]==1) {
                    grid[x][y]=2; fresh--;

q.offer(new int[]{x, y});
                }
            }
        }
    }
    return fresh == 0 ? time : -1;
}
```

Time: O(M * N) Space: O(M * N)

Use for: multi-source BFS spread problems

10 Topological Sort (BFS)
Kahn's Algorithm

```
List<Integer> topoSort(int n, int[][] edges) {
    List<List<Integer>> g = buildGraph(n, edges);
    int[] indeg = new int[n];
    for(int i=0; i<n; i++)
        for(int nei : g.get(i)) indeg[nei]++;

    Queue<Integer> q = new LinkedList<>();
    for(int i=0; i<n; i++) if(indeg[i]==0) q.offer(i);

    List<Integer> order = new ArrayList<>();
    while(!q.isEmpty()) {
        int node = q.poll(); order.add(node);
        for(int nei : g.get(node)) {
            if(--indeg[nei]==0) q.offer(nei);
        }
    }
    return order.size()==n ? order : new ArrayList<>();
}
```

Time: O(V + E) Space: O(V)

Use for: DAG ordering / dependencies

11 Dijkstra (Min Heap)
Shortest Path (Non-negative)

```
int[] dijkstra(int n, int[][] edges, int src) {
    List<List<int[]>> g = new ArrayList<>();
    for(int i=0; i<n; i++) g.add(new ArrayList<>());
    for(int[] e: edges) {
        g.get(e[0]).add(new int[]{e[1], e[2]});
        // if undirected: g.get(e[1]).add(new int[]{e[0], e[2]});
    }
    int[] dist = new int[n]; Arrays.fill(dist, (int)1e9);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(
        (a,b)->a[0]-b[0] // {dist, node}
    );
    pq.offer(new int[]{0, src});
    while(!pq.isEmpty()) {
        int[] cur = pq.poll(); int d = cur[0], u = cur[1];
        if(d > dist[u]) continue;
        for(int[] e : g.get(u)) {
            int v = e[0], w = e[1];
            if(dist[v] > d + w) {
                dist[v] = d + w;

pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

Time: O((V + E) log V) Space: O(V)

Use for: single source shortest path

12 Union Find (DSU)
Disjoint Set Union

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for(int i=0; i<n; i++) { parent[i]=i; rank[i]=0; }
    }
    int find(int x) {
        if(parent[x]≠x) parent[x]=find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if(px==py) return false;
        if(rank[px] > rank[py]) parent[py] = px;
        else if(rank[py] > rank[px]) parent[px] = py;
        else { parent[py] = px; rank[px]++; }
        return true;
    }
}
```

Time: O(α(V)) per op Space: O(V)

Use for: components, cycle in undirected

GRAPH REPRESENTATIONS

Adjacency List

- Best for sparse graphs
- Space: O(V + E)
- Iterate neighbors fast

0	[1, 2]
1	[0, 3]
2	[0, 3]
3	[1, 2]

Adjacency Matrix

- Best for dense graphs
- Space: O(V^2)
- Check edge exists fast

	0	1	2	3
0	0	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

COMMON DIRECTIONS (Grid Problems)

4 Directions

```
int[][] dirs4 = {{1,0}, {-1,0}, {0,1}, {0,-1}};
```

graph TD A[ ] --> B[ ] C[ ] --> D[ ] E[ ] --> F[ ] G[ ] --> H[ ]

8 Directions

```
int[][] dirs8 = {{-1,-1}, {-1,0}, {-1,1}, {0,-1}, {0,1}, {1,-1}, {1,0}, {1,1}};
```

graph TD A[ ] --> B[ ]

EDGE TYPES

- Directed / Undirected
- Weighted / Unweighted
- Cyclic / Acyclic (DAG)
- Connected / Disconnected

COMPLEXITY QUICK REFERENCE | V = Vertices | E = Edges | M = Rows | N = Cols |

- **DFS / BFS :** O(V + E)
- **Topological Sort :** O(V + E)
- **Dijkstra (Min Heap) :** O((V + E) log V)
- **Number of Islands :** O(M * N)
- **Rotting Oranges :** O(M * N)
- **Pacific Atlantic :** O(M * N)
- **Connected Components :** O(V + E)
- **Clone Graph :** O(V + E)
- **Union Find (per op) :** O(α(V))

α(V) = Inverse Ackermann (≈ constant)
For all practical input sizes, treat as O(1).

PAGE 17

GRAPH PATTERN GUIDE

COMPLETE GRAPH ALGORITHMS CHEAT SHEET (DSA ONLY)

Core Patterns. Process + Code + Use + Pitfalls. Revise before your interview!

PRINTABLE
A4 FORMAT
DSA ONLY

HOW TO USE THIS PAGE

1. Identify the pattern from the problem.
2. Pick the right algorithm.
3. Follow the process (steps).
4. Use the template code.
5. Watch out for common mistakes.
6. Optimize & handle edge cases.

Think → Choose → Code → Test → Optimize

GRAPH PROBLEM SOLVING ROADMAP (HOW TO THINK)

graph LR A[Start Problem] ⟶ B{Is it Graph?} B -- NO ⟶ C[Not a Graph, Use other DS] B -- YES ⟶ D{Weighted Edges?} C -- YES ⟶ E[Shortest Path Min Cost - Dijkstra] D -- YES ⟶ E D -- NO ⟶ F[Unweighted Shortest Path - BFS] E ⟶ G[Need Ordering Dependencies - Topological Sort BFS/DFS] F ⟶ G G ⟶ H[Need Connectivity / Merge Components - Union Find / Connected Components]

★ Tip: Master these patterns and 90% of graph problems are solved!

1 DFS (Depth First Search)

Explore as deep as possible, then backtrack.

PROCESS (STEPS) A

1. Mark current node as visited.
2. Process the node (add to result etc.).
3. Recur for all unvisited neighbors.
4. Backtrack when no unvisited neighbors.

CODE TEMPLATE (Java)

```
void dfs(int node, List<List<Integer>> g, boolean[] vis) {
    vis[node] = true;
    // process node
    for (int nei : g.get(node)) {
        if (!vis[nei]) dfs(nei, g, vis);
    }
}
```

💡 **AHA:** Go deep first, then backtrack. ☑ **Use When:** Explore all nodes, components, cycle detection, topo sort. ⚠ **Watch Out:** Mark visited before recursion to avoid cycles.

Time: O(V + E) | Space: O(V)

2 BFS (Breadth First Search)

Visit level by level using a queue.

PROCESS (STEPS) A

1. Push start node in queue.
2. Mark visited.
3. While queue not empty:
   - Pop front
   - Process
   - Push unvisited neighbors

CODE TEMPLATE (Java)

```
void bfs(int src, List<List<Integer>> g) {
    boolean[] vis = new boolean[g.size()];

    Queue<Integer> q = new LinkedList<>();

    q.offer(src);
    vis[src] = true;
    while (!q.isEmpty()) {
        int node = q.poll();
        for (int nei : g.get(node)) {
            if (!vis[nei]) {

                vis[nei] = true;

                q.offer(nei);
            }
        }
    }
}
```

💡 **AHA:** First visit = shortest path (unweighted). ☑ **Use When:** Minimum steps, nearest node, level order. ⚠ **Watch Out:** Mark visited when enqueuing, not dequeuing.

Time: O(V + E) | Space: O(V)

3 Topological Sort (BFS - Kahn's)

Linear ordering of DAG based on dependencies.

PROCESS (STEPS) A

1. Compute indegree of all nodes.
2. Push all nodes with indegree 0.
3. Pop node, add to result.
4. Reduce indegree of neighbors.
5. If indegree becomes 0, push to queue.
6. If result size < V → Cycle exists.

CODE TEMPLATE (Java)

```
List<Integer> topoSort(int n, List<List<Integer>> g) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : g.get(u)) indeg[v]++;

    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.offer(i);

    List<Integer> res = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        res.add(u);
        for (int v : g.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return res.size() == n ? res : new ArrayList<>();
}
```

💡 **AHA:** Process node only after all prerequisites are satisfied. ☑ **Use When:** DAG, dependencies, course schedule, build order. ⚠ **Watch Out:** If processed nodes < n → cycle exists.

Time: O(V + E) | Space: O(V)

4 Dijkstra (Min Heap)

Shortest path from single source (positive weights).

PROCESS (STEPS) A

1. Init dist[] = INF, dist[src] = 0.
2. Push (0, src) to min-heap.
3. Pop min (d, node). If d > dist[node], skip.
4. Relax all edges of node.
5. Update dist & push to heap.

CODE TEMPLATE (Java)

```
int[] dijkstra(int n, List<List<int[]>> g, int src) {
    int INF = (int)1e9;
    int[] dist = new int[n];

Arrays.fill(dist, INF);
dist[src] = 0;

PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);

pq.offer(new int[]{0, src});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] e : g.get(u)) {
            int v = e[0], w = e[1];
            if (dist[u] + w < dist[v]) {

dist[v] = dist[u] + w;

pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

💡 **AHA:** Relax edges. First time you pop a node, its shortest distance is finalized.

☑ **Use When:** Positive weighted shortest path, network delay, min cost.

⚠ **Watch Out:** Ignore outdated entries: if d > dist[node] skip.

Time: O((V + E) log V) | Space: O(V)

5 Union Find (Disjoint Set Union)

Maintain groups and merge components.

PROCESS (STEPS) A

1. Make each node its own parent.
2. Find: return root with path compression.
3. Union: merge by rank/size.

CODE TEMPLATE (Java)

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n];
rank = new int[n];
        for (int i = 0; i < n; i++)
parent[i] = i;
    }
    int find(int x) {
        if (parent[x] ≠ x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py])
parent[px] = py;
        else if (rank[px] > rank[py])
parent[py] = px;
        else {
parent[py] = px; rank[px]++;
}
        return true;
    }
}
```

💡 **AHA:** Build groups instead of traversing them.

☑ **Use When:** Connectivity, merge components, cycle in undirected graph.

⚠ **Watch Out:** Always use path compression + union by rank.

Time: O(α(V)) per op | Space: O(V)

6 Grid DFS / BFS

For grid based problems (islands, regions, flow).

PROCESS (STEPS) A

1. Check bounds & valid cell.
2. Mark visited.
3. Traverse 4 or 8 directions.
4. Count / collect / compare.

CODE TEMPLATE (Java)

```
// 4 Directions
int[][] dir4 = {{1,0}, {-1,0}, {0,1}, {0,-1}};

// 8 Directions
int[][] dir8 = {{1,0}, {-1,0}, {0,1}, {0,-1},

{1,1}, {1,-1}, {-1,1},
{-1,-1}};

boolean isValid(int r, int c, int n, int m, boolean[][] vis) {
    return r ⩾ 0 && r<n && c ⩾ 0 && c<m && !vis[r][c];
}
```

💡 **AHA:** Grid problems = graph problems with constraints.

☑ **Use When:** Islands, surrounded regions, flood fill, matrix distance.

⚠ **Watch Out:** Boundary check & mark visited immediately.

Time: O(M * N) | Space: O(M * N)

GRAPH REPRESENTATIONS

Adjacency List (Preferred)

- Best for sparse graphs
- Space: O(V + E)
- Iterate neighbors fast

graph LR 0((0)) --> 1[1, 2] 1((1)) --> 2[0, 3] 2((2)) --> 3[0, 3] 3((3)) --> 4[1, 2]

Adjacency Matrix

- Best for dense graphs
- Space: O(V^2)
- Check edge exists fast

	0	1	2	3
0	0	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

DIRECTIONS (Grid Problems)

4 Directions (Up, Down, Left, Right)

graph TD A --> B C --> A A --> D E --> A

```
int[][] dir4 = {{1,0},{-1,0},
{0,1},{0,-1}};
```

8 Directions (All)

graph TD A --> B A --> C

```
int[][] dir8 = {{1,0},{-1,0},
{0,1},{0,-1},{1,1},{1,-1},
{-1,1},{-1,-1}};
```

EDGE TYPES

- **Directed / Undirected**
- **Weighted / Unweighted**
- **Cyclic / Acyclic (DAG)**
- **Connected / Disconnected**
- **Sparse / Dense**

COMPLEXITY QUICK REFERENCE

DFS / BFS	Connected Components	Topological Sort (BFS)	Dijkstra (Min Heap)	Union Find (DSU)	Grid DFS / BFS	WHERE α(V) =
Time: O(V + E) Space: O(V)	Time: O(V + E) Space: O(V)	Time: O(V + E) Space: O(V)	Time: O((V + E) log V) Space: O(V)	Time: O(α(V)) per op Space: O(V)	Time: O(M * N) Space: O(M * N)	Inverse Ackermann (almost constant)

FINAL INTERVIEW CHECKLIST

- ☑ Understand the problem clearly.
- ☑ Pick the correct graph pattern.
- ☑ Choose representation (List / Matrix).
- ☑ Dry run on small example.
- ☑ Handle edge cases.
- ☑ Write clean code using template.
- ☑ Check complexity.
- ☑ Optimize if needed.
- ☑ Test & confirm output.

🏆 **GOLDEN RULE**

Master patterns, write clean code, crack FAANG!

PRACTICE PATTERNS DAILY • BUILD RECOGNITION • WRITE CLEAN CODE • CRACK FAANG INTERVIEWS 🚀

PAGE 18

📖 GRAPH PATTERN GUIDE

PAGE 18 — GRAPH INTERVIEW BIBLE (ULTIMATE ONE-PAGE REVISION)

Recognize the pattern. Pick the right tool. Solve with confidence. Crack FAANG!

⭐ **THE LAST PAGE**
5 MINUTES BEFORE INTERVIEW? OPEN ME!

1 PATTERN RECOGNITION FLOW

❓ **Understand the Problem**
What is being asked?
What are the entities?

⬇

🎲 **Identify Graph?**
Are there connections between entities?

⬇

📄 **Choose Representation**
Adjacency List (default)
Adjacency Matrix (dense)

⬇

🧠 **Pick Helper Structure**
Visited, Parent, Queue, Stack, Dist, Indegree, DSU

⬇

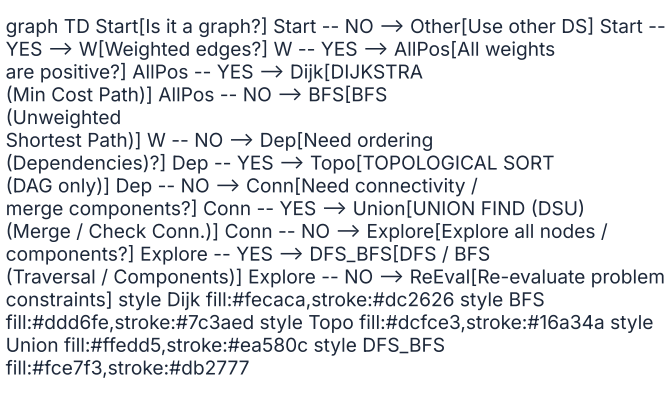
</> **Choose Algorithm**
DFS / BFS / Topo / DSU / Dijkstra

⬇

🎯 **Solve & Optimize**
Handle edge cases, analyze, and optimize

Think → Choose → Code → Test → Optimize

2 PROBLEM → SOLUTION DECISION TREE



3 HIDDEN GRAPH RECOGNITION (THINK LIKE THIS!)

▦	**Grid / Matrix** → 4 or 8 directions, islands, regions, paths
🎓	**Courses / Prerequisites** → Topological Sort (DAG)
👥	**Social / Friends / Network** → DFS / BFS / Union Find
✈	**Flights / Routes / Maps** → BFS (min stops) / Dijkstra (min cost)
🧩	**Groups / Components** → DFS / BFS / Union Find
💧	**Flow / Water / Spread** → DFS / BFS (multi-source BFS)
🔢	**Dependencies / Order** → Topological Sort
🏔	**Shortest Path (Weighted)** → Dijkstra
🔗	**Dynamic Connectivity** → Union Find

4 CORE GRAPH ALGORITHMS - WHEN, WHY & HOW

Algorithm	When to Use	Key Idea (AHA)	Steps (Quick Process)	Core Template (Java)	Time	Space
DFS (Depth First Search)	Explore all nodes, components, cycles, topo sort, paths	Go deep first, then backtrack.	1. Mark node visited 2. Recur for all unvisited neighbors 3. Backtrack	`void dfs(int node, List<List<Integer>> g, boolean[] vis) { vis[node] = true; for (int nei : g.get(node)) { if (!vis[nei]) dfs(nei, g, vis); } }`	O(V + E)	O(V) (Recursion Stack)
BFS (Breadth First Search)	Shortest path (unweighted), level order, nearest	First visit = shortest path.	1. Push start in queue, mark visited 2. While queue not empty: • Pop front • Push unvisited neighbors	`Queue<Integer> q = new LinkedList<>(); q.offer(start); vis[start] = true; while (!q.isEmpty()) { int node = q.poll(); for (int nei : g.get(node)) { if (!vis[nei]) { vis[nei] = true; q.offer(nei); } } }`	O(V + E)	O(V) (Queue)
Topological Sort (Kahn's BFS)	DAG ordering, course schedule, dependencies	Process node after all prerequisites are satisfied.	1. Compute indegree of all nodes 2. Push all nodes with indegree 0 3. Pop, add to order, reduce indegree 4. If processed < V ➔ cycle exists	`Queue<Integer> q = new LinkedList<>(); int[] indeg = new int[n]; // compute indegree for(int i=0; i<n; i++) if(indeg[i]==0) q.offer(i); List<Integer> order = new ArrayList<>(); while(!q.isEmpty()) { int node = q.poll(); order.add(node); for(int nei: g.get(node)) if(--indeg[nei]==0) q.offer(nei); }`	O(V + E)	O(V)
Dijkstra (Min Heap)	Shortest path in weighted graph (positive weights)	Relax edges. First time pop from heap = shortest distance.	1. Init dist[] = INF, dist[src] = 0 2. Push (0, src) in min-heap 3. Pop min, skip if outdated 4. Relax neighbors 5. Repeat	`PriorityQueue<int[]> pq = new ...((a,b)→a[0]-b[0]); int[] dist = new int[n]; Arrays.fill(dist, INF); dist[src]=0; pq.offer(new int[]{0, src}); while(!pq.isEmpty()) { int[] cur = pq.poll(); int d = cur[0], node = cur[1]; if(d > dist[node]) continue; for(int[] nei:`	O((V+E) log V)	O(V+E)

Algorithm	When to Use	Key Idea (AHA)	Steps (Quick Process)	Core Template (Java)	Time	Space

				g.get(node)) {
				int w = nei[1], next = nei[0];
				if(dist[node] + w < dist[next]) {
				dist[next] = dist[node] + w;
				pq.offer(new int[] {dist[next], next});
				}
				}
				}		

Union Find (DSU)	Connectivity, merge components, dynamic graph	Build groups instead of traversing.	1. Make each node its own parent
			2. Find with path compression
			3. Union by rank/size
			4. Find to check connectivity
				int find(int x){
				if(parent[x]≠x)
				parent[x]=find(parent[x]); return parent[x]; }
				void union(int x, int y){
				x=find(x); y=find(y);
				if(x==y) return;
				if(rank[x]<rank[y])
				parent[x]=y; else if(rank[x]>rank[y])
				parent[y]=x;
				else { parent[y]=x;
				rank[x]++; }
				}
					O(α(V)) per op	O(V)

Grid DFS / BFS	Islands, regions, surrounded, basin, matrix path, flood	Traverse 4 or 8 directions with bounds check.	1. Check bounds & valid cell 2. Mark visited 3. Traverse directions 4. Count / collect / compare	int[][] dir4 = {{1,0},{-1,0},{0,1},{0,-1}}; int[][] dir8 = {{1,0},{-1,0},{0,1},{0,-1}, {1,1},{1,-1},{-1,1},{-1,-1}}; boolean isValid(int r, int c){ return r⩾0 && r<m && c⩾0 && c<n && !vis[r][c]; }	O(V)	O(V)

5 GRAPH REPRESENTATIONS

Adjacency List (Preferred)

- Best for sparse graphs
- Space: O(V + E)
- Iterate neighbors fast

graph LR 0((0)) --- 1((1, 2)) 1_((1)) --- 2((0, 3)) 2_((2)) --- 3((0, 3)) 3_((3)) --- 4((1, 2)) style 0 fill:#fff,stroke:#000 style 1 fill:#fff,stroke:#000 style 1_ fill:#fff,stroke:#000 style 2 fill:#fff,stroke:#000 style 2_ fill:#fff,stroke:#000 style 3 fill:#fff,stroke:#000 style 3_ fill:#fff,stroke:#000 style 4 fill:#fff,stroke:#000

Adjacency Matrix

- Best for dense graphs
- Space: O(V^2)
- Check edge exists fast

	0	1	2	3
0	0	1	1	0
1	1	0	0	1
2	1	0	0	1
3	0	1	1	0

6 DIRECTIONS (GRID PROBLEMS)

4 Directions
(Up, Down, Left, Right)

```
int[][] dir4 =
{{1,0},{-1,0},{0,1},
{0,-1}};
```

8 Directions
(All)

```
int[][] dir8 =
{{1,0},{-1,0},{0,1},
{0,-1},
{1,1},{1,-1},{-1,1},
{-1,-1}};
```

7 EDGE TYPES

- **Directed / Undirected** ➔ One way / Two way
- **Weighted / Unweighted** ➔ Has cost / No cost
- **Cyclic / Acyclic (DAG)** ➔ Has cycle / No cycle
- **Connected / Disconnected** ➔ Single / Multiple comps
- **Sparse / Dense** ➔ $E \ll V^2$ / $E \approx V^2$

8 COMPLEXITY SUMMARY

Algorithm	Time Complexity	Sp C
DFS	O(V + E)	O
BFS	O(V + E)	O
Topological Sort	O(V + E)	O
Dijkstra (Min Heap)	O((V + E) log V)	O
Union Find (DSU)	O(α(V)) per op	O
Grid DFS / BFS	O(V)	O

9 COMMON PITFALLS (DON'T LOSE MARKS!)

- ❌ Not marking visited ➔ infinite loops
- ❌ Wrong data structure (Stack vs Queue vs PQ)
- ❌ Index out of bounds in grid problems
- ❌ Forgetting to handle disconnected components
- ❌ Ignoring edge direction
- ❌ Topological sort on cyclic graph
- ❌ Not checking if all nodes processed (cycle)
- ❌ Dijkstra with negative weights
- ❌ Union Find without path compression / rank
- ❌ Modifying list while iterating

10 AHA MOMENTS (MEMORIZE THESE!)

- 💡 **DFS** ➔ Go deep, then backtrack.
- 💡 **BFS** ➔ First visit = shortest path (unweighted).
- 💡 **Topo Sort** ➔ Process only after all prerequisites done.
- 💡 **Dijkstra** ➔ Relax edges, pick next closest.
- 💡 **DSU** ➔ Build groups, don't traverse.
- 💡 **Grid** ➔ Direction + Bounds + Visit = Safe traversal.
- 💡 **Disconnected** ➔ Run algorithm from every unvisited node.
- 💡 **Cycle in Topo** ➔ If result size < V.
- 💡 **Why PQ in Dijkstra?** ➔ Always take min distance.

11 INTERVIEW CHECKLIST

- ☑ Restate the problem
- ☑ Identify graph & entities
- ☑ Choose representation
- ☑ Pick helper structure
- ☑ Choose algorithm
- ☑ Handle edge cases
- ☑ Write clean code
- ☑ Analyze complexity
- ☑ Dry run / Test on cases
- ☑ Confirm output

12 FAANG GRAPH SOLVING PIPELINE

Read & Understand — Clarify input, output & constraints ➔ **Model the Graph** — Nodes, edges, direction, weight ➔ **Choose Representation** — List (default) or Matrix ➔ **Pick Helpers** — Visited, Parent, Queue/Stack, Dist, Indegree, DSU ➔ **Choose Algorithm** — DFS / BFS / Topo / Dijkstra / DSU ➔ **Implement** — Write clean, modular code ➔ **Test & Validate** — Edge cases, dry run, small examples ➔ **Optimize** — Time / Space if needed ➔ **Submit with confidence!**

GOLDEN RULE
Master patterns, write clean code, crack FAANG!

REMEMBER

★ Patterns > Solutions
★ Think before you code
★ Handle edge cases
★ Simple code wins
★ Practice daily
★ Become pattern expert

YOU'VE GOT THIS!
Stay consistent.
Keep grinding. Win big!

GRAPH ALGORITHM CHEAT SHEET (TEMPLATES)

Core Algorithms • Step Process • Java Templates • When & Why to Use
(A4 PRINT DSA ONLY)

PAGE 17A

1 DFS (Depth First Search) Go deep first, then backtrack.

PROCESS (STEPS)

1. **Mark current node visited.**
2. **Process the node** (add to result, etc.).
3. **Recur for all** unvisited neighbors.
4. **Backtrack when no** unvisited neighbors remain.

JAVA TEMPLATE (Adjacency List)

```
void dfs(int node,
List<List<Integer>> g, boolean[]
vis) {
    vis[node] = true;
// mark
    // process node here
    for (int nei : g.get(node)) {
        if (!vis[nei]) {
            dfs(nei, g, vis);
        }
    }
}
```

💡 AHA	🎯 USE WHEN	⚠️ WATCH OUT	⏱ COMPLEXITY
Go deep first, explore until no option left, then backtrack.	• Explore all nodes • Connected comps • Cycle detection • Topo Sort (DFS) • Path existence	• Mark visited before recursion • Stack overflow on deep graph • Forgetting backtrack	**Time:** O(V + E) **Space:** O(V) (Recursion Stack)

2 BFS (Breadth First Search) Visit level by level using a queue.

PROCESS (STEPS)

1. **Push start node** in queue.
2. **Mark visited.**
3. **While queue not empty:**
   - Pop front
   - Process
   - Push all unvisited neighbors

JAVA TEMPLATE (Adjacency List)

```
void bfs(int src,
List<List<Integer>> g) {
    boolean[] vis = new
boolean[g.size()];
    Queue<Integer> q = new
LinkedList<>();
    q.offer(src); vis[src] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        // process node here
        for (int nei : g.get(node))
{
            if (!vis[nei]) {
                vis[nei] = true;
                q.offer(nei);
            }
        }
    }
}
```

💡 AHA	🎯 USE WHEN	⚠️ WATCH OUT	⏱ COMPLEXITY
First visit to a node gives the shortest path (unweighted).	• Shortest path (unweighted) • Level order traversal • Nearest node/level • Minimum steps	• Mark visited when enqueuing (not when dequeuing) • Forgetting to push all neighbors	**Time:** O(V + E) **Space:** O(V) (Queue)

3 TOPOLOGICAL SORT (Kahn's BFS)

Process nodes after all prerequisites are satisfied.

PROCESS (STEPS)

1. **Compute indegree** of all nodes.
2. **Push all nodes with indegree = 0.**
3. **While queue not empty:**
   - Pop node
   - Add to result
   - Reduce indegree of neighbors
   - If indegree = 0, push to queue
4. **If result size < V** → cycle exists.

JAVA TEMPLATE (Kahn's Algorithm)

```
List<Integer> topoSort(int n,
List<List<Integer>> g) {
    int[] indeg = new int[n];
    for (List<Integer> nbrs : g)
        for (int v : nbrs)
indeg[v]++;

    Queue<Integer> q = new
LinkedList<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0)
q.offer(i);

    List<Integer> order = new
ArrayList<>();
    while (!q.isEmpty()) {
        int node = q.poll();
        order.add(node);
        for (int nei : g.get(node))
{
            if (--indeg[nei] == 0)
q.offer(nei);
        }
    }
    return (order.size() == n) ?
order : new ArrayList<>();
}
```

AHA	USE WHEN	WATCH OUT	COMPLEXITY
Node enters result only after all its prerequisites are done.	• DAG ordering • Course schedule • Build order • Task dependencies	• If result size < V, there is a cycle • Don't forget to reduce indegree	**Time:** O(V + E) **Space:** O(V) (Queue + Array)

4 DIJKSTRA (Min Heap)

Relax edges. First time you pop a node, you have its shortest distance.

PROCESS (STEPS)

1. **Init dist[] = INF**, dist[src] = 0.
2. **Push (0, src) in min-heap.**
3. **Pop min (d, node).** If d > dist(node) → skip.
4. **For each neighbor, relax edge:** if newDist < dist[v] update & push.
5. **Repeat until heap is empty.**

JAVA TEMPLATE (PriorityQueue)

```
int[] dijkstra(int n,
List<List<int[]>> g, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist,
Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new
PriorityQueue<>(
        (a, b) ->
Integer.compare(a[0], b[0]));
    pq.offer(new int[]{0, src}); //
(dist, node)

    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], node =
cur[1];
        if (d > dist[node])
continue; // outdated
        for (int[] edge :
g.get(node)) {
            int nei = edge[0], wt =
edge[1];
            int nd = d + wt;
            if (nd < dist[nei]) {
                dist[nei] = nd;
                pq.offer(new int[]
{nd, nei});
            }
        }
    }
    return dist;
}
```

AHA	USE WHEN	WATCH OUT	COMPLEXITY
Greedily pick the closest unprocessed node using min-heap.	• Shortest path in weighted graph (positive weights) • Network routing	• Works only for non-negative edges • Skip outdated entry (d > dist(node))	**Time:** O((V + E) log V) **Space:** O(V) (Heap + Dist Array)

5 UNION FIND (DSU) Build groups instead of traversing them.

PROCESS (STEPS)

1. **Make each node** its own parent.
2. **Find:** return root with path compression.
3. **Union:** merge by rank/size.

JAVA TEMPLATE (Path Compression + Union by Rank)

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++)
parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
parent[x] = find(parent[x]);
        return parent[x];
    }
    void union(int x, int y) {
        int px = find(x), py =
find(y);
        if (px == py) return;
        if (rank[px] < rank[py])
parent[px] = py;
        else if (rank[py] <
rank[px]) parent[py] = px;
        else { parent[py] = px;
rank[px]++; }
    }
    boolean connected(int x, int y)
{ return find(x) == find(y); }
}
```

AHA	USE WHEN	WATCH OUT	COMPLEXITY
Merge components without traversing them.	• Connectivity check • Merge components • Dynamic graph • Kruskal's MST	• Always use path compression • Always union by rank/size	**Time:** O(α(V)) per op **Space:** O(V) (α = Inverse Ackermann)

6 GRID DFS / BFS For grid based problems (islands, regions, path, etc.)

PROCESS (STEPS)

1. **Check bounds & valid cell.**
2. **Mark visited.**
3. **Traverse 4 or 8 directions.**
4. **Count / collect / compare.**

JAVA TEMPLATE

```
// 4 Directions
int[][] dir4 = {{1,0},{-1,0},{0,1},
{0,-1}};
// 8 Directions
int[][] dir8 = {{1,0},{-1,0},{0,1},
{0,-1},
                {1,1},{1,-1},{-1,1},
{-1,-1}};

boolean isValid(int r, int c, int n,
int m) {
    return r >= 0 && r<n && c >= 0 &&
c<m;
}
// BFS on grid example (4
directions)
void bfsGrid(int sr, int sc, char[]
[] grid, boolean[][] vis) {
    int n = grid.length, m =
grid[0].length;
    Queue<int[]> q = new
LinkedList<>();
    q.offer(new int[]{sr, sc});
vis[sr][sc] = true;
    int[][] dir = {{1,0},{-1,0},
{0,1},{0,-1}};
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        for (int[] d : dir) {
            int nr = cur[0] + d[0],
nc = cur[1] + d[1];
            if (isValid(nr,nc,n,m)
&& !vis[nr][nc] && grid[nr][nc] ==
'1') {
                vis[nr][nc] = true;
                q.offer(new int[]
{nr, nc});
            }
        }
    }
}
```

AHA	USE WHEN	WATCH OUT	COMPLEXITY
Grid problems = graph problems with constraints.	• Islands / Regions • Surrounded area • Shortest path in grid • Flood fill	• Bounds check always • Mark visited immediately	**Time:** O(V + E) **Space:** O(V) (V = n*m)

V = Vertices (Nodes) **E** = Edges **α(V)** = Inverse Ackermann (≈ constant)

QUICK RECALL - ALGORITHM SELECTION GUIDE

Explore Everything	Shortest Path (Unweighted)	Shortest Path (Weighted +)	Ordering / Dependencies	Connectivity / Merge	Grid Problems
→ **DFS**	→ **BFS**	→ **Dijkstra**	→ **Topological Sort**	→ **Union Find**	→ **Grid DFS / BFS**

GOLDEN RULE
Know the pattern.
Pick the right tool.
Write clean code.
Crack FAANG!

PAGE 17B

📖 GRAPH QUICK REFERENCE

All the must-know facts, structures & gotchas in one page.

A4 PRINT
DSA ONLY

1 GRAPH REPRESENTATIONS

	ADJACENCY LIST (Preferred)	ADJACENCY MATRIX
Best for	Sparse graphs	Dense graphs
Space	O(V + E)	$O(V^2)$
Add Edge	O(1)	O(1)
Check Edge (u,v)	O(deg(u))	O(1)
Iterate Neighbors	O(deg(u))	O(V)
Memory Usage	Low	High
Example (Undirected)	graph LR 0 --- 1 0 --- 2 1 -- - 3 2 --- 3 **Adj List** 0: [1,2] 1: [0,3] 2: [0,3] 3: [1,2]	0 1 2 3 0 0 1 1 0 1 1 0 0 1 2 1 0 0 1 3 0 1 1 0
Use When	**Almost all interview problems**	**When graph is dense or need O(1) edge check**

2 DIRECTIONS (GRID MOVEMENTS)

4 DIRECTIONS
(Up, Down, Left, Right)

```
int[][] dir4 =
{{-1,0}, {1,0}, {0,-1},
{0,1}};
```

8 DIRECTIONS
(All Directions)

```
int[][] dir8 =
{{-1,0}, {1,0}, {0,-1},
{0,1},
{-1,-1}, {-1,1}, {1,-1},
{1,1}};
```

💡 **Use In:** Grid problems, Islands, Flood Fill, Pacific Atlantic, Rotting Oranges, etc.

3 EDGE TYPES

Directed / Undirected
One way / Two way

Weighted / Unweighted
Has cost / No cost

Cyclic / Acyclic (DAG)
Has cycle / No cycle

Connected / Disconnected
One comp / Multiple comps

Sparse / Dense
$E << V^2$ / $E \approx V^2$

4 COMPLEXITY SUMMARY

Algorithm	Time Complexity	Space Complexity
DFS	**O(V + E)**	**O(V)**
BFS	**O(V + E)**	**O(V)**
Topological Sort (BFS)	**O(V + E)**	**O(V)**
Topological Sort (DFS)	**O(V + E)**	**O(V)**
Dijkstra (Min Heap)	**O((V + E) log V)**	**O(V)**
Union Find (DSU)	**O(α(V)) per op**	**O(V)**
Grid DFS / BFS	**O(M * N)**	**O(M * N)**

V = Vertices, E = Edges, α(V) = Inverse Ackermann (≈ constant)

5 HELPER DATA STRUCTURES

Structure	Use For
Visited Array / Set	Avoid revisiting nodes
Queue (FIFO)	BFS, Level order, Topo Sort (Kahn's)
Stack / Recursion	DFS, Topo Sort (DFS)
Priority Queue (Min Heap)	Dijkstra, Prim's MST
Indegree Array	Topological Sort (Kahn's BFS)
Parent Array	DSU, Path reconstruction
Distance Array	Shortest path, Dijkstra
DSU (Parent, Rank/Size)	Connectivity & Components
Grid / Directions Array	Grid based problems

6 COMMON RETURN TYPES

Type	Used In
`boolean`	Cycle exists, can finish, connected?
`int`	Count (components, islands, paths)
`int[]`	Distances, parents, order
`List<Integer>`	Topological order, path
`List<List<Integer>>`	Graph, adjacency list, result paths
`Node`	Clone graph, any node return
`void`	Modify in-place

💡 **Tip: Always read the return type first. It reveals the intention of the problem.**

7 COMMON BUGS (DON'T LOSE MARKS!)

- ❌ Not marking visited at the right time.
- ❌ Wrong data structure (Stack vs Queue vs PQ).
- ❌ Index out of bounds in grid problems.
- ❌ Forgetting to handle disconnected components.
- ❌ Ignoring edge direction in directed graph.
- ❌ Topological sort: not checking if all nodes processed.
- ❌ Dijkstra: not skipping outdated entries from PQ.
- ❌ Union Find: not using path compression / rank.
- ❌ Off by one errors in loops and indices.
- ❌ Modifying list while iterating.

🧠 **Think before you code. Avoid careless mistakes!**

8 CHOOSE REPRESENTATION

How to decide?

1. Is graph dense? → YES → **Use Adjacency Matrix**
   ↓ NO
2. Need fast edge existence check? → YES → **Use Adjacency Matrix**
   ↓ NO
3. Iterate neighbors frequently? → YES → **Use Adjacency List**
   ↓ NO

Use Adjacency List (Default Choice)

💡 **In interviews, Adjacency List is correct 90% of the time.**

9 GRID PROBLEM CHECKLIST

- ☑ Check bounds at every move.
- ☑ Mark visited before exploring neighbors.
- ☑ Decide: 4 or 8 directions?
- ☑ Use BFS when minimum steps/levels needed.
- ☑ Use DFS when exploring all paths/regions.
- ☑ Watch for obstacles / blocked cells.
- ☑ Start from all sources if needed (multi-source BFS).

💡 **Grid problems are graphs in disguise!**

10 PATTERN → ALGORITHM MAP

Problem Pattern	Key Clue	Algorithm	Data Structure	Example Problems
Explore everything	Visit all nodes / components	DFS	Stack / Recursion	Number of Islands, Clone Graph, Graph Valid Tree
Minimum steps / level	Shortest path (unweighted)	BFS	Queue	Shortest Path, Rotting Oranges, Word Ladder
Ordering / Dependencies	Prerequisites / DAG	Topological Sort	Queue / Stack	Course Schedule, Alien Dictionary
Shortest path (weighted)	Has weights (positive)	Dijkstra	Min Heap (PQ)	Network Delay Time, Path With Minimum Effort
Merge / Connectivity	Need components / connections	Union Find (DSU)	Parent + Rank	Number of Provinces, Redundant Connection

11 INTERVIEW CHECKLIST

- ☑ Understand the problem & constraints.
- ☑ Identify it as graph & its pattern.
- ☑ Choose representation (List / Matrix).
- ☑ Choose helper data structure.
- ☑ Pick the right algorithm.
- ☑ Handle all edge cases.
- ☑ Mark visited correctly.
- ☑ Check disconnected components.
- ☑ Validate output format.
- ☑ Optimize time & space.
- ☑ Dry run on small example.
- ☑ Test all cases.
- ☑ Write clean & modular code.
- ☑ Double check everything before submit.

12 GOLDEN RULES

- ⭐ Solve the pattern, not the problem.
- ⭐ Pick the right tool for the job.
- ⭐ Think ➔ Choose ➔ Code ➔ Test ➔ Optimize.
- ⭐ Simplicity beats complexity.
- ⭐ Read the problem twice.
- ⭐ Dry run before you code.
- ⭐ Clean code = Easy to debug.

13 KEY TAKEAWAY

Master the patterns.
Everything else is logic.
Think smart.
Code clean.
Crack FAANG! 🚀

💡 **This page is your quick revision companion. Keep it close. Revise it daily. Become a Graph Master! 💪**

PAGE 18A

GRAPH INTERVIEW DECISION BIBLE

THE ULTIMATE DECISION GUIDE - THINK SMART, CHOOSE RIGHT, CODE FAST

A4 PRINT
DSA ONLY

1 ALGORITHM DECISION TREE (How to choose the right algorithm)

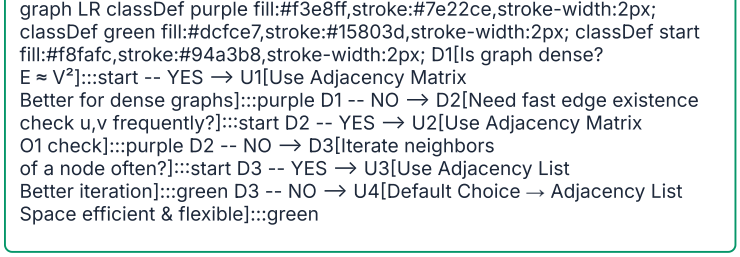
💡 **Golden Rule: Understand the problem 80%, thinking 15%, coding only 5%.**

2 HIDDEN GRAPH PATTERN RECOGNITION

PATTERN TYPE	THINK OF	EXAMPLES
⊞ Grid / Matrix	2D cells connected 4 or 8 directions	**Number of Islands Pacific Atlantic, Surrounded Regions Rotting Oranges, Word Search**
🎓 Courses / Prerequisites	Dependencies / order	**Course Schedule I / II Alien Dictionary**
👥 Friends / Social Network	People are nodes	**Number of Provinces Accounts Merge, Clone Graph**
✈️ Flights / Routes / Maps	Cities are nodes	**Network Delay Time Cheapest Flights Within K Stops Path With Minimum Effort**
🔗 Groups / Components	Find connected parts	**Connected Components Redundant Connection**
💧 Flow / Water / Spread	Spread in steps	**Pacific Atlantic Rotting Oranges**
⛓️ Dependencies / Order	Tasks / jobs depend	**Course Schedule Build Order Problems**
🛤️ Shortest Path (Weighted)	Min cost / distance	**Dijkstra Problems Minimum Cost Path**
🔄 Dynamic Connectivity	Add / remove edges	**Number of Islands II Dynamic Graph Problems**

★ **Tip: If you can draw nodes & edges, it's probably a graph!**

3 REPRESENTATION SELECTION GUIDE



4 HELPER STRUCTURE SELECTION

IF YOU NEED TO...	USE	WHY?
Traverse all nodes deeply	**Stack / Recursion**	DFS, Backtracking
Traverse level by level	**Queue (FIFO)**	BFS, Level order
Get minimum quickly	**Priority Queue (Min Heap)**	Dijkstra, Prim's MST
Group / Merge components	**Union Find (DSU)**	Efficient connectivity
Store visited nodes	**Visited Array / Set**	Avoid cycles / repeats
Store parent / path	**Parent Array / Map**	Reconstruction
Store distances	**Distance Array / Map**	Shortest path
Store indegree	**Array / Map**	Topological Sort

5 ALGORITHM CHOOSER AT A GLANCE

DFS
(Depth First Search)

USE WHEN
- Explore all nodes
- Find components
- Backtracking
- Cycle detection (directed)
- Topological Sort (DFS)

TIME O(V + E) **SPACE** O(V) Recursion

BFS
(Breadth First Search)

USE WHEN
- Shortest path (unweighted)
- Level order traversal
- Minimum steps / levels
- Find components

TIME O(V + E) **SPACE** O(V) Queue

TOPOLOGICAL SORT
(Kahn's BFS / DFS)

USE WHEN
- DAG ordering
- Course schedule
- Build order
- Task dependencies

TIME O(V + E) **SPACE** O(V)

DIJKSTRA
(Min Heap)

USE WHEN
- Shortest path
- Weighted graph
- Positive weights
- Single source

TIME O((V + E) log V) **SPACE** O(V)

UNION FIND (DSU)
(Disjoint Set Union)

USE WHEN
- Connectivity check
- Components count
- Merge groups
- Dynamic connectivity

TIME O(α(V)) per op **SPACE** O(V)

GRID DFS / BFS
(Matrix Problems)

USE WHEN
- Islands / regions
- Flood fill
- Surrounded regions
- Shortest path in grid

TIME O(M * N) **SPACE** O(M * N)

6 FAANG GRAPH SOLVING PIPELINE (Follow this every time)

1. READ & UNDERSTAND
- Read carefully
- Identify what is asked
- List entities & constraints

→

2. MODEL AS GRAPH
- Define Nodes
- Define Edges
- Decide direction & weight

→

3. CHOOSE REPRESENTATION
- Adjacency List / Matrix
- List preferred (default)
- Consider constraints

→

4. PICK HELPER STRUCTURES
- Visited, Queue, Stack
- Dist, Parent, Indegree
- DSU, PQ (if needed)

→

5. CHOOSE ALGORITHM
- DFS / BFS / Topo / Dijkstra / DSU
- Match with pattern

→

6. CODE & TEST
- Implement clean code
- Dry run small cases
- Handle edge cases

→

7. OPTIMIZE & FINALIZE
- Check complexity
- Optimize if needed
- Submit with confidence

7 KEY NOTATIONS

V = Number of Vertices (Nodes)

E = Number of Edges

O(V) = Linear in vertices

O(E) = Linear in edges

α(V) = Inverse Ackermann (≈ constant)

8 COMPLEXITY SNAPSHOT

Algorithm	Time Complexity	Space Complexity
DFS	O(V + E)	O(V)
BFS	O(V + E)	O(V)
Topo Sort (Kahn's BFS)	O(V + E)	O(V)
Topo Sort (DFS)	O(V + E)	O(V)
Dijkstra (Min Heap)	O((V + E) log V)	O(V)
Union Find (DSU)	O(α(V)) per op	O(V)
Grid DFS / BFS	O(M * N)	O(M * N)

9 BIGGEST INTERVIEW PITFALLS

- ✗ Not marking visited → infinite loop
- ✗ Wrong data structure (Stack vs Queue vs PQ)
- ✗ Index out of bounds in grid
- ✗ Forgetting disconnected components
- ✗ Ignoring edge direction in directed graph
- ✗ Topo sort: not all nodes processed (cycle)
- ✗ Dijkstra with negative weights
- ✗ Union Find without path compression / rank
- ✗ Modifying list while iterating

10 REMEMBER THIS!

- 💡 Pattern recognition is king.
- 💡 Draw the graph (mentally or on paper).
- 💡 Choose the right tool.
- 💡 Simplicity beats cleverness.
- 💡 Write clean & modular code.
- 💡 Test all cases.
- 💡 Stay calm. Think. Solve. Conquer.

YOU GOT THIS! CRACK FAANG! 🚀

⭐ **THINK LIKE AN ENGINEER. SOLVE LIKE A MASTER. CRACK FAANG! 💪**

GRAPH FINAL 5-MINUTE REVISION

THE LAST PAGE YOU READ BEFORE THE INTERVIEW
(FAANG Cheat Sheet)

PAGE 18B

1 AHA MOMENTS - REMEMBER THESE!

💡 **Graph = Connections between entities.**
Node = Entity, Edge = Relationship.

💡 **Most problems are pattern based.**
Recognize pattern → Select algorithm.

💡 **Representation matters!**
Adj List for sparse, Matrix for dense / fast edge check.

💡 **Think in terms of traversal.**
Explore, Order, Shortest Path, Connectivity.

💡 **Weight decides your path.**
Unweighted → BFS, Weighted → Dijkstra.

💡 **Dependencies need order.**
Use Topological Sort for DAG.

💡 **Groups & Merges?**
Use Union Find (DSU).

💡 **In grid problems → it is still a graph!**
Cells are nodes, moves are edges.

2 PATTERN RECOGNITION MAP

PROBLEM CLUE	THINK	USE
2D cells, directions, islands, regions, flood	Grid Graph	BFS / DFS (Grid)
Courses, tasks, prerequisites	Dependencies / Order	Topological Sort
Friends, accounts, network, social	Connectivity / Components	DFS / BFS / Union Find
Flights, routes, maps, cities	Shortest Path	BFS (unweighted) / Dijkstra (weighted)
Water flow, spread, rotting oranges	Multi-source BFS	BFS (Multi-source)
Provinces, redundant connection	Connected Components	Union Find (DSU)
Find order, sequence, valid ordering	Topological Ordering	Topological Sort
Cheapest path, min cost	Minimum Cost Path	Dijkstra (Min Heap)

3 ALGORITHM SELECTION CHEAT SHEET

graph TD Start[Given a Graph Problem] --> W{Weighted Edges?} W -- NO --> SP1{Need Shortest Path?} W -- YES --> SP2{Need Shortest Path?} SP1 -- YES --> BFS[BFS Unweighted] SP1 -- NO --> Ord1{Need Ordering Dependencies?} Ord1 -- YES --> Topo1[Topological Sort] Ord1 -- NO --> Conn{Need Connectivity / Components?} Conn -- YES --> UF[Union Find DSU] Conn -- YES --> DFS[DFS / BFS Connected Components] SP2 -- YES --> Dijk[Dijkstra Min Heap] SP2 -- NO --> DAG{Is Graph a DAG?} DAG -- YES --> Topo2[Topological Sort] DAG -- NO --> Spec[Special Case Think & Optimize]

4 TEMPLATE RECALL (WHAT TO WRITE)

Algorithm	What to Write First
DFS	Visited array + dfs(node)
BFS	Queue + visited + bfs(start)
Topological Sort	Indegree (or DFS stack)
Dijkstra	MinHeap + dist[ ]
Union Find (DSU)	parent[ ] + rank/size[ ]
Grid DFS/BFS	Directions + bounds check

💡 **Tip: Don't memorize code.** Memorize the skeleton & modify.

5 COMPLEXITY QUICK RECALL

Algorithm	Time Complexity	Space Complexity
DFS	O(V + E)	O(V)
BFS	O(V + E)	O(V)
Topological Sort (BFS)	O(V + E)	O(V)
Topological Sort (DFS)	O(V + E)	O(V)
Dijkstra (Min Heap)	O((V + E) log V)	O(V)
Union Find (DSU)	O(α(V)) per op	O(V)
Grid DFS / BFS	O(M * N)	O(M * N)

💡 α(V) is inverse Ackermann function (≈ constant)

6 COMMON BUGS (DON'T LOSE MARKS!)

❌ Not marking visited at the right time.
❌ Forgetting to handle disconnected components.
❌ Wrong data structure (Stack vs Queue vs PQ).
❌ Index out of bounds in grid problems.
❌ Ignoring edge direction in directed graph.
❌ Topo sort: not checking all nodes processed.
❌ Dijkstra: not skipping outdated entries from PQ.
❌ Union Find: not using path compression / rank.
❌ Off by one errors in loops and indices.
❌ Modifying list while iterating.

💡 Avoid these = Easy AC + Happy Interviewer!

7 DATA STRUCTURES QUICK RECALL

Data Structure	Use For
Array / List	Adjacency List, Visited, Dist
Queue (FIFO)	BFS, Level Order, Multi-source BFS
Stack / Recursion	DFS, Topo Sort (DFS)
Priority Queue (Min Heap)	Dijkstra, Prim's MST
Map / HashMap	Graph with labels, Count
Set	Visited, Unique nodes
Union Find (DSU)	Connectivity, Components

9 MINI CHECKLIST BEFORE SUBMIT

- [x] Have I understood the problem & constraints?
- [x] Have I identified the graph pattern?
- [x] Have I chosen correct representation?
- [x] Have I selected the right algorithm?
- [x] Have I handled all edge cases?
- [x] Have I marked visited properly?
- [x] Have I checked time & space complexity?
- [x] Have I tested with small examples?
- [x] Have I validated the output format?
- [x] Clean code + Good variable names?

8 RETURN TYPES - THINK FIRST

Return Type	Use When
boolean	Existence, Valid / Invalid, Possible / Not Possible
int	Count, Number of components, Steps, Levels
int[ ]	Distances, Parents, Order
List<Integer>	Order, Path, Component
List<List<Integer>>	All paths, Adjacency list, Components
Node	Clone graph, Any node return
void	Modify in-place, Just traversal

📖 Read return type first. It reveals the problem intent.

10 FAANG GRAPH SOLVING PIPELINE (REPEAT EVERY TIME)

1. READ & UNDERSTAND
- Understand problem
- List entities, relations
- Note constraints

➔

2. MODEL AS GRAPH
- Define nodes
- Define edges
- Directed / Undirected
- Weighted / Unweighted

➔

3. CHOOSE REPRESENTATION
- Adj List (sparse)
- Adj Matrix (dense / fast lookup)

➔

4. PICK HELPER STRUCTURES
- Visited, Queue, Stack, Heap, Map, DSU, Indegree, Dist

➔

5. CHOOSE ALGORITHM
- DFS / BFS / Topo / Dijkstra / DSU / Grid BFS/DFS

➔

6. CODE & TEST
- Write clean code
- Dry run small cases
- Handle edge cases

➔

7. OPTIMIZE & SUBMIT
- Check complexity
- Optimize if needed
- Submit with confidence

🏆 GOLDEN RULE: PATTERN RECOGNITION + RIGHT TOOL + CLEAN CODE = FAANG SUCCESS!

⭐ THINK LIKE AN ENGINEER. SOLVE LIKE A MASTER. CRACK FAANG! 🚀